
Docker Learning Course

PXL Hogeschool - Systems Advanced

December 19, 2025

Contents

1	Module: Docker Container Management	1
1.1	Creating and Running Containers	2
1.1.1	Container Basics	2
1.1.2	The docker run Command	2
1.2	Managing Containers	3
1.2.1	Listing Containers	3
1.2.2	Stopping and Removing Containers	4
1.2.3	Container Identification	5
1.3	Passing Arguments to Containers	5
1.3.1	Command Arguments	5
1.3.2	Automatically Removing Containers	6
1.4	Environment Variables	6
1.4.1	Setting Variables	6
1.4.2	Configuring Applications	6
1.5	Interacting with Running Containers	6
1.5.1	Running Commands in Containers	6
1.5.2	Inspecting Containers	7
1.5.3	Viewing Logs	8
1.5.4	Monitoring Resource Usage	8
2	Working with Docker Images	11
2.1	Docker Hub: The Built-in Public Image Registry	11
2.2	Image Tags and Configuration	12
2.3	Environment Variables	12
2.4	Pulling and Running Docker Images	13
2.5	Inspecting Images	14
2.6	Managing Local Images	15
2.7	Understanding Container Image Layers	16
2.8	Creating Custom Images with docker commit	19
2.9	Exporting and Importing Images	20
2.10	Exercises	21
3	Networking and Data Persistence	23
3.1	Understanding Docker Port Mapping	23
3.1.1	Concept	23
3.1.2	Example	24
3.2	Integrating Containers with Data Persistence	24
3.2.1	Lifecycle	25

3.3	Using Docker Volumes for Data Persistence	25
3.3.1	Purpose	25
3.3.2	Key Commands	25
3.3.3	Example	26
3.4	Using Bind Mounts for Data Persistence	27
3.4.1	Concept	27
3.4.2	Syntax	27
3.4.3	Example	27
3.5	Example: A Persistent MySQL Database	28
3.5.1	Step 1: Create the volume	28
3.5.2	Step 2: Run MySQL container	28
3.6	Exercises	29
4	Secure Shell (SSH)	31
4.1	Introduction	32
4.2	What is SSH?	32
4.2.1	Why SSH Exists	33
4.3	Encryption	33
4.3.1	Asymmetric encryption (RSA, ECDSA, ED25519)	34
4.3.2	Symmetric encryption (AES, ChaCha20)	34
4.4	How SSH Connects	34
4.5	SSH Clients and Servers	34
4.5.1	SSH Client	34
4.5.2	SSH Server	34
4.6	OpenSSH	34
4.7	OpenSSH Client	35
4.8	OpenSSH Server	35
4.8.1	Securing SSH Key Pair	36
4.9	Communication Flow	37
4.10	Host Keys, Fingerprints	37
4.11	SSH connection types	37
4.11.1	SSH connection with password	37
4.11.2	SSH connection with PKI	39
4.12	SSH-Agent	42
4.12.1	Store the passphrase	42
4.12.2	SSH-Agent on Windows	43
4.13	Commands over SSH	44
4.13.1	Copy files over SSH	44
4.14	Troubleshooting	45
4.14.1	Connection Debugging	45
4.14.2	Cleaning Up Known Hosts	45
4.15	Lab Environment (Docker-Based)	45
4.15.1	Architecture	45
4.15.2	Containers	46

5	Docker Build and Dockerfile - Comprehensive Guide	47
5.1	Architecture	47
5.2	Dockerfile	48
5.2.1	Quick Reference Example	48
5.3	Base Images	48
5.3.1	Common Base Image Comparison	49
5.4	Layering	49
5.4.1	Layer Caching Strategy	49
5.5	Core Instructions	50
5.5.1	FROM	50
5.5.2	WORKDIR	50
5.5.3	COPY	51
5.5.4	ADD	51
5.5.5	RUN	52
5.5.6	ENTRYPOINT vs CMD	53
5.5.7	EXPOSE	55
5.5.8	ENV	56
5.6	Multi-Stage Builds	57
5.6.1	Advanced Multi-Stage Examples	58
5.7	Efficient Dockerfiles	59
5.7.1	Minimizing Layers	59
5.7.2	Optimizing Build Context	59
5.7.3	Using Build Arguments	60
5.8	Security	61
5.8.1	Non-root User	61
6	WORKDIR deep dive	65
6.1	Syntax	65
6.1.1	What WORKDIR Affects	65
6.1.2	Directory Creation Behavior	65
6.1.3	Relative vs Absolute Paths	65
6.1.4	Common Mistakes and How to Avoid Them	66
6.1.5	Using Environment Variables with WORKDIR	67
6.1.6	WORKDIR and User Permissions	68
6.1.7	WORKDIR in Multi-Stage Builds	68
6.1.8	Best Practices Summary	68
6.1.9	Default Working Directory	69
6.1.10	Runtime Override	69
6.1.11	Debugging WORKDIR Issues	69
7	COPY deep dive	71
7.1	Syntax	71
7.2	Understanding Build Context	71
7.3	Source Path Behavior	71
7.3.1	Leading Slashes and Parent Navigation	71

7.3.2	Trailing Slashes in Source	71
7.4	Destination Path Rules	72
7.4.1	The Trailing Slash Significance	72
7.4.2	Multiple Sources Require Trailing Slash	72
7.5	Directory Copying Behavior	72
7.6	File Overwriting Behavior	73
7.7	Common Mistakes and Solutions	73
7.7.1	Mistake 0: Paths with Spaces	73
7.7.2	Mistake 1: Missing Trailing Slash with Multiple Files	73
7.7.3	Mistake 2: Expecting Directory Name to be Preserved	73
7.7.4	Mistake 3: Using Absolute Paths from Host	74
7.7.5	Mistake 4: Build Context Confusion	74
7.7.6	Mistake 5: Inefficient Layer Ordering	74
7.7.7	Mistake 6: Wildcards and Hidden Files	75
7.8	Advanced COPY Usage	75
7.8.1	Ownership and Permissions	75
7.8.2	Copying from Multi-Stage Builds	75
7.8.3	Using .dockerignore	75
7.9	Platform-Specific Considerations	76
7.9.1	Building Linux Containers on Different Platforms	76
7.9.2	File Permissions and Line Endings	76
7.9.3	Using .dockerignore Cross-Platform	76
7.10	Best Practices Summary	76
7.11	Debugging COPY Issues	77
7.12	Quick Reference	77
8	docker storage	79
8.1	Introduction	79
8.2	Mount types	79
8.3	Mount type details	79
8.4	Volume use cases	80
8.5	bind mount use cases	80
8.6	tmpfs mount use cases	80
8.7	Tips for using bind mounts or volumes	80
8.8	Volumes	80
8.8.1	Choose the -v or --mount flag (again)	81
8.8.2	Create and manage volumes	82
8.8.3	Start a container with a volume	82
8.8.4	Populate a volume using a container	83
8.8.5	Use a read-only volume	84
8.9	Bind mounts	85
8.9.1	Choose the -v or --mount flag	85
8.9.2	Differences between -v and --mount behavior	85
8.9.3	Start a container with a bind mount	86
8.9.4	Path Conversion on Windows	86

8.9.5	Mount into a Non-Empty Directory	86
8.9.6	Use a Read-Only Bind Mount	87
8.10	tmpfs Mounts	87
8.10.1	Limitations of tmpfs Mounts	87
8.10.2	The <code>-tmpfs</code> or <code>-mount</code> Flag	88
8.10.3	Specify tmpfs Options	88
9	docker networking	89
9.1	Introduction	89
9.2	Published ports	89
9.3	IP address and hostname	90
9.4	DNS services	90
9.4.1	Nameservers with IPv6 addresses	90
9.4.2	Custom hosts	91
9.5	Network drivers	91
9.6	Bridge	91
9.6.1	user-defined bridges vs default bridge	91
9.6.2	Manage a user-defined bridge	92
9.6.3	Connect a container to a user-defined bridge	92
9.6.4	Disconnect a container from a user-defined bridge	92
9.6.5	Use the default bridge network	92
9.7	Host	92
9.8	IPvlan	93
9.9	Macvlan	93
9.10	None	93
10	Docker Compose	95
10.1	Introduction	97
10.2	Compose Specification and Compose File Essentials	98
10.2.1	The Compose File	98
10.2.2	Compose File Versions	98
10.2.3	Basic Structure	98
10.2.4	Services Section	99
10.2.5	Example Compose File	99
10.2.6	Multiple Compose Files and Overrides	99
10.3	Docker Compose CLI	100
10.3.1	Starting and Stopping Services	100
10.3.2	Start services	100
10.3.3	Stop services	100
10.3.4	Start and run in detached mode	100
10.3.5	Managing Containers	100
10.3.6	Stopping and removing containers, networks, and volumes	100
10.3.7	Pausing services	100
10.3.8	Unpausing services	100
10.3.9	Logs and Monitoring	101

10.3.10 Viewing logs	101
10.3.11 Following log output	101
10.3.12 Scaling and Updating Services	101
10.3.13 Scaling a service	101
10.3.14 Rebuilding and restarting a service	101
10.3.15 Advanced Management	101
10.3.16 Removing stopped containers	101
10.3.17 Running a one-off command	101
10.3.18 Viewing the status of services	101
10.3.19 Pulling service images	102
10.4 Service Definition	102
10.4.1 Defining a Service	102
10.4.2 Service Configuration Examples	102
10.4.3 Command	102
10.4.4 Ports and Networking	103
10.4.5 Environment Variables	104
10.4.6 Profiles	104
10.5 Networks and Volumes	104
10.5.1 Networks in Compose	104
10.5.2 Volumes for Persistent Data	105
10.5.3 Combining Networks and Volumes	105
10.5.4 Networking Best Practices	106
10.6 Network Types	106
10.6.1 Overview of Docker Network Types	106
10.6.2 Configuring Networks in Compose	106
10.6.3 Network Scopes and Drivers	106
10.6.4 Network Options and IPAM Configuration	107
10.7 Service Discovery	107
10.7.1 Understanding Service Discovery	107
10.7.2 DNS-Based Service Discovery	107
10.7.3 Service Discovery Across Multiple Networks	107
10.7.4 Service Discovery Example	107
10.8 Network Aliases	108
10.9 Host Networking	109
10.10 Volumes	109
10.10.1 1. Automatically Created Volumes	109
10.10.2 2. Bind Mounts	109
10.10.3 Volume Persistence	110
10.11 Environment Variables	110
10.11.1 Using .env File	110
10.11.2 environment Attribute in Compose File	111
10.11.3 env_file Attribute in Compose File	111
10.11.4 Substituting via Shell	111
10.11.5 Using –env-file	112
10.12 Restarting services	112

10.13 Startup / Shutdown Order	113
10.13.1 Simple example	113
10.13.2 Example using service_started and service_healthy	114
10.14 Compose build	114
10.14.1 Build Context	115
10.15 Compose Secrets (AVOID!)	116
10.15.1 Why avoid this solution?	116
10.15.2 How should I inject secrets securely into my ci/cd pipelines?	116
10.15.3 Compose secrets explained	117

1 Module: Docker Container Management

- Module: Docker Container Management
 - Creating and Running Containers
 - * Container Basics
 - * The `docker run` Command
 - Managing Containers
 - * Listing Containers
 - * Stopping and Removing Containers
 - * Container Identification
 - Passing Arguments to Containers
 - * Command Arguments
 - * Automatically Removing Containers
 - Environment Variables
 - * Setting Variables
 - * Configuring Applications
 - Interacting with Running Containers
 - * Running Commands in Containers
 - * Inspecting Containers
 - * Viewing Logs
 - * Monitoring Resource Usage

Docker provides a platform to build, ship, and run applications in containers. Containers encapsulate software in a complete filesystem that contains everything needed to run: code, runtime, system tools, and libraries.

1.1 Creating and Running Containers

1.1.1 Container Basics

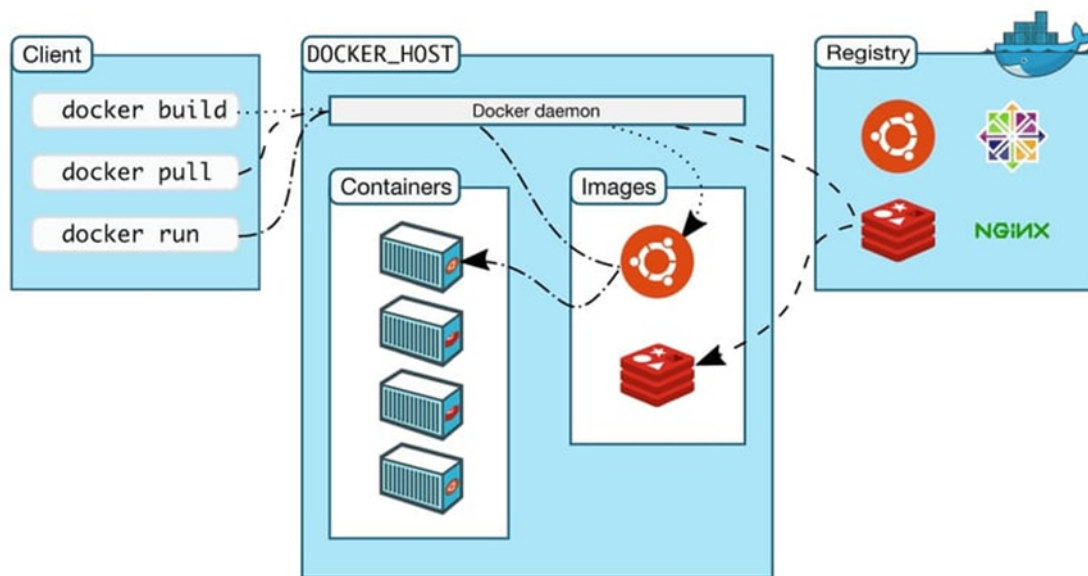


Figure 1.1: docker overview

Each Docker container operates in isolation:

- It has its own filesystem, separate from the host.
- It has distinct network capabilities.
- It maintains an independent process tree.

Docker uses the `docker run` command to combine three actions:

1. Pull the image (if not already available).
2. Create the container.
3. Start the container.

1.1.2 The `docker run` Command

```
docker run [OPTIONS] IMAGE [ARG...]
```

Key options:

- `-d` — Run container in detached mode (in the background).
- `-p [host_port]:[container_port]` — Map a host port to a container port.
- `--name [container_name]` — Assign a name to the container.

Example: Run an Nginx web server in detached mode, accessible at <http://localhost:8080>:

```
docker run --name mynginx -d -p 8080:80 nginx
```

Output:

```

❏ docker run --name mynginx -d -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
e2f8e296d9df: Pull complete
9def903993e4: Pull complete
320b0949be89: Pull complete
266626526d42: Pull complete
52bc359bcbd7: Pull complete
d7ecdcd7702a: Pull complete
d921c57c6a81: Pull complete
Digest: sha256:1beed3ca46acebe9d3fb62e9067f03d05d5bfa97a00f30938a0a3580563272ad
Status: Downloaded newer image for nginx:latest
a0726af08246726eda37a11212e2f46d07b4d3c9b15ea62328bfaeccf4d76e09

```

Open localhost:8080 in your browser.

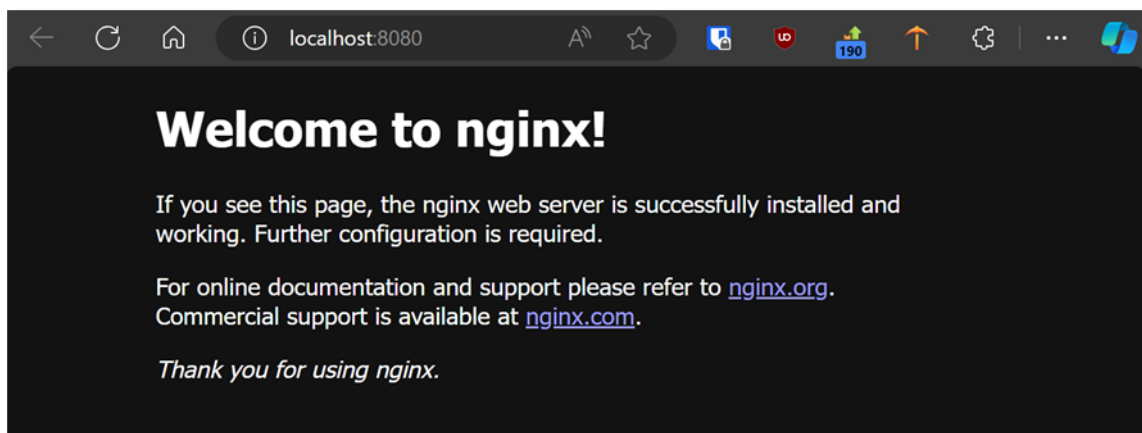


Figure 1.2: nginx-example

1.2 Managing Containers

1.2.1 Listing Containers

To view running containers:

```
docker container ls
```

Output:

```

❏ docker container ls
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
a9da8c7226ff   nginx    "/docker-entrypoint..." 27 seconds ago Up 25 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp mynginx2

```

Lists all running containers, including stopped containers:

```
docker container ls -a
```

Output:

```

❏
docker container ls -a
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS      PORTS                               NAMES
a9da8c7226ff   nginx    "/docker-entrypoint..." 31 seconds ago  Up 30 seconds  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  mynginx2
a0726af08246   nginx    "/docker-entrypoint..." 3 minutes ago   Exited (0) 44 seconds  mynginx

```

Equivalent legacy command: `docker ps`

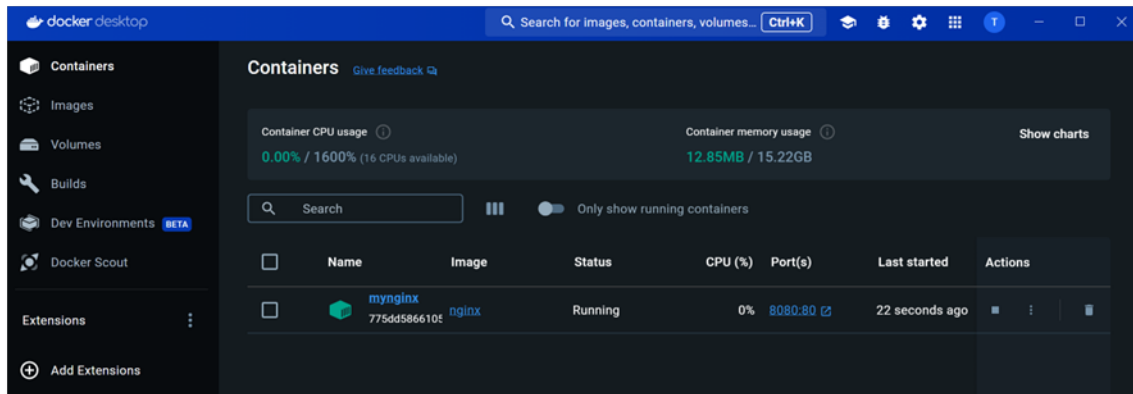


Figure 1.3: docker desktop ls

1.2.2 Stopping and Removing Containers

```

docker container stop <container-id>
docker container rm <container-id>

```

Use `-f` to force removal of a running container.

Example:

```

docker container stop mynginx
docker container rm mynginx

```

Output:

```

❏
docker container stop mynginx
mynginx❏

docker container rm mynginx
mynginx❏

docker container ls
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS      PORTS                               NAMES

```

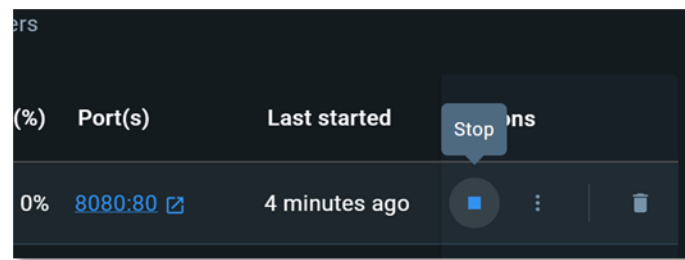


Figure 1.4: docker desktop rm

1.2.3 Container Identification

Containers can be identified by:

- **Long UUID:** 5e7ae9fbf9ee636eeea383ace3dfc7eb390dac013f13ae91beb72f98952a9483
- **Short UUID:** 5e7ae9fbf9ee
- **Name:** mynginx

If no name is provided, Docker assigns a random one (e.g. dreamy_yonath).

Example:

```

❏ docker run --name mynginx -d -p 8080:80 nginx
a4c47fd7eb8180d7e61a7e45d17dbcc1a11e49f3f46806cd18f1bfe3e02b0383❏

docker container ls
CONTAINER ID   IMAGE     COMMAND                  NAMES                  CREATED        STATUS        PORTS
a4c47fd7eb81   nginx    "/docker-entrypoint..." 5 seconds ago        Up 5 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp mynginx

```

1.3 Passing Arguments to Containers

1.3.1 Command Arguments

Arguments following the image name are passed to the container process.

```
docker run ubuntu echo "Hello from Docker"
```

This runs `echo` inside the Ubuntu container, then exits.

For interactive commands, use:

```
docker run -it ubuntu bash
```

1.3.2 Automatically Removing Containers

Use `--rm` to delete a container automatically after it exits.

```
docker run --rm python python -c "print('Hello from Python inside Docker!')"  
docker run --rm curlimages/curl curl https://www.example.com  
docker run --rm instrumentisto/nmap -p 80,443 pxl.be
```

1.4 Environment Variables

1.4.1 Setting Variables

```
docker run -it -e "USERNAME=pxl" ubuntu bash
```

Inside the container, the variable `USERNAME` equals `pxl`.

1.4.2 Configuring Applications

Passing environment variables is the main configuration method for many containerized services.

Example - MySQL container:

```
docker run -d --name mymysql -e MYSQL_ROOT_PASSWORD=mysecret -e MYSQL_DATABASE=mydb  
mysql
```

This creates a MySQL container with a root password and a default database.

Verify:

```
docker exec -it mymysql mysql -uroot -pmysecret  
SHOW DATABASES;  
exit;
```

1.5 Interacting with Running Containers

1.5.1 Running Commands in Containers

`docker exec` allows running additional commands in an active container.

```
docker exec [OPTIONS] CONTAINER COMMAND [ARG...]
```

Example:

```
docker exec -it mynginx /bin/bash
```

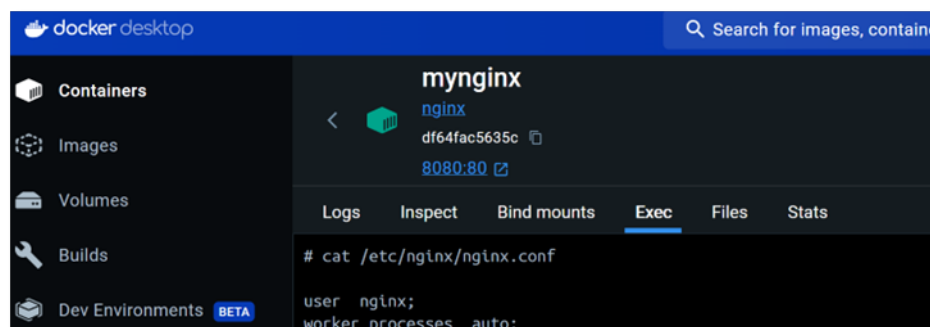


Figure 1.5: docker desktop exec

1.5.2 Inspecting Containers

`docker inspect` returns detailed JSON information about Docker objects.

```
docker inspect mymysql
```

Includes configuration, environment variables, and network settings.

Platform examples:

- PowerShell:

```
(docker inspect mymysql | ConvertFrom-Json).Config.Env
```

- Bash:

```
docker inspect mymysql | jq '.[0].Config.Env'
```

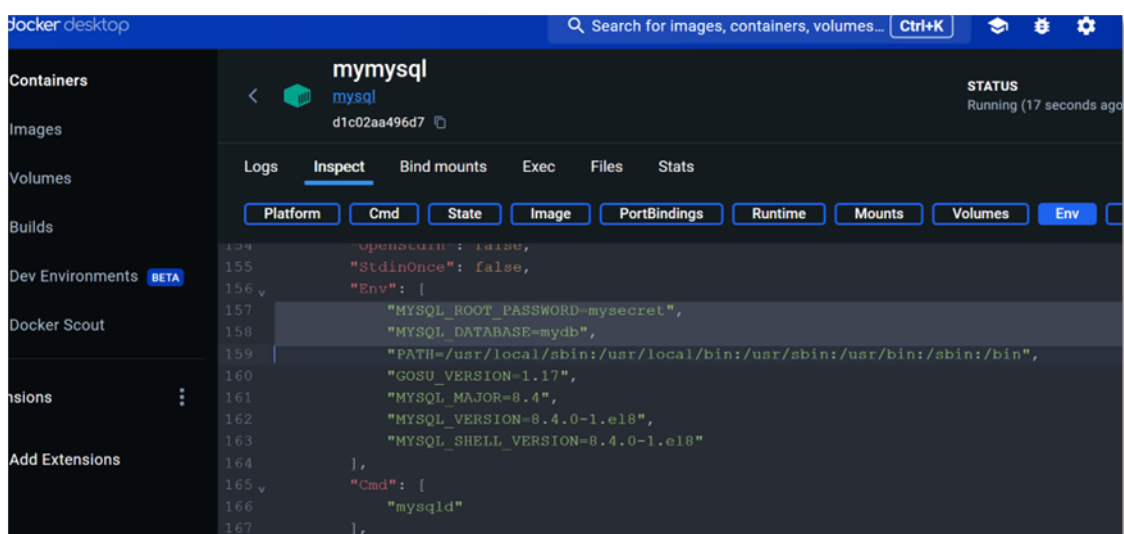


Figure 1.6: docker desktop inspect

1.5.3 Viewing Logs

```
docker logs CONTAINER
```

Retrieves output from a container's `stdout` and `stderr`. Example:

```
docker logs mymysql
```

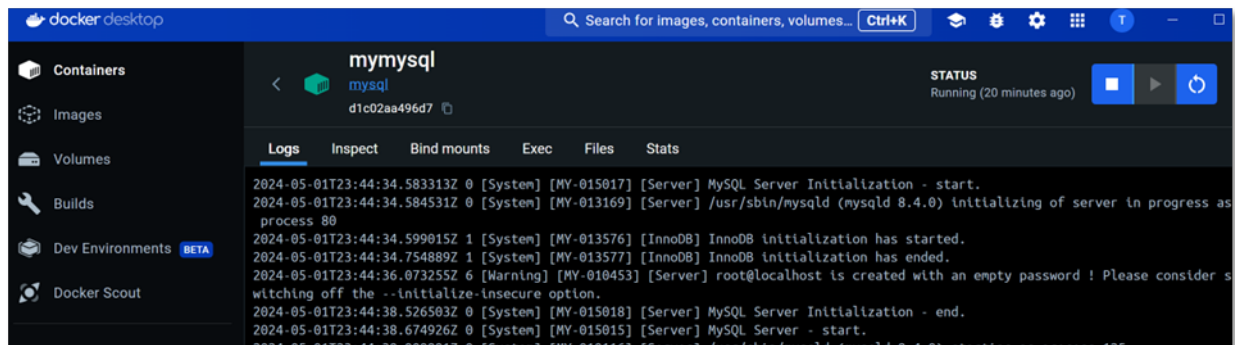


Figure 1.7: docker desktop logs

1.5.4 Monitoring Resource Usage

```
docker stats [CONTAINER]
```

Displays real-time CPU, memory, and network usage for all or specific containers. Example:

```
docker stats mymysql
```

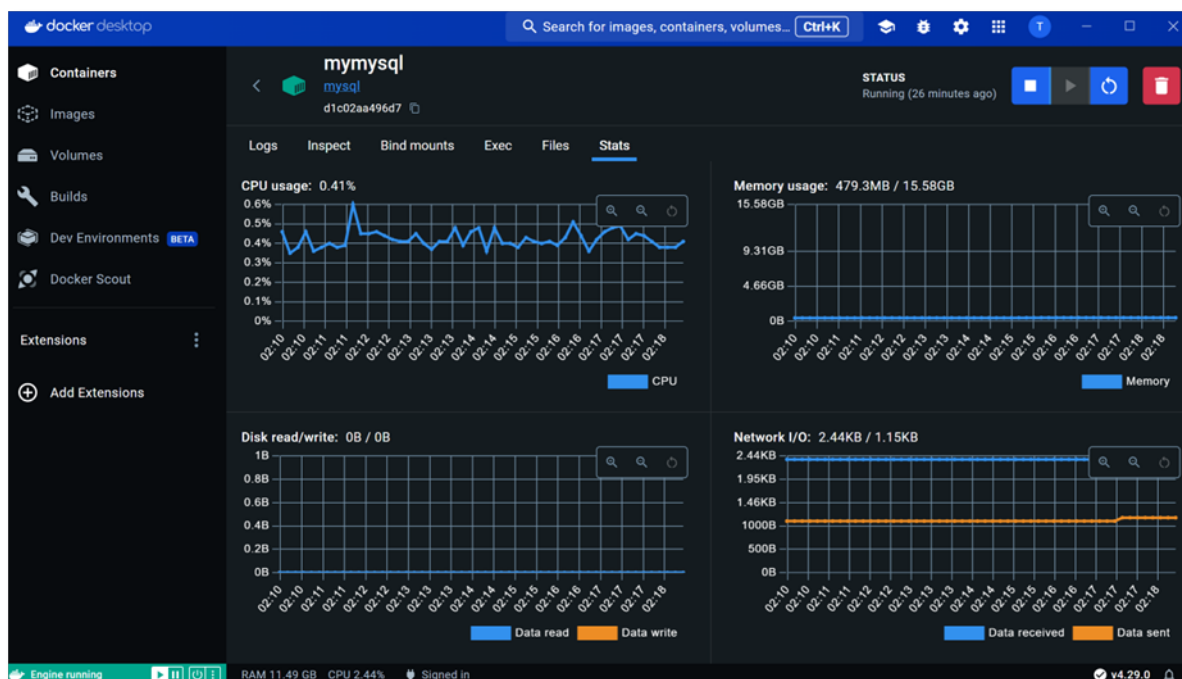


Figure 1.8: docker desktop stats

2 Working with Docker Images

- [Docker Hub: The Built-in Public Image Registry](#)
- [Image Tags and Configuration](#)
- [Environment Variables](#)
- [Pulling and Running Docker Images](#)
- [Inspecting Images](#)
- [Managing Local Images](#)
- [Understanding Container Image Layers](#)
- [Creating Custom Images with `docker commit`](#)
- [Exporting and Importing Images](#)
- [Exercises](#)

2.1 Docker Hub: The Built-in Public Image Registry

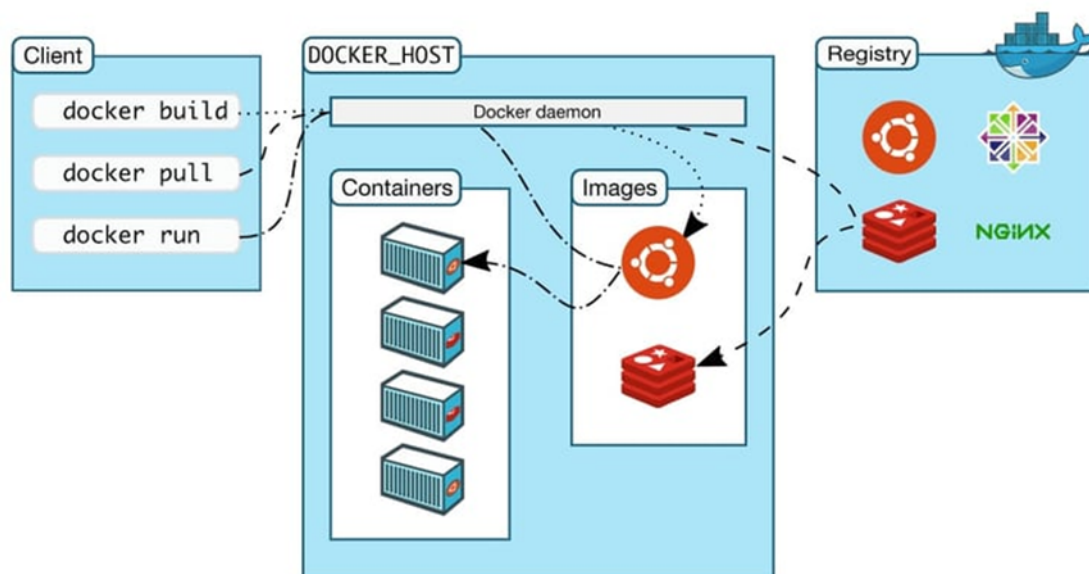


Figure 2.1: Docker overview

Docker Hub is a central repository for Docker images.

It contains a mixture of official and community-contributed images and is integrated with the Docker CLI, which allows you to pull and push images directly without extra configuration.

By default, Docker uses Docker Hub as its image registry, so you do not need to specify it explicitly.

Key features:

- Centralized hosting of public and private images.
- Official images maintained by vendors or the Docker team.
- Community images for various tools and systems.
- Integrated authentication and repository management.
- **Searchable through both the Docker CLI and the web interface.**

Web access: <https://hub.docker.com/>

Other popular registries are also available, such as:

- **GitHub Container Registry (ghcr.io):** `docker pull ghcr.io/owner/image:tag`
 - **Google Container Registry (gcr.io):** `docker pull gcr.io/project/image:tag`
 - **Amazon ECR:** `docker pull <account-id>.dkr.ecr.<region>.amazonaws.com/image:tag`
 - **Quay.io:** `docker pull quay.io/organization/image:tag`
 - **Microsoft Container Registry (mcr.microsoft.com):** `docker pull mcr.microsoft.com/image:tag`
 - **Azure Container Registry:** `docker pull <registry-name>.azurecr.io/image:tag`
-

2.2 Image Tags and Configuration

When you deploy applications such as MySQL with Docker, you often start from an official image.

Each image can have multiple *tags*, representing versions or variants. For example, `mysql:5.7` refers to the MySQL version 5.7 image.

You can specify a tag when pulling an image:

```
docker pull mysql:5.7
```

If no tag is specified, Docker defaults to `latest`.

2.3 Environment Variables

Images are configured through **environment variables**. These variables are listed on the Docker Hub description page for each image. For example, the MySQL image supports environment variables like `MYSQL_ROOT_PASSWORD`, `MYSQL_DATABASE`, and `MYSQL_USER`.

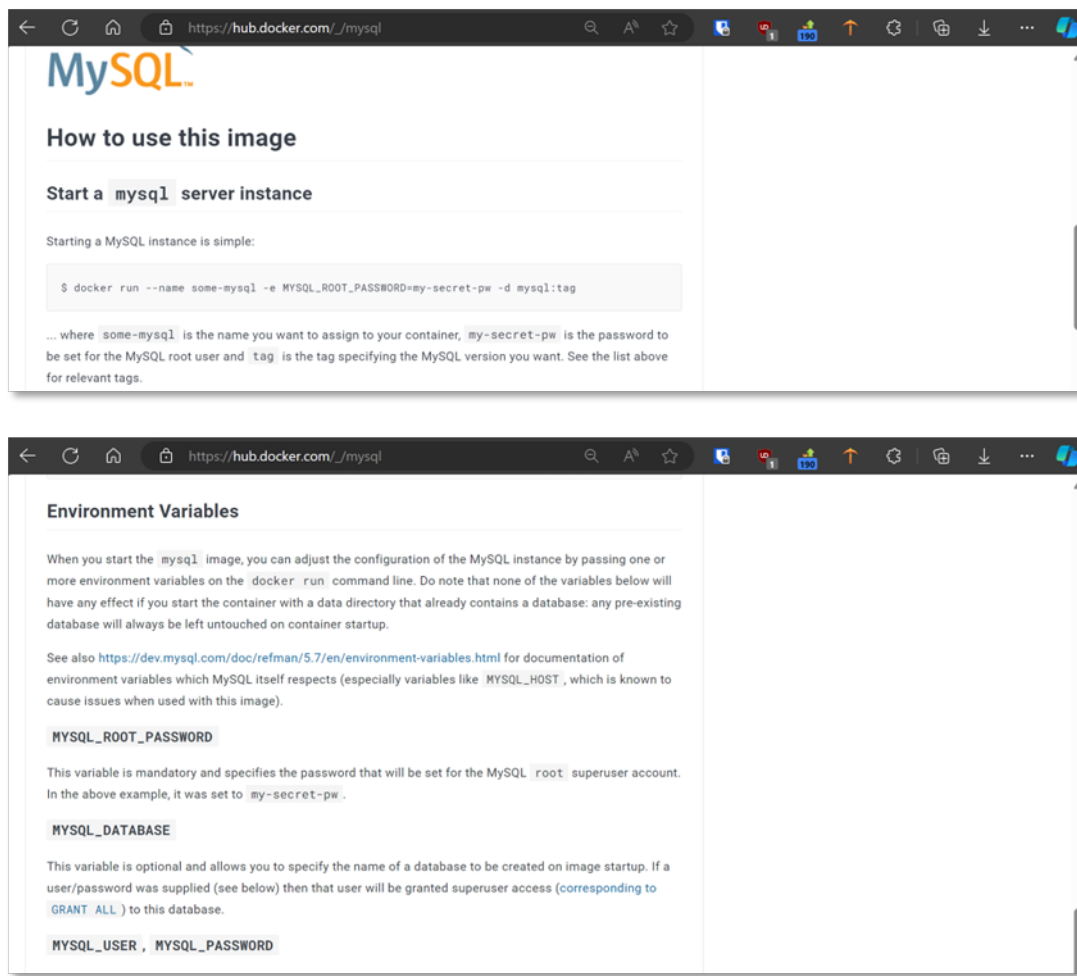


Figure 2.2: Dockerhub MySQL info

2.4 Pulling and Running Docker Images

You can either **pull** an image or **run** it directly. Both commands will automatically download the image if it is not already available locally.

```
docker pull [OPTIONS] IMAGENAME[:TAG]
docker run [OPTIONS] IMAGENAME[:TAG] [ARGS...]
```

If **TAG** is omitted, Docker fetches the **latest** tag.

docker run combines several actions:

1. Pulls the image (if needed)
2. Creates a new container
3. Starts the container

Example: Running a MySQL container with predefined settings:

```
docker run -d --name mymysql -e MYSQL_ROOT_PASSWORD=mysecret -e MYSQL_DATABASE=mydb
mysql:5.7
```

Output:

```
❏
docker run -d --name mymysql -e MYSQL_ROOT_PASSWORD=mysecret -e MYSQL_DATABASE=mydb
mysql:5.7
Unable to find image 'mysql:5.7' locally
5.7: Pulling from library/mysql
68c3898c2015: Pull complete
[...]
ae71319cb779: Pull complete
Digest: sha256:4bc6bc963e6d8443453676cae56536f4b8156d78bae03c0145cbe47c2aad73bb
Status: Downloaded newer image for mysql:5.7
ae024cfb23d4cb31d362bb68982059c5f2a570e2a3c5f505fd175e2018c92cc7
```

Example: Pulling the latest Nginx image:

```
docker pull nginx
```

Output:

```
❏
docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:1beed3ca46acebe9d3fb62e9067f03d05d5bfa97a00f30938a0a3580563272ad
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
```

2.5 Inspecting Images

The `docker inspect` command displays detailed metadata about an image in JSON format.

```
docker inspect mysql:5.7
```

It returns information about environment variables, volumes, network settings, and other configuration details.

Useful commands:

```
docker image ls | grep mysql
docker inspect mysql:5.7
```

Output:

```
docker inspect mysql:5.7
[
  {
    "Id": "sha256:4bc6bc963e6d8443453676cae56536f4b8156d78bae03c0145cbe47c2aad73bb",
    "RepoTags": [
      "mysql:5.7"
    ],
    [...]

    "Env": [
      "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
      "GOSU_VERSION=1.16",
      "MYSQL_MAJOR=5.7",
```

```

        "MYSQL_VERSION=5.7.44-1.el7",
        "MYSQL_SHELL_VERSION=8.0.35-1.el7"
    ],
    "ExposedPorts": {
        "3306/tcp": {},
        "33060/tcp": {}
    }
}

```

Platform-specific JSON parsing examples:

- **PowerShell:**

```
(docker inspect mysql:5.7 | ConvertFrom-Json).Config.Env
```

- **Bash (using jq):**

```
docker inspect mysql:5.7 | jq '[0].Config.Env'
```

2.6 Managing Local Images

You can manage stored images using Docker image management commands.

Action	Command
List local images	<code>docker image ls</code>
Remove a specific image	<code>docker image rm IMAGE</code>
Remove dangling (unused, untagged) images	<code>docker image prune</code>
Remove all unused images	<code>docker image prune -a</code>

Regular cleanup keeps disk usage low and avoids conflicts between outdated images.

When all else fails: `prune system -a -f` removes everything.

```

❏ docker image ls
REPOSITORY TAG IMAGE ID CREATED SIZE
nginx latest 1beed3ca46ac 2 days ago 225MB
ubuntu latest 66460d557b25 5 weeks ago 117MB
mysql 5.7 4bc6bc963e6d 23 months ago 689MB

❏ docker image prune -a
WARNING! This will remove all images without at least one container associated to them.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B

❏ docker system prune -a -f
Deleted Containers:
ae024cfb23d4cb31d362bb68982059c5f2a570e2a3c5f505fd175e2018c92cc7
[...]
a4c47fd7eb8180d7e61a7e45d17dbcc1a11e49f3f46806cd18f1bfe3e02b0383

Deleted Networks:
k3d-k3s-default

```

```
minikube
```

```
Deleted build cache objects:
```

```
y8srhvp3eidmr09ad640id18
```

```
[...]
```

```
u5pq5bnscgpadssll9ynkh5x
```

```
Total reclaimed space: 1.331GB
```

2.7 Understanding Container Image Layers

Docker images are made up of multiple **layers** stacked using the **overlay2 filesystem** (a union filesystem).

Each layer represents a file system change introduced during the image build process.

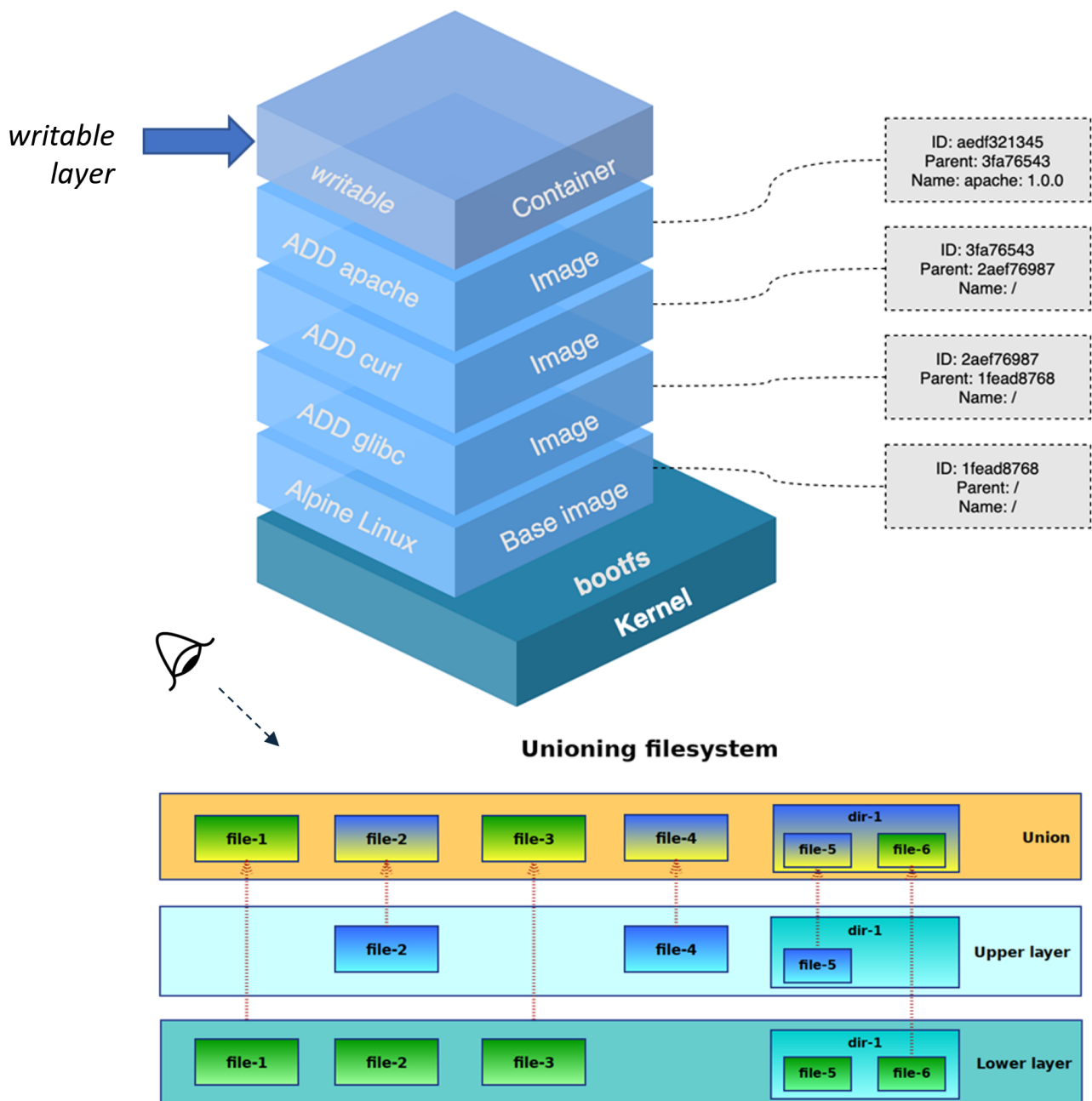


Figure 2.3: docker image layers

Layer hierarchy:

- **Base layer:** Operating system (e.g., Ubuntu, CentOS).
- **Intermediate layers:** Application dependencies and configuration.
- **Top layer:** Latest updates, bug fixes, or specific setup changes.

When a container starts, Docker adds one extra **writable layer** on top of the image layers. This layer captures all runtime changes (new files, modifications, deletions). It disappears when the container is removed unless changes are committed or stored in volumes.

To view image layers:

```
docker history mysql:5.7
```

Output:

❏ docker history mysql:5.7				
IMAGE	COMMENT	CREATED	CREATED BY	SIZE
4bc6bc963e6d	buildkit.dockerfile.v0	23 months ago	CMD ["mysqld"]	0B
<missing>	buildkit.dockerfile.v0	23 months ago	EXPOSE map[3306/tcp:{} 33060/tcp:{}]	0B
<missing>	buildkit.dockerfile.v0	23 months ago	ENTRYPOINT ["docker-entrypoint.sh"]	0B
<missing>	buildkit.dockerfile.v0	23 months ago	RUN /bin/sh -c ln -s usr/local/bin/docker-...en	4.1kB
<missing>	buildkit.dockerfile.v0	23 months ago	COPY docker-entrypoint.sh /usr/local/bin/ # ...	32.8kB
<missing>	buildkit.dockerfile.v0	23 months ago	VOLUME [/var/lib/mysql]	0B
<missing>	buildkit.dockerfile.v0	23 months ago	RUN /bin/sh -c set -eux; yum install -y --...s	298MB
<missing>	buildkit.dockerfile.v0	23 months ago	ENV MYSQL_SHELL_VERSION=8.0.35-1.el7	0B
<missing>	buildkit.dockerfile.v0	23 months ago	RUN /bin/sh -c set -eu; { echo '[mysql-...to	16.4kB
<missing>	buildkit.dockerfile.v0	23 months ago	RUN /bin/sh -c set -eux; yum install -y --...s	79.8MB
<missing>	buildkit.dockerfile.v0	23 months ago	RUN /bin/sh -c set -eu; { echo '[mysql5...7	16.4kB
<missing>	buildkit.dockerfile.v0	23 months ago	ENV MYSQL_VERSION=5.7.44-1.el7	0B
<missing>	buildkit.dockerfile.v0	23 months ago	ENV MYSQL_MAJOR=5.7	0B
<missing>	buildkit.dockerfile.v0	23 months ago	RUN /bin/sh -c set -eux; key='859...BE8D7C586F	20.5kB
<missing>	buildkit.dockerfile.v0	23 months ago	RUN /bin/sh -c set -eux; yum install -y --...s	14.7MB
<missing>	buildkit.dockerfile.v0	23 months ago	RUN /bin/sh -c set -eux; arch="\$(uname -m)..."	2.42MB
<missing>	buildkit.dockerfile.v0	23 months ago	ENV GOSU_VERSION=1.16	0B
<missing>	buildkit.dockerfile.v0	23 months ago	RUN /bin/sh -c set -eux; groupadd --system ...	41kB
<missing>	buildkit.dockerfile.v0	23 months ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	0B
<missing>	buildkit.dockerfile.v0	23 months ago	/bin/sh -c #(nop) ADD file:74...b33794f8e61e810	155MB

output interpretation:

The output provides a detailed listing of the layers that make up the mysql:5.7 Docker image, including the instructions used to create each layer and the sizes of changes introduced at each step:

- **Top Layer:** Executes CMD ["mysqld"] - This is the command that runs by default when the container starts, initializing the MySQL server.
- **Installation of Packages:** Various RUN commands show the installation of MySQL and its dependencies using yum, the package manager, adding substantial size.
- **Environment Variables:** Sets up several environment variables like MYSQL_SHELL_VERSION, MYSQL_VERSION, and MYSQL_MAJOR
- **Base Image Setup:** The bottom-most layer starts from a base image (indicated by a ADD file: command), adding 155MB, which includes the base operating system (Oraclelinux) and essential files.

Each entry in the history lists either a command run during the build (RUN, CMD, ENTRYPOINT) or an operation like setting an environment variable (ENV), adding files (ADD, COPY), or setting up metadata (EXPOSE,

VOLUME). Later we will learn how to create our own images with a config file called Dockerfile.

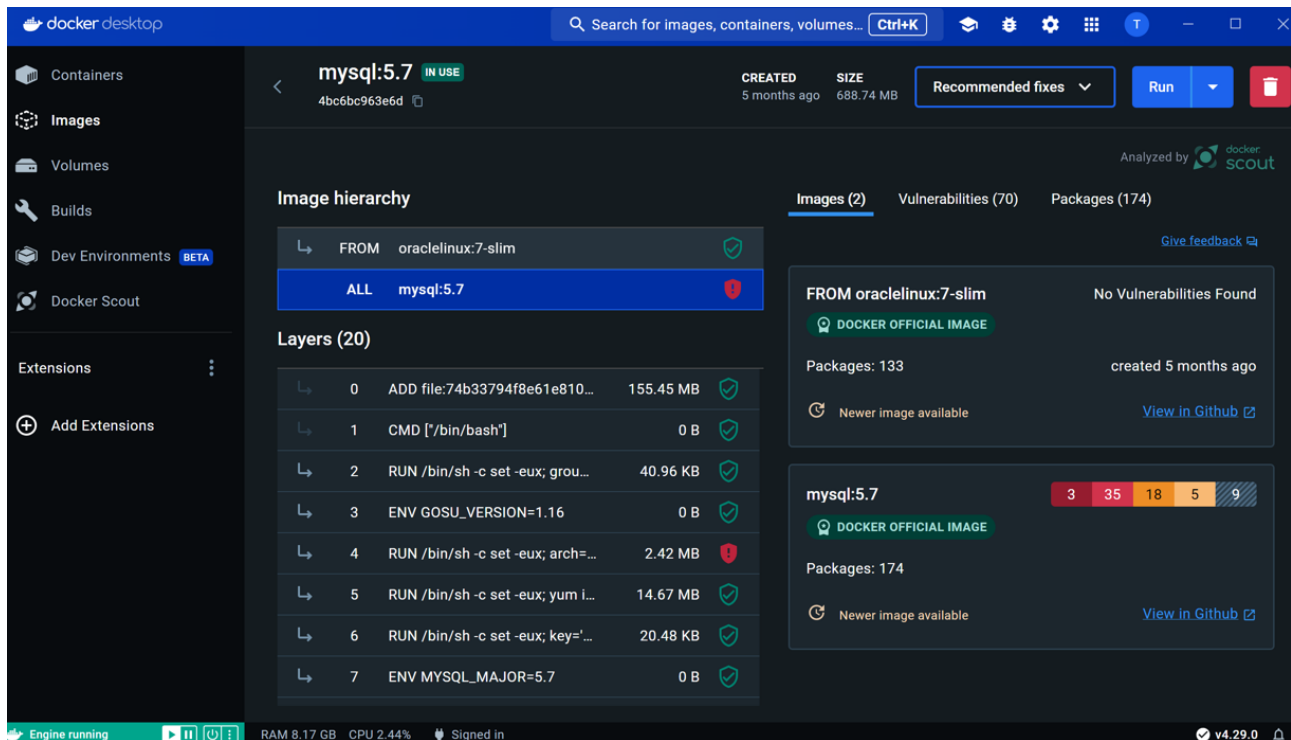


Figure 2.4: docker desktop layers

You can output in JSON format:

```
docker history mysql:5.7 --format json
```

2.8 Creating Custom Images with docker commit

The preferred way to build images is **declaratively**, using a [Dockerfile](#) (covered later). However, it is also possible to create an image manually with `docker commit`.

Example: Create a new image based on `alpine` with `curl` installed.

```
docker run -it --name my-alpine-container alpine /bin/sh
apk update
apk add -q curl
exit
docker commit my-alpine-container my-alpine-image
```

Output:

```
❌
docker run -it --name my-alpine-container alpine /bin/sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
2d35ebdb57d9: Pull complete
Digest: sha256:4b7ce07002c69e8f3d704a9c5d6fd3053be500b7f1c69fc0d80990c2ad8dd412
Status: Downloaded newer image for alpine:latest
```

```

/ # apk update
fetch https://dl-cdn.alpinelinux.org/alpine/v3.22/main/x86_64/APKINDEX.tar.gz
fetch https://dl-cdn.alpinelinux.org/alpine/v3.22/community/x86_64/APKINDEX.tar.gz
v3.22.2-114-g3bc4c29c93b [https://dl-cdn.alpinelinux.org/alpine/v3.22/main]
v3.22.2-133-g54576a3163a [https://dl-cdn.alpinelinux.org/alpine/v3.22/community]
OK: 26320 distinct packages available
/ # apk add -q curl
/ # exit

docker commit my-alpine-container my-alpine-image
sha256:02c6ad9a8f9567b2430b3cd2a2f8efbc7a98a251d4baad15408f92feb061a820

```

Verify:

```

docker image ls
docker run -it my-alpine-image /bin/sh
  curl -v www.pxl.be
  exit
docker history my-alpine-image

```

Output:

```

❏ docker image ls
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
my-alpine-image     latest      02c6ad9a8f95     5 seconds ago   25.7MB
alpine              latest      4b7ce07002c6     4 weeks ago     12.8MB
mysql               5.7         4bc6bc963e6d     23 months ago   689MB

```

```

❏ docker-labs ↵[ main][!]?❏
docker run -it my-alpine-image /bin/sh
/ # curl -v www.pxl.be
* Host www.pxl.be:80 was resolved.
* IPv6: (none)
* IPv4: 193.190.154.242
* Trying 193.190.154.242:80...
* Connected to www.pxl.be (193.190.154.242) port 80
* using HTTP/1.x
> GET / HTTP/1.1
> Host: www.pxl.be
> User-Agent: curl/8.14.1
> Accept: */*
[...]
/ # exit❏

```

```

docker history my-alpine-image
IMAGE          CREATED          CREATED BY          SIZE
COMMENT
02c6ad9a8f95   About a minute ago   /bin/sh             7.88
MB
4b7ce07002c6   4 weeks ago         CMD ["/bin/sh"]      0B
buildkit.dockerfile.v0
<missing>      4 weeks ago         ADD alpine-minirootfs-3.22.2-x86_64.tar.gz .../ 8.99
MB buildkit.dockerfile.v0

```

2.9 Exporting and Importing Images

Images can be exported as tar archives and imported on other hosts. This is useful for backups, air-gapped systems, or image transfer without Docker Hub.

Export an image:

```
docker save -o ubuntu.tar ubuntu:latest
```

Import an image:

```
docker load -i ubuntu.tar
```

Example: pull `busybox:latest` image, export it to a file, remove the image and import it from the file.

```
docker image ls
docker image pull busybox:latest
docker save -o busybox_latest.tar busybox:latest
ls -la busybox_latest.tar
docker image rm busybox:latest
docker load -i busybox_latest.tar
docker image ls
```

Output:

```
❏
docker image ls
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE❏

docker image pull busybox:latest
latest: Pulling from library/busybox
e59838ecfec5: Pull complete
Digest: sha256:e3652a00a2fabd16ce889f0aa32c38eec347b997e73bd09e69c962ec7f8732ee
Status: Downloaded newer image for busybox:latest
docker.io/library/busybox:latest❏

docker save -o busybox_latest.tar busybox:latest❏

ls -la busybox_latest.tar
-a--- 2.2M  6 Nov 10:23 busybox_latest.tar❏

docker image rm busybox:latest
Untagged: busybox:latest
Deleted: sha256:e3652a00a2fabd16ce889f0aa32c38eec347b997e73bd09e69c962ec7f8732ee❏

docker load -i busybox_latest.tar
Loaded image: busybox:latest❏

docker image ls
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
busybox             latest      e3652a00a2fa     13 months ago   6.78MB
```

2.10 Exercises

1. Download a PNG image to your system.
2. Convert it to JPEG using the `ImageMagick` CLI tool inside a Docker container, without installing the tool locally.

3 Networking and Data Persistence

- [Understanding Docker Port Mapping](#)
 - [Concept](#)
 - [Example](#)
- [Integrating Containers with Data Persistence](#)
 - [Lifecycle](#)
- [Using Docker Volumes for Data Persistence](#)
 - [Purpose](#)
 - [Key Commands](#)
 - [Example](#)
- [Using Bind Mounts for Data Persistence](#)
 - [Concept](#)
 - [Syntax](#)
 - [Example](#)
 - ★ [Create a test page](#)
 - ★ [Run container with bind mount](#)
 - ★ [Verify](#)
- [Example: A Persistent MySQL Database](#)
 - [Step 1: Create the volume](#)
 - [Step 2: Run MySQL container](#)
- [Exercises](#)

3.1 Understanding Docker Port Mapping

Containers often need to communicate with the outside world or with each other. To enable this, Docker allows port mapping between the host and the container.

3.1.1 Concept

Each container has its own isolated network stack. Port mapping exposes internal container ports to the host system.

The basic syntax is:

```
docker run -p [host port]:[container port] [image]
```

- `-p` specifies the port mapping.
- `[host port]` is the port number on the Docker host (e.g. your laptop).
- `[container port]` is the port number inside the container.
- `[image]` is the name of the Docker image to use.

3.1.2 Example

```
docker run -d -p 8080:80 nginx  
curl http://localhost:8080
```

This command starts an Nginx container in detached mode (`-d`) and maps port 8080 on the host to port 80 in the container. When you visit `http://localhost:8080`, you reach the Nginx web server inside the container.

Output:

```
❏  
docker run -d -p 8080:80 nginx  
94432838851b809f981931ab717cb0dc6584d2c5441d80ce2434002bed61d858❏  
  
curl http://localhost:8080  
<!DOCTYPE html>  
<html>  
<head>  
<title>Welcome to nginx!</title>  
<style>  
[...]
```

3.2 Integrating Containers with Data Persistence

By default, container data is ephemeral. Any data stored inside a container's writable layer disappears when the container is removed.

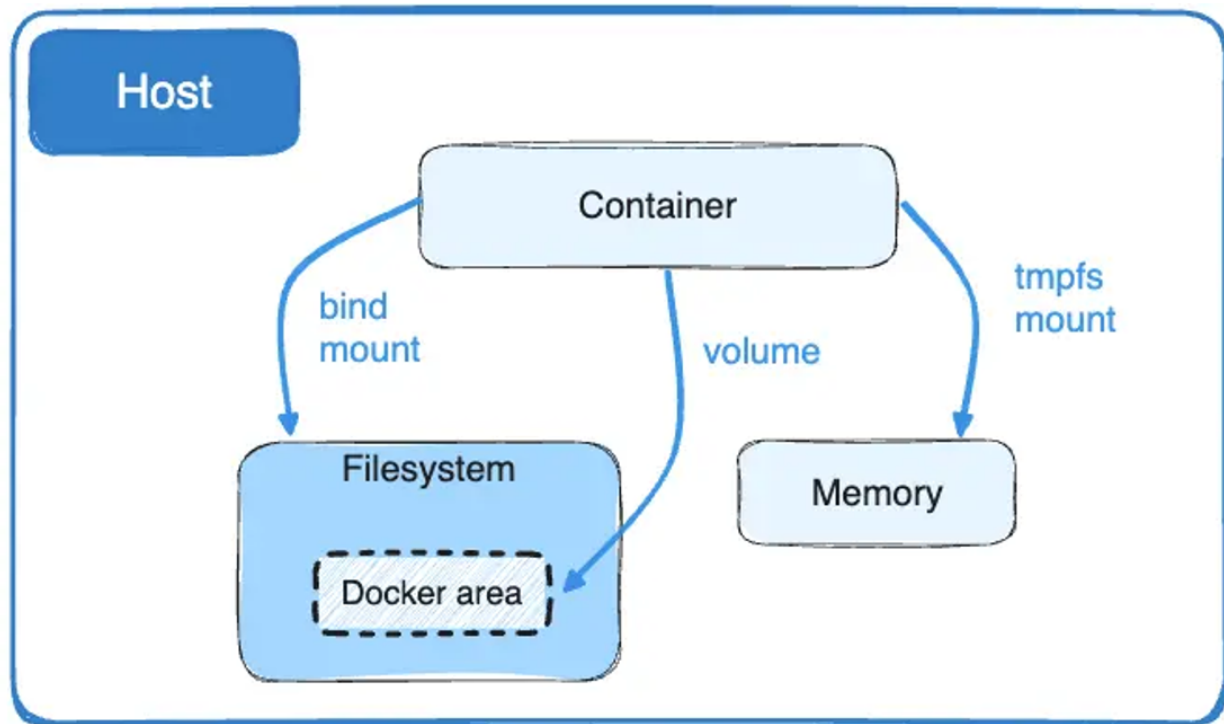


Figure 3.1: Docker mounts

3.2.1 Lifecycle

- Each container has a writable top layer for temporary data.
- When the container is deleted, its writable layer and all data inside are removed.

To persist data beyond the container's lifecycle, Docker provides **volumes** and **bind mounts**.

Method	Managed By	Use Case
Volume	Docker	Databases, logs
Bind Mount	User/Host	Development, shared config

3.3 Using Docker Volumes for Data Persistence

3.3.1 Purpose

Volumes are the recommended mechanism for persistent data. Docker manages them independently of containers.

3.3.2 Key Commands

```
docker volume create [volume name]
docker volume ls
docker volume inspect [volume name]
docker volume rm [volume name]
```

3.3.3 Example

```
docker volume create mydata
docker run --name mynginx1 -d -v mydata:/usr/share/nginx/html nginx
docker volume ls
```

Data written to `/usr/share/nginx/html` in the container is stored in the `mydata` volume. It persists even after the container is removed.

To inspect:

```
docker inspect mydata
```

The “Mountpoint” field shows where Docker stores the volume files (on Docker Desktop, this path is inside the VM).

To view the contents:

```
docker run --rm -v mydata:/data busybox ls -la /data
```

Output:

```
❯ docker volume create mydata
mydata❯

docker run --name mynginx1 -d -v mydata:/usr/share/nginx/html nginx
ec2e503519d03637735f4d8e36ed5959bc0c439fa6afa62905ee27a703ff2865❯

docker volume ls
DRIVER      VOLUME NAME
local      mydata❯

docker inspect mydata
[
  {
    "CreatedAt": "2025-11-06T09:40:23Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
    "Name": "mydata",
    "Options": null,
    "Scope": "local"
  }
]❯

docker run --rm -v mydata:/data busybox ls -la /data
total 16
drwxr-xr-x  2 root    root      4096 Nov  6 09:40 .
drwxr-xr-x  1 root    root      4096 Nov  6 09:40 ..
-rw-r--r--  1 root    root       497 Oct 28 12:05 50x.html
-rw-r--r--  1 root    root       615 Oct 28 12:05 index.html
```

3.4 Using Bind Mounts for Data Persistence

3.4.1 Concept

Bind mounts connect a directory on the host directly to a path inside the container. This provides immediate synchronization between the two locations.

3.4.2 Syntax

```
docker run -v [host path]:[container path]:[options] [image]
```

- `[host path]` → directory on the host system
- `[container path]` → target mount point inside the container
- `[options]` → optional flags such as `ro` (read-only)
- `[image]` → Docker image to use

3.4.3 Example

3.4.3.1 Create a test page

```
mkdir -p nginx_test && echo "<html><body>Hallo PXL</body></html>" > nginx_test/index.html
cat ./nginx_test/index.html
```

3.4.3.2 Run container with bind mount

```
docker run --name mynginx2 -p 8880:80 -d -v "$(pwd)/nginx_test:/usr/share/nginx/html"
nginx
```

Output:

```
❏
mkdir -p nginx_test && echo "<html><body>Hallo PXL</body></html>" > nginx_test/index.html❏
cat ./nginx_test/index.html
<html><body>Hallo PXL</body></html>❏
docker run --name mynginx2 -p 8880:80 -d -v "$(pwd)/nginx_test:/usr/share/nginx/html"
nginx
70fd04dd9b8f67f683f9d40aef6882fec7374ee50e5ad0a25c2f1ae74df687
```

Now, `./nginx_test/` on the host is accessible inside the container at `/usr/share/nginx/html`.

3.4.3.3 Verify

```
curl http://localhost:8880/
```

Output:

```
❏
curl http://localhost:8880
<html><body>Hallo PXL</body></html>
```

If you edit the host file and refresh the page in the browser, changes appear instantly.

```
code ./nginx_test/index.html
```

3.5 Example: A Persistent MySQL Database

To persist MySQL data, use a volume.

3.5.1 Step 1: Create the volume

```
docker volume create mysql-data
```

3.5.2 Step 2: Run MySQL container

```
docker run -d --name mymysql -e MYSQL_ROOT_PASSWORD=mypass -v mysql-data:/var/lib/mysql
-p 3306:3306 mysql
```

Explanation:

- `-d` runs detached.
- `--name mymysql` assigns a name.
- `-e MYSQL_ROOT_PASSWORD=mypass` sets root password.
- `-v mysql-data:/var/lib/mysql` mounts volume for persistent storage.
- `-p 3306:3306` maps host and container ports for external database access.

To inspect stored data:

```
docker run --rm -v mysql-data:/data busybox ls -la /data
```

Output:

```
❏
docker run -d --name mymysql -e MYSQL_ROOT_PASSWORD=mypass -v mysql-data:/var/lib/mysql
-p 3306:3306 mysql:5.7
990f4e3aafa5d097a23915f56f608590df2a5834948e0d2a594966496b6dfafb❏

docker run --rm -v mysql-data:/data busybox ls -la /data
total 99400
-rw-r----- 1 999      999      4194304 Nov  6 09:54 #ib_16384_0.dblwr
-rw-r----- 1 999      999     12582912 Nov  6 09:54 #ib_16384_1.dblwr
drwxr-x---  2 999      999        4096 Nov  6 09:54 #innodb_redo
drwxr-x---  2 999      999        4096 Nov  6 09:54 #innodb_temp
drwxrwxrwt  7 999      999        4096 Nov  6 09:54 .
drwxr-xr-x  1 root     root        4096 Nov  6 09:55 ..
-rw-r----- 1 999      999         56 Nov  6 09:54 auto.cnf
-rw-----  1 999      999        1705 Nov  6 09:54 ca-key.pem
-rw-r--r--  1 999      999        1108 Nov  6 09:54 ca.pem
-rw-r--r--  1 999      999        1108 Nov  6 09:54 client-cert.pem
-rw-----  1 999      999        1709 Nov  6 09:54 client-key.pem
-rw-r----- 1 999      999     12582912 Nov  6 09:54 ibdata1
-rw-r----- 1 999      999     12582912 Nov  6 09:54 ibtmp1
```

```
drwxr-x---  2 999      999          4096 Nov  6 09:54 mysql
-rw-r----- 1 999      999       26214400 Nov  6 09:54 mysql.ibd
lrwxrwxrwx  1 999      999           27 Nov  6 09:54 mysql.sock -> /var/run/mysqld/
mysql.sock
[...]
```

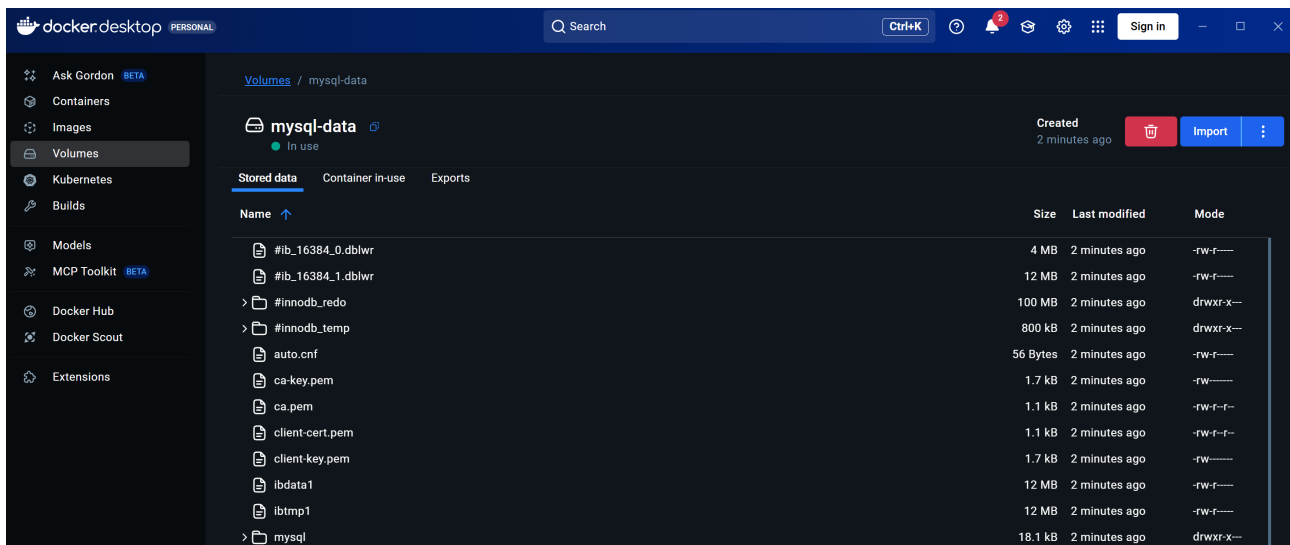


Figure 3.2: docker desktop volume

3.6 Exercises

1. Run an Apache web server:

```
docker run -d -p 8080:80 httpd
```

Browse to <http://localhost:8080> to verify access.

2. View logs in real time:

```
docker logs -f [container_id]
```


4 Secure Shell (SSH)

- Introduction
- What is SSH?
 - Why SSH Exists
- Encryption
 - Asymmetric encryption (RSA, ECDSA, ED25519)
 - Symmetric encryption (AES, ChaCha20)
- How SSH Connects
- SSH Clients and Servers
 - SSH Client
 - SSH Server
- OpenSSH
- OpenSSH Client
- OpenSSH Server
 - Securing SSH Key Pair
- Communication Flow
- Host Keys, Fingerprints
- SSH connection types
 - SSH connection with password
 - SSH connection with PKI
- SSH-Agent
 - Store the passphrase
 - SSH-Agent on Windows
- Commands over SSH
 - Copy files over SSH
- Troubleshooting
 - Connection Debugging
 - Cleaning Up Known Hosts
- Lab Environment (Docker-Based)
 - Architecture
 - Containers

4.1 Introduction

Secure Shell (SSH) is the **standard protocol** for secure remote access to Linux/UNIX machines. It provides:

- Encrypted communication
- Authentication
- Integrity protection
- Remote shell access
- File transfer
- Port forwarding (out of scope for this lab)

In this lab you will learn the underlying principles, and practice using SSH in a controlled **Docker-based lab environment**.

4.2 What is SSH?

SSH (Secure Shell) is a secure network protocol that allows encrypted communication between a client and a remote server.

Port 22 is the default port for SSH connections.

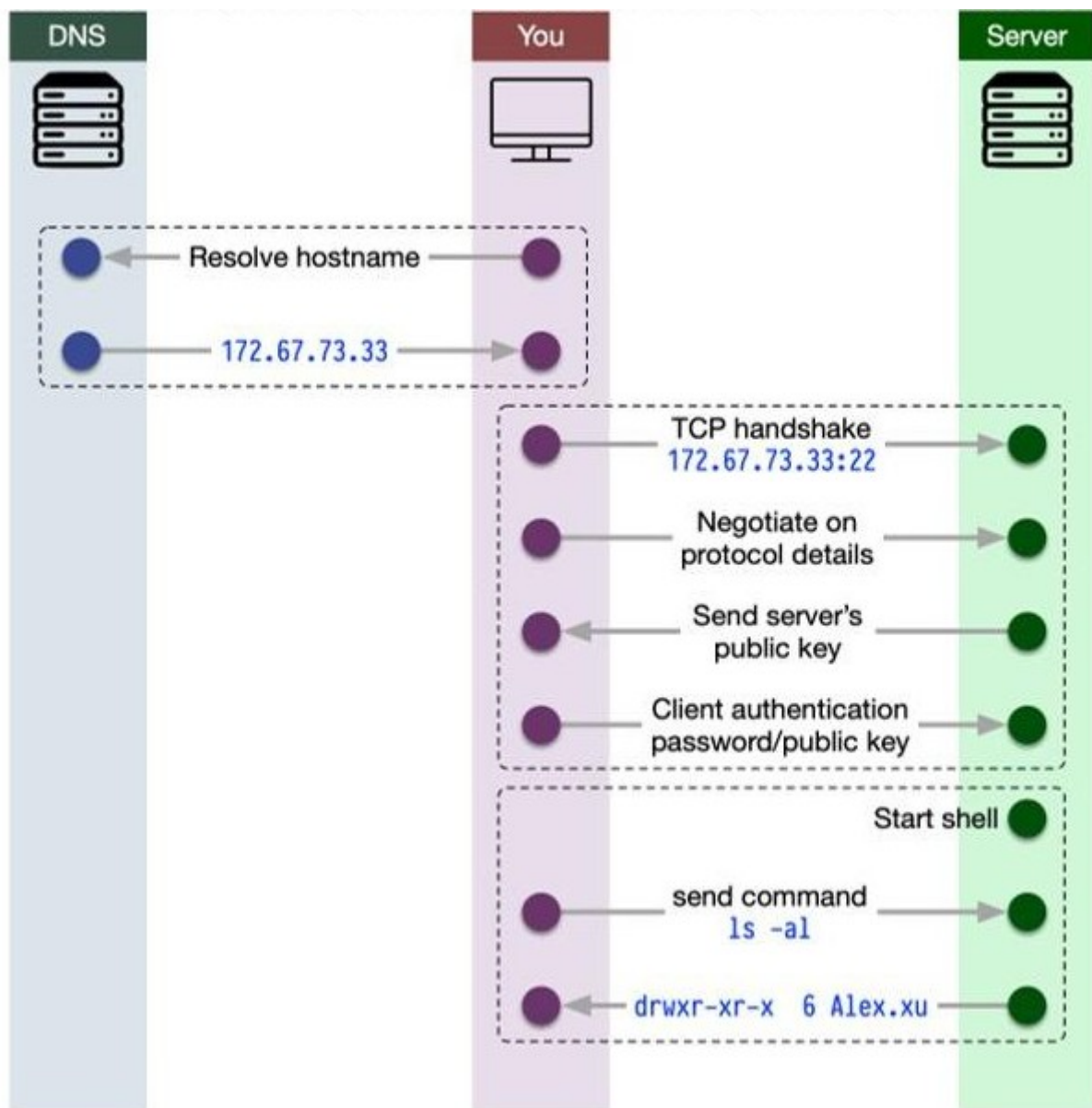


Figure 4.1: alt text

4.2.1 Why SSH Exists

Before SSH, remote access protocols like telnet and rsh transmitted **everything unencrypted**, including passwords. SSH solves these issues by:

- Encrypting all traffic
- Verifying server identity
- Supporting multiple authentication mechanisms

4.3 Encryption

SSH uses a combination of:

4.3.1 Asymmetric encryption (RSA, ECDSA, ED25519)

Used during the **initial key exchange** only.

4.3.2 Symmetric encryption (AES, ChaCha20)

Used for all subsequent data transfer.

4.4 How SSH Connects

A typical SSH session uses:

Step	Purpose
Key exchange	Agree on encryption keys
Server authentication	Verify server identity via host keys
User authentication	Password or public key
Encrypted channel	Safe remote shell communication

4.5 SSH Clients and Servers

4.5.1 SSH Client

Command:

```
ssh [<user>@]<hostname>
```

4.5.2 SSH Server

Runs the daemon:

```
/usr/sbin/sshd
```

4.6 OpenSSH

OpenSSH is the most widely used implementation of the SSH protocol.

4.7 OpenSSH Client

Included on Linux, macOS, and Windows.

```
sudo apt install openssh-client
```

Usage:

```
ssh user@hostname
```

Configuration files:

- Linux:

System-wide configuration is in:

```
/etc/ssh/ssh_config
```

On the user side the client configuration is in:

```
~/.ssh/config
```

- Windows:

System-wide configuration:

```
$env:ProgramData\ssh\
```

User configuration:

```
$env:USERPROFILE\.ssh\
```

These files may not exist by default.

You can set the VisualHostKey on yes to see the host key fingerprint when connecting to see ASCII art of the key.

4.8 OpenSSH Server

sshd is the server daemon.

```
sudo apt install openssh-server #(see Dockerfile in lab repo)
```

Configuration files:

```
/etc/ssh/sshd_config
```

- ListenAddress: IP address to listen on (default: all)
- PermitRootLogin “no”: for security (use sudo after login)
- PasswordAuthentication “no”: for security. Only login with key instead of password
- DenyUsers: these users are not allowed to login over ssh
- DenyGroups: users of these groups are not allowed to login
- man sshd_config

SSH uses port **22** by default on the server.

```
grep ssh /etc/services
```

Check if sshd is running:

```
ps aux | grep sshd
```

```
root@ssh-server1:~# grep ssh /etc/services
ssh                22/tcp             # SSH Remote Login Protocol
root@ssh-server1:~# ps aux | grep sshd
root      1  0.0  0.0 12020  7984 ?        Ss   09:20   0:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
root     87  0.0  0.0  3528  1612 pts/0    S+   11:34   0:00 grep --color=auto sshd
root@ssh-server1:~#
```

Figure 4.2: alt text

Host keys:

```
/etc/ssh/ssh_host_*
```

```
/etc/ssh/ssh_host_rsa_key.pub
/etc/ssh/ssh_host_dsa_key.pub
/etc/ssh/ssh_host_ecdsa_key.pub
/etc/ssh/ssh_host_ed25519_key.pub
```

These public keys identify the server to clients.

4.8.1 Securing SSH Key Pair

Adjust the SSH server configuration in:

```
/etc/ssh/sshd_config
```

The setting:

```
PasswordAuthentication no
```

Determines whether password-based logins are allowed.

If you do not change this setting, both authentication methods — public key and password — will continue to work.

Reload the SSH server configuration using:

On Linux hosts with systemd:

```
sudo systemctl reload sshd
```

On Linux hosts without systemd (e.g., containers):

```
service ssh reload
```

4.9 Communication Flow

What Happens When You Run `ssh <hostname>`

1. DNS Resolution
 - Resolve hostname → IP.
 2. TCP Handshake
 - Connect to port **22**.
 3. Host Key Exchange
 - The server proves identity using its **host keys**.
 4. Authentication
 - Password or
 - Public/private key
 5. Encrypted Session
 - All communication is encrypted.
-

4.10 Host Keys, Fingerprints

Example fingerprint:

```
ssh-keygen -l -f /etc/ssh/ssh_host_ed25519_key
```

View the public key:

```
cat /etc/ssh/ssh_host_ed25519_key.pub
```

4.11 SSH connection types

4.11.1 SSH connection with password

Login as user on the remote server using username and password authentication. With this method, the user must already exist on that remote system. The hostname can be an IP address or DNS name.

```
ssh user@hostname
```

or

Log in on the server using the current client user, provided that this user also exists on the server.

```
ssh hostname
```

The first time you connect, you will be asked whether you trust this “unknown server.”

```
$ docker exec -it ssh-server1 /bin/bash
root@ssh-server1:/# ssh student@ssh-server2
The authenticity of host 'ssh-server2 (172.18.0.3)' can't be established.
ED25519 key fingerprint is SHA256:0b3Irjy5ca44RCekTtnTEdJMxQpd0Xx0nbZpJjrZD5A.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ssh-server2' (ED25519) to the list of known hosts.
student@ssh-server2's password:
Welcome to Ubuntu 24.04 LTS GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64

student@ssh-server2:~$
```

If you confirm, the server's public key will be stored on the client in the home directory of the current user under:

```
~/ .ssh/known_hosts
```

It can also be configured system-wide by manually storing the public key(s) of SSH servers in:

```
/etc/ssh/ssh_known_hosts
```

On the server we can check the fingerprint of the private key and the content of the public key.

```

$ docker exec -it ssh-server1 /bin/bash
root@ssh-server1:/# ssh student@ssh-server2
The authenticity of host 'ssh-server2 (172.18.0.3)' can't be established.
ED25519 key fingerprint is SHA256:0b3Irjy5ca44RcEkTnTnEdJmXQpd0Xx0nbZpJjrZD5A
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ssh-server2' (ED25519) to the list of known hosts.
student@ssh-server2's password:
Welcome to Ubuntu 24.04 LTS GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64

student@ssh-server2:~$ exit
logout
Connection to ssh-server2 closed.
root@ssh-server1:/# cat ~/.ssh/known_hosts
[1]uY0xyI1l6zgGBSY0dtui2CnIdg4=[a+0zFrgiknLxG/CLdRJIss4/R9w= ssh-ed25519
AAAAAC3NZaC1IIZDIINTE5AAAAIMISLVBRI9920uMAN7WEXG0r1hZ0tGFF7wRCVF667/cpD5q
[1]t1t1FPINEXM313GfoYvX10pbyIg=[wft/Nh3wgUze7Y2tXEq4Ek6UAU= ssh-tsa
AAAAAB3nZaC1yc2EAAADAAQABAA8BQCd7i1Lhf9A9rNdMT8QnXhd0g7f3xplyoz+9o1Xv0nbx0Qsp50bT
3k9DQEJCX0bIwzvtz+ilG/KMV3TG1i30NotkguseqGalpFzjbUwrgPXsq4y7sgYArdwg8pJgiFAaycu
eQyG7n+YKRTXcRoMLTFBRELFP9AZXyVb2Shh/VF9RBLglpBneE10+BeujC6r4jy91oM7KzthIIVSs42z
97Rk1McVvFFtWESWQy+LyeTRAxUyrmTjLZhHeoGt5KpFo6G0ZJ1QRbyBN3HIz2y6WwrcdpnJhU3V1sRS8
yFCCZ9ZLVU6AtPdQASGe08jpfY123VxvMwLkAcX19mxwBzTlMDykyKSw+1hT3H/A3CqZGX
xFNU5N00+FXdAACCoC7jki6fY9cda5w5zG+ZwT1TOBDSH7gMRB/Ld1NEhrQxJtx3/cKj/v71n5joGqEf
3090JMa1kQEyvjxIHSHKHTt3+PNC9i42s4rDnyS1NEM86a8pSDPRUILLdyJ78EE=
[1]Ppyh8G6ICKlYwDvXZPIDiJwOuM=[DirBRBOnhFBjZuak7/2+HC4HHW8= ecdsa-sha2-nistp256
AAAAEZVjZHNhLXNoYT1tbnZldHhAYNTYAAAAIbmlzdHhAYNTYAAABBBMoarNziFePcAl+217d1lhwk4wH
K/Lh5wD1URT7TZ0YC+RaCudcrObG1EqvDkK8U0FDtKhY4jQiled3xB/tVs=
root@ssh-server1:/#
root@ssh-server1:/# ssh student@ssh-server2
student@ssh-server2's password:
Welcome to Ubuntu 24.04 LTS GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64

Last login: Tue Aug 6 08:49:09 2024 from 172.18.0.2
student@ssh-server2:~$

```

Figure 4.3: alt text

What if the server key changes?

If the server's host key changes (e.g., server reinstallation), the client will warn you about a possible security breach and refuse to connect.

- We remove the current container and the keypair (volume).
- We start a new instance of ssh-server2.

- Now the contents of ~/.ssh/known_hosts no longer match the new server key.
- The new server is no longer a known host but an unknown host.
- You can delete the old server's entry from the known_hosts file.
- See the connection error message.

```
# student @ laptop in ~\github\ssh-lab
$ docker stop ssh-server2; docker rm ssh-server2
ssh-server2
ssh-server2
# student @ laptop in ~\github\ssh-lab
$ docker volume rm -f ssh-lab_ssh-keys-server2
ssh-lab_ssh-keys-server2
# student @ laptop in ~\github\ssh-lab
$ docker compose up -d ssh-server2
[+] Running 2/2
  Volume "ssh-lab_ssh-keys-server2" Created
  Container ssh-server2 Started
0.4s
# student @ laptop in ~\github\ssh-lab
$ docker exec -it ssh-server1 /bin/bash
root@ssh-server1:/# ssh student@ssh-server2
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!    @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ED25519 key sent by the remote host is
SHA256:G3i7UMeyzGsdfFuha7pTpflLlsf+zXzSWoGELuTID0k.
Please contact your system administrator.
Add correct host key in /root/.ssh/known_hosts to get rid of this message.
Offending ECDSA key in /root/.ssh/known_hosts:3
  remove with:
    ssh-keygen -f '/root/.ssh/known_hosts' -R 'ssh-server2'
Host key for ssh-server2 has changed and you have requested strict checking.
Host key verification failed.
root@ssh-server1:/#
```

Remove the old host key with:

```
ssh-keygen -f '/root/.ssh/known_hosts' -R 'ssh-server2'
```

4.11.2 SSH connection with PKI

Login using public/private key authentication.

This makes it possible to login without a password.

1. Make sure you have an SSH client installed on the host from which you want to connect.
2. Install an SSH server on the host you want to connect to, and ensure the configuration in /etc/ssh/sshd_config is correct.
3. Generate a private/public key pair on the client using ssh-keygen.
4. Add the client's public key to the ~/.ssh/authorized_keys text file on the server.
5. You can do this manually using scp, or—on Linux—conveniently with ssh-copy-id on the client.
6. The same public key can be reused to connect passwordless over SSH to multiple servers.

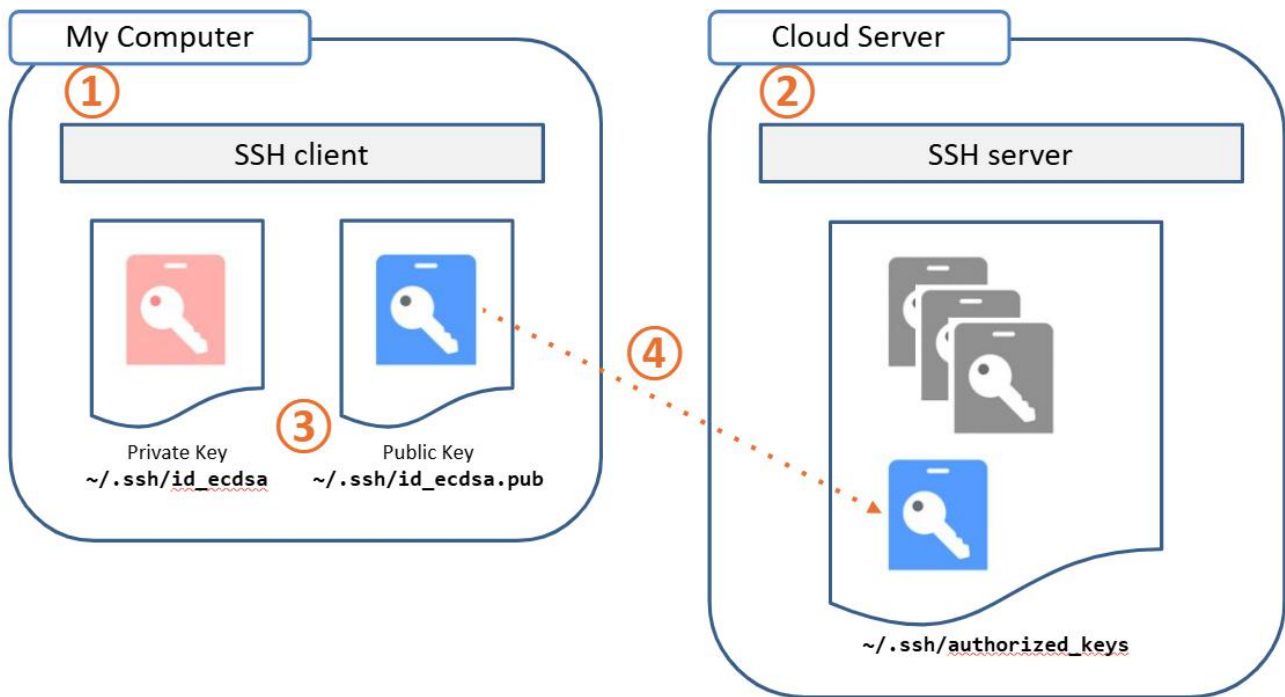


Figure 4.4: alt text

Create a key pair on the client:

```
ssh-keygen -t ed25519
```

- You can use other algorithms like RSA or ECDSA and specify key size with `-b`.
 - `-b 512` for ED25519 is fixed.
 - `-b 256` for ECDSA is recommended.
 - `-b 4096` for RSA is recommended.
- Private key can be protected with a passphrase.
- When a keypair is created, two files are generated:
 - Private key (keep this secret) - `~/.ssh/id_ed25519`
 - Public key - `~/.ssh/id_ed25519.pub`

```
root@ssh-server1:/# ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_ed25519
Your public key has been saved in /root/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:3QtKXhuej3YAf30RiycEt6WlouOya81SN2r3a+WXVLY root@ssh-
server1
The key's randomart image is:
+--[ED25519 256]--+
|
| . . o |
| o = o |
| .. . = + E |
| So=o.o o. |
| o +=B.+ + |
```

```
| o=+*. * +o|
| + Boo .oo|
| .oB.o.o+..|
+----[SHA256]-----+
root@ssh-server1:/# ls -la ~/.ssh/
total 28
drwx----- 1 root root 4096 Aug 6 11:15 .
drwx----- 1 root root 4096 Aug 6 09:52 ..
-rw----- 1 root root 411 Aug 6 11:15 id_ed25519
-rw-r--r-- 1 root root 98 Aug 6 11:15 id_ed25519.pub
-rw----- 1 root root 978 Aug 6 11:14 known_hosts
root@ssh-server1:/#
```

We copy the public key to the server, for the user you want to log in as (step 4).

```
ssh-copy-id [-i ~/.ssh/id_ed25519.pub] <username>@<serverip>
```

To allow the key to be copied into that user's home directory, you must provide the password of that user.

The option `-i ~/.ssh/id_ed25519.pub` is not required if you use the default filename and path.

On the SSH server, the public key will be placed in the `authorized_keys` file inside the home directory of the target user.

```
root@ssh-server1:/# ssh-copy-id student@ssh-server2
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed:
"/root/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new
key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if
you are prompted now it is to install the new keys
student@ssh-server2's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'student@ssh-
server2'"
and check to make sure that only the key(s) you wanted were
added.

root@ssh-server1:/# ssh student@ssh-server2
Welcome to Ubuntu 24.04 LTS GNU/Linux 5.15.153.1-microsoft-
standard-WSL2 x86_64
Last login: Tue Aug 6 11:14:49 2024 from 172.18.0.2
student@ssh-server2:~$ cat ~/.ssh/authorized_keys
ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAICQ+oMgv4QfHL/LPFdHAUcPLbW6GMwHIudwfNvuRp
xCL root@ssh-server1
student@ssh-server2:~$
```

Now you can log in without a password:

Protect your private key with a passphrase for better security.

To add or change the passphrase of your private key, use:

```
ssh-keygen -p -f /path/to/your/private/key
```

`-p`: change passphrase `-f`: specify the file of the private key

Use the passphrase: `pxlpass`

```
root@ssh-server1:/#ssh-keygen -p -f ~/.ssh/id_ed25519
```

```
Key has comment 'root@ssh-server1'
Enter new passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved with the new passphrase.
```

```
root@ssh-server1:/# █
```

```
root@ssh-server1:/#ssh student@ssh-server2

Enter passphrase for key '/root/.ssh/id_ed25519':

Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64)

- Documentation:  <https://help.ubuntu.com>
- Management:    <https://landscape.canonical.com>
- Support:        <https://ubuntu.com/pro>

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

student@ssh-server2:~$
```

A passphrase-protected private key provides better security, as an attacker would need both the private key file and the passphrase to gain access.

Nevertheless, using SSH keys without a passphrase is still more secure than password-based authentication, as it eliminates the risk of password guessing or brute-force attacks but the user must enter the passphrase each time they use the key unless they use an SSH agent to cache the decrypted key in memory.

4.12 SSH-Agent

`ssh-agent` is a program that holds private keys used for public key authentication (SSH). It runs in the background and caches decrypted private keys, allowing users to authenticate to SSH servers without repeatedly entering passphrases.

- `ssh-agent` and `ssh-add` are installed by default on Windows 10/11.
- You can configure the `ssh-agent` service to start automatically when you log in.
- SSH agent forwarding (`ssh -A`) has issues when using the native Windows SSH client compared to the native Linux SSH client (which also runs correctly inside WSL2).
- The Linux implementation handles agent forwarding reliably, while the Windows client may fail or behave inconsistently.
- Note: SSH agent forwarding is typically used when connecting through an intermediate bastion host so you can continue connecting deeper into a private network while keeping your private key only in your local memory.

https://learn.microsoft.com/en-us/windows-server/administration/openssh/openssh_keymanagement

4.12.1 Store the passphrase

Storing the passphrase in `ssh-agent` allows you to use your private key without entering the passphrase each time you connect to an SSH server.

When using ssh-agent, you only need to unlock your private key once and can then use it for multiple ssh connections.

To store the passphrase of your private key in the ssh-agent (RAM), follow these steps:

1. Start the ssh-agent in the background
2. Add your private key to the ssh-agent and enter the passphrase when prompted
3. Verify that the key has been added
4. Connect to the SSH server without being prompted for the passphrase

```
root@ssh-server1:/# ssh-agent bash
root@ssh-server1:/# ssh-add ~/.ssh/id_ed25519
Enter passphrase for /root/.ssh/id_ed25519:
Identity added: /root/.ssh/id_ed25519 (root@ssh-server1)

root@ssh-server1:/# ssh student@ssh-server2
Welcome to Ubuntu 24.04 LTS GNU/Linux 5.15.153.1-microsoft-standard-WSL2
x86_64

student@ssh-server2 :~$ exit
logout
Connection to ssh-server2 closed.
root@ssh-server1:/# exit
exit
root@ssh-server1:/#
```

4.12.2 SSH-Agent on Windows

```
# Start the ssh-agent service
Start-Service ssh-agent
# Add your private key to the ssh-agent
ssh-add $env:USERPROFILE\.ssh\id_ed25519
# Verify that the key has been added
ssh-add -l
# Connect to the SSH server without being prompted for the passphrase
ssh student@ssh-server2
```

unable to start ssh-agent service, error :1058

Step	Action	Details
1. Check SSH-Agent Service Status	Open Services Console	Press Win + R , type services.msc , press Enter
	Locate service	Find OpenSSH Authentication Agent
	Check status	If status is Disabled , it must be changed
2. Enable and Start Service	Enable service	Right-click service → Properties → set Startup type: Automatic → Apply → OK
	Start service	Click Start in the Properties window
	Verify running	Service should show Running (refresh list)

Step	Action	Details
3. Restart Terminal	Restart shells	Close and reopen Command Prompt, PowerShell, or WSL
	Test again	Try starting or using ssh-agent again

To check the status of the `ssh-agent` service in PowerShell, you can use the following script:

```
# checkSSHAgentStatus.ps1
$service = Get-Service -Name "ssh-agent"

if ($service.Status -eq 'Running') {
    Write-Host "OpenSSH Authentication Agent service is already running."
} else {
    Write-Host "OpenSSH Authentication Agent service is not running."
}
```

```
# startSSHAgent.ps1
$service = Get-Service -Name "ssh-agent"

if ($service.Status -ne 'Running') {
    # Set the service to start automatically
    Set-Service -Name "ssh-agent" -StartupType Automatic
    # Start the service
    Start-Service -Name "ssh-agent"
    Write-Host "OpenSSH Authentication Agent service started and enabled."
} else {
    Write-Host "OpenSSH Authentication Agent service is already running."
}
```

4.13 Commands over SSH

You can execute commands on a remote server over SSH without opening an interactive shell session.

```
ssh user@hostname 'command_to_execute'
```

Example:

```
ssh student@ssh-server2 'ls -la ~'
```

```
ssh student@ssh-server2 'echo $HOSTNAME; uname -a'
```

After executing the command, the SSH session will automatically close. The command is executed on the remote server, but the output is displayed locally.

4.13.1 Copy files over SSH

You can copy files between your local machine and a remote server using `scp` (secure copy) over SSH.

```
scp /path/to/local/file user@hostname:/path/to/remote/destination
```

Example: Copy a file from the local machine to the remote server

```
scp ~/myscript.sh student@ssh-server2:/home/student/my_super_script.sh
```

The target directory must exist on the remote server; otherwise, the command will fail.

Example: Copy a file from the remote server to the local machine

```
scp student@ssh-server2:/home/student/notes.txt ~/downloaded_notes.txt
```

4.14 Troubleshooting

4.14.1 Connection Debugging

```
ssh -vvv student@ssh-server1
```

4.14.2 Cleaning Up Known Hosts

```
ssh-keygen -R <hostname>  
ssh-keygen -R <ip>
```

4.15 Lab Environment (Docker-Based)

4.15.1 Architecture

<https://github.com/PXL-Systems-Advanced/ssh-lab>

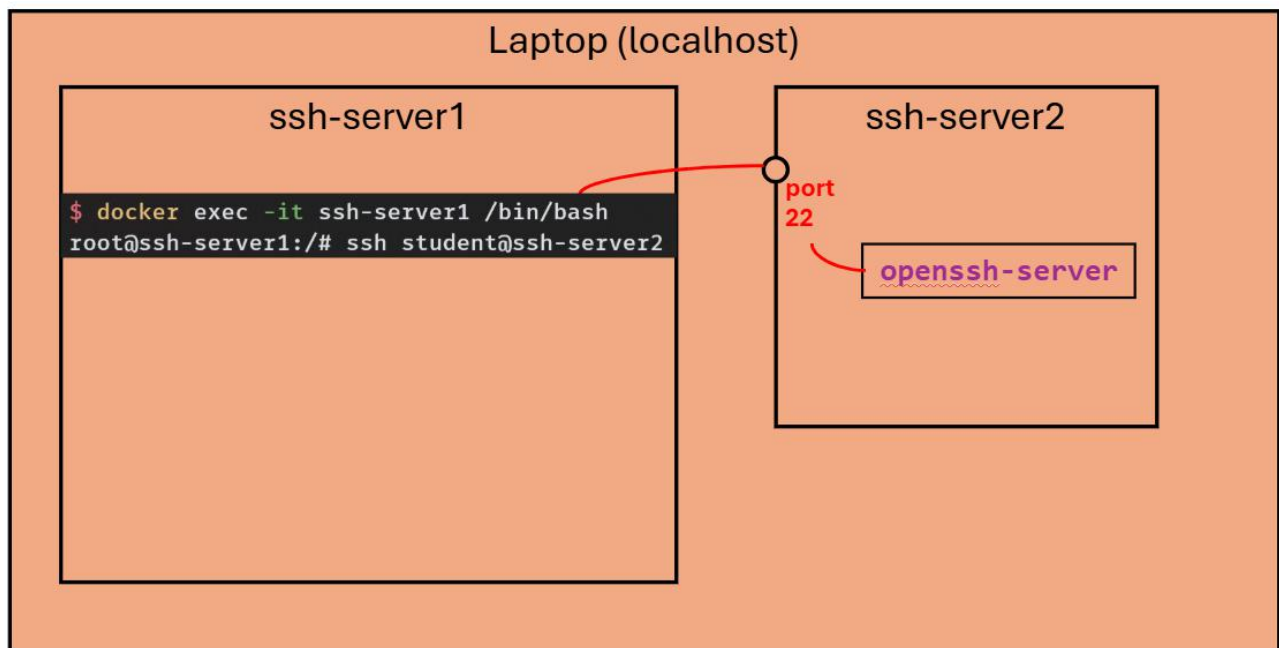


Figure 4.5: alt text

4.15.2 Containers

- `ssh-server1`
- `ssh-server2`

5 Docker Build and Dockerfile - Comprehensive Guide

5.1 Architecture

Docker Build implements a client-server architecture, where:

- **Buildx** is the client and the user interface for running and managing builds
- **BuildKit** is the server, or builder, that handles the build execution.

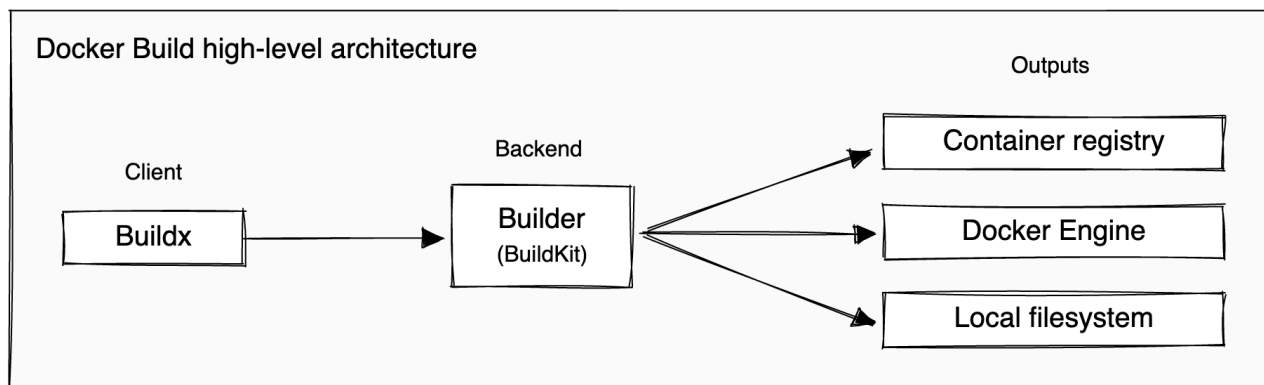


Figure 5.1: Alt text

Buildx is a CLI tool that provides a user interface for working with builds. You're using Buildx by default when you invoke the `docker build` command.

BuildKit, or `buildkitd`, is the daemon process that executes the build workloads.

A build execution starts with the invocation of a `docker build` command. Buildx interprets your build command and sends a build request to the BuildKit backend. The build request includes:

- The Dockerfile
- Build arguments
- Export options
- Caching options

BuildKit resolves the build instruction and executes the build steps. For the duration of the build, Buildx monitors the build status and prints the progress to the terminal.

If the build requires resources from the client, such as local files or build secrets, BuildKit requests the resources that it needs from Buildx.

5.2 Dockerfile

A **Dockerfile** is a text document that contains all the commands a user could call on the command line to assemble an image. The Dockerfile is the starting point for creating a Docker image. It follows a specific syntax and set of instructions.

Here's a basic structure of a Dockerfile:

```
# Comment - This line is a comment and will not affect your build
INSTRUCTION arguments
```

The **INSTRUCTION** is an uppercase keyword, followed by its arguments. Here's what some of the most common Dockerfile instructions do:

- **FROM**: Sets the base image for subsequent instructions. Every Dockerfile must start with a **FROM** instruction.
- **RUN**: Executes any commands in a new layer on top of the current image and commits the results.
- **CMD**: Provides defaults for an executing container. If Dockerfile has more than one **CMD**, only the last **CMD** will take effect.
- **ENTRYPOINT**: Sets the command and parameters that will be executed first when a container is run.
- **LABEL**: Adds metadata to an image, such as version, description, or maintainer info.
- **EXPOSE**: Informs Docker that the container listens on the specified network ports at runtime.
- **ENV**: Sets the environment variable **<key>** to the value **<value>**.
- **ADD**: Copies new files, directories, or remote file URLs from **<src>** and adds them to the filesystem of the image at the path **<dest>**.
- **COPY**: Copies new files or directories from **<src>** and adds them to the filesystem of the container at the path **<dest>**.

5.2.1 Quick Reference Example

```
# Complete Dockerfile showcasing common instructions
FROM node:18-alpine
LABEL maintainer="dev@example.com"
ENV NODE_ENV=production
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
ENTRYPOINT ["node"]
CMD ["server.js"]
```

5.3 Base Images

Choosing the right base image is crucial as it forms the foundation of your Docker image. It's recommended to use official base images provided by Docker Hub whenever possible, as they are well-maintained, secure, and regularly updated. Some common base images are **ubuntu**, **alpine**, and **debian**.

When selecting a base image, consider:

- **Size:** Images like `alpine` are very lightweight and can help keep the overall size of your image down.
- **Security:** Smaller images often have a reduced attack surface.
- **Compatibility:** Ensure that the base image is compatible with your application.
- **Maintenance:** Prefer images that are frequently updated for security patches.

5.3.1 Common Base Image Comparison

Base Image	Size	Use Case	Package Manager
alpine	~5MB	Minimal production images	apk
debian:slim	~25MB	Balance of size and compatibility	apt
ubuntu	~70MB	Development, familiar environment	apt
distroless	~2MB	Security-focused, no shell	none

5.4 Layering

Docker images are built up from a series of layers. Each layer represents an instruction in the Dockerfile. Layers are stacked and each one is a delta of the changes from the previous layer.

Consider this Dockerfile snippet:

```
# syntax=docker/dockerfile:1
FROM python:3.8-slim
COPY . /app
RUN pip install -r /app/requirements.txt
```

Each instruction creates a new layer:

- `FROM` creates a layer from the `python:3.8-slim` Docker image.
- `COPY` adds a layer with your app's files.
- `RUN` adds a layer with your installed packages.

Layers are cached, and if the instruction does not change, Docker will reuse the cached layer, speeding up the build process. The cache is invalidated if the instruction or any layer before it has changed.

5.4.1 Layer Caching Strategy

```
# Good ordering - leverages cache effectively
FROM node:18-alpine
WORKDIR /app
# Dependencies change less frequently
COPY package*.json ./
RUN npm ci
# Application code changes frequently
COPY . .
RUN npm run build

# Poor ordering - cache breaks often
FROM node:18-alpine
```

```
WORKDIR /app
COPY . . # Any code change invalidates all subsequent layers
RUN npm ci
RUN npm run build
```

5.5 Core Instructions

5.5.1 FROM

```
FROM <image> [AS <name>]
```

The **FROM** instruction in a Dockerfile is the first instruction and is required for any valid Dockerfile. It defines the base image to use to start the build process. It can be any image, including the images you have created previously, or images from Docker Hub or other Docker image registries.

- Use tags like `debian:12-slim`.
- `<image>` is the name of the image and can optionally include a tag or a digest. If no tag is specified, Docker will default to the `latest` tag.
- `[AS <name>]` is an optional clause that can be used to give a name to a new build stage in a **multi-stage build** process. (see later)

When you specify **FROM**, the Docker daemon will begin the build process by pulling the specified image from the registry unless it's already present locally. Subsequent instructions in the Dockerfile modify this image. When you commit the changes, you effectively create a new image.

Examples:

```
# Start from the Ubuntu image with tag 20.04
FROM ubuntu:20.04
```

```
# Start from a smaller base image, like Alpine
FROM alpine:3.12
```

```
# Multi-stage build where a name is given to the stage
FROM node:14 AS build-env
```

```
# Using a specific digest for reproducible builds
FROM python:3.11@sha256:abc123def456...
```

5.5.2 WORKDIR

```
WORKDIR /path/to/workdir
```

The **WORKDIR** instruction is used to define the working directory of a Docker container at any given time. The working directory refers to the directory in which the command specified in the **CMD** or **RUN** Dockerfile instruction will be executed.

When you specify **WORKDIR**, the directory is created if it does not already exist. If there are multiple **WORKDIR** instructions in a Dockerfile, they will be executed sequentially, and the final **WORKDIR** will be the working directory when the container runs.

For instance:

```
WORKDIR /a
WORKDIR b
WORKDIR c
RUN pwd
```

The output of `pwd` will be `/a/b/c`.

Specify Work Directories: It's a good practice to set a `WORKDIR` instead of using `cd` in a `RUN` instruction. This sets a clear, consistent location for subsequent commands to operate within.

5.5.3 COPY

```
COPY source destination
```

The `COPY` instruction is used to copy files and directories from the build context (the set of files located in the directory from which the `docker build` command is run) into the Docker image.

This will copy the `source` file or directory from the build context to the `destination` path in the image's filesystem.

5.5.3.1 Advanced COPY usage

```
# Copy with ownership change (reduces layers)
COPY --chown=user:group source destination

# Copy multiple sources
COPY file1.txt file2.txt /app/

# Use wildcards
COPY *.json /app/

# Preserve permissions
COPY --chmod=755 script.sh /usr/local/bin/
```

5.5.4 ADD

```
ADD source destination
```

The `ADD` instruction is similar to `COPY` but with two key differences:

1. It can handle remote URLs as the source, so you can use it to download files from the internet and copy them directly into the image.
2. It can automatically unpack compressed files. If the source is a local tar archive in a recognized compression format (identity, gzip, bzip2 or xz), `ADD` will unpack it into the destination directory.

Just like `COPY`, `ADD` will copy the `source` from the build context to the `destination` path in the image's filesystem. But if `source` is a URL or a compressed file, `ADD` will download or unpack it.

Use COPY over ADD: Because `COPY` is more transparent than `ADD`, it is recommended to use `COPY` unless you specifically need to download a file from a URL or auto-extract a compressed local file.

5.5.5 RUN

```
RUN <command>
```

or using the shell form:

```
RUN ["executable", "param1", "param2", ...]
```

The **RUN** instruction is one of the core commands in a Dockerfile. It is used to execute a command inside your Docker image during the build process. Each **RUN** instruction creates a new layer in the image, and the state of the container after the command has been executed is committed to the image as a new layer.

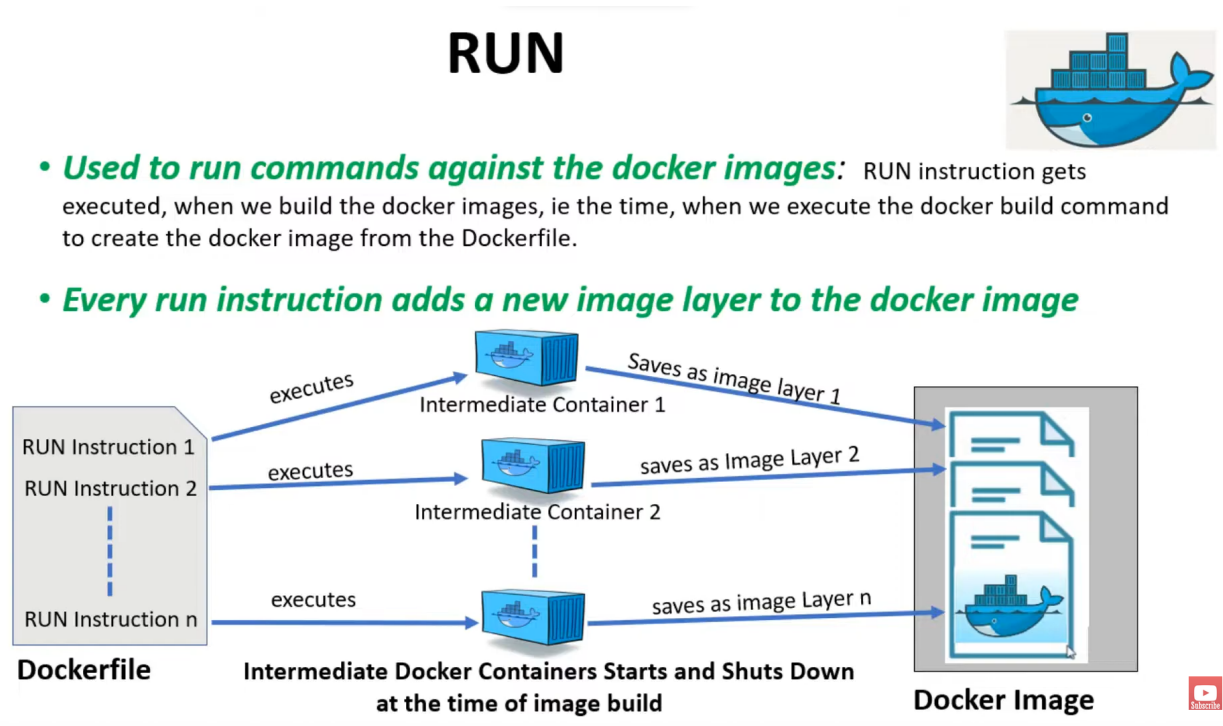


Figure 5.2: Alt text

The first form, known as the shell form, runs the command in a shell, which by default is `/bin/sh -c` on Linux or `cmd /S /C` on Windows. *** End Patch The second form, known as the exec form, runs the executable directly, without a shell.

Examples:

- Installing packages:

```
RUN apt-get update && apt-get install -y package-name
```

In this example, `apt-get update` updates the list of available packages and their versions, but it doesn't install or upgrade any packages. `apt-get install -y package-name` installs a package without prompting for confirmation.

- Using the exec form to invoke a command:

```
RUN ["apt-get", "update"]
RUN ["apt-get", "install", "-y", "package-name"]
```

In most cases, the shell form is used because it is more convenient for running simple commands. However, when you need to run a command without a shell (for example, when you don't want to deal with shell processing of your command's arguments), the exec form can be very useful.

Best Practices:

- To reduce the number of layers and keep the image size down, it's common to chain commands together using logical operators:

```
RUN apt-get update && apt-get install -y \
    package1 \
    package2 \
    && rm -rf /var/lib/apt/lists/*
```

This approach minimizes the number of layers by combining multiple commands into a single `RUN` instruction and also cleans up after itself to keep the image size small.

- For readability and maintenance, `RUN` commands can be broken over multiple lines using the backslash (`\`) as a continuation character.

Remember that each `RUN` instruction is executed independently and in a temporary container that is based on the current image. After the command executes, any changes to the filesystem are committed to a new layer, and the temporary container is removed. This is why Docker images are composed of multiple layers stacked on top of each other.

5.5.6 ENTRYPOINT vs CMD

In Docker, `ENTRYPOINT` and `CMD` define what runs when a container starts. They can work together but serve distinct roles.

Instruction	Purpose	Overridden by	Typical Use
<code>ENTRYPOINT</code>	Defines the main executable	<code>--entrypoint</code> flag	Always run the same base command
<code>CMD</code>	Provides default arguments or a fallback command	Arguments after image name	Default parameters or commands

5.5.6.1 Exec vs Shell Form

Both `ENTRYPOINT` and `CMD` support two forms:

- **Exec form (preferred):** Uses JSON array syntax. Runs the command directly without a shell.

```
ENTRYPOINT ["python", "/app/script.py"]
```

Handles signals correctly (important for PID 1) and avoids shell interpretation.

- **Shell form:** Uses plain string syntax and runs through `/bin/sh -c`.

```
ENTRYPOINT python /app/script.py
```

Allows shell features like variable expansion or pipes but does not forward signals cleanly.

Use the exec form for predictable behavior.

5.5.6.2 ENTRYPOINT

- Sets the executable that always runs.
- Arguments passed to `docker run` are appended.

Example:

```
ENTRYPOINT ["echo", "Hello"]
```

`docker run <image>` results in `Hello`.

`docker run <image> World` results in `Hello World`.

5.5.6.3 CMD

- Provides default arguments for `ENTRYPOINT`, or acts as the default command if no `ENTRYPOINT` exists.
- Overridden by arguments passed to `docker run`.

Example:

```
CMD ["echo", "Hello World!"]
```

`docker run <image>` results in `Hello World!`.

`docker run <image> World` results in the error `"World": executable file not found in $PATH` because with only a `CMD` defined, any arguments after the image name replace the entire default command instead of being appended. Docker therefore tries to execute `World` as the program, which does not exist.

`docker run <image> echo Hello PXL` results in `Hello PXL`.

5.5.6.4 ENTRYPOINT and CMD Together

When both are defined:

```
ENTRYPOINT ["echo", "Hello"]  
CMD ["World"]
```

Default: `Hello World` Override:

```
docker run <image> Docker
```

Result: `Hello Docker`

Example for applications:

```
ENTRYPOINT ["python", "/app/my_script.py"]  
CMD ["--help"]
```

```
docker run myimage --verbose runs python /app/my_script.py --verbose
```

5.5.6.5 Overriding at Runtime

- **Override CMD:** supply new arguments after the image name.

```
docker run myimage --version
```

If the Dockerfile is:

```
ENTRYPOINT ["python", "/app/my_script.py"]  
CMD ["--help"]
```

Then the above command runs: `python /app/my_script.py --version`

- **Override ENTRYPOINT:**

```
docker run --entrypoint /bin/bash myimage
```

5.5.7 EXPOSE

```
EXPOSE <port> [<port>/<protocol>...]
```

The `EXPOSE` instruction in a Dockerfile is a way of documenting which ports the container listens on at runtime. `EXPOSE` does not actually publish the port; it functions as a type of documentation between the person who builds the image and the person who runs the container.

- `<port>` specifies the port number that the container will listen on.
- `<protocol>` can be either `tcp` (default) or `udp`. If the protocol is not specified, `tcp` is assumed.

For example:

```
EXPOSE 80
```

This line in a Dockerfile indicates that the container is expected to listen on port 80 at runtime.

You can also specify multiple ports:

```
EXPOSE 80 443
```

Or specify the protocol explicitly:

```
EXPOSE 80/tcp 443/tcp
```

When you run a container using `docker run`, the `EXPOSE` instruction does not actually make the port accessible to the host. To do so, you need to provide the `-p` flag (for publish) in the `docker run` command, like so:

```
docker run -p 8080:80 myimage
```

In this example, Docker maps port 80 of the container to port 8080 of the host system.

The `EXPOSE` instruction is also useful when you are linking containers using legacy Docker container linking, or when defining what ports to expose in a Docker Compose file.

In summary, `EXPOSE` serves as documentation and a reminder of what ports are intended to be published when the container is run, but does not actually publish the port itself.

5.5.8 ENV

```
ENV <key>=<value>
```

The `ENV` instruction in a Dockerfile sets the environment variable `<key>` to the value `<value>` within the container. This variable will be available to all subsequent instructions in the build stage and can be used in the container at runtime as well.

You can also set multiple environment variables in one `ENV` instruction by providing each key-value pair:

```
ENV <key1>=<value1> <key2>=<value2> ...
```

For example:

```
ENV MY_VERSION=1.2.3
```

This command sets the environment variable `MY_VERSION` to the value `1.2.3`. Once set, it can be used in subsequent `RUN`, `CMD`, and `ENTRYPOINT` instructions. For example:

```
RUN echo $MY_VERSION
```

When the image is run as a container, the environment variable is also set within the container. You can check the value of the environment variable by running `echo $MY_VERSION` inside the container.

Here's another example setting multiple environment variables:

```
ENV MY_VERSION=1.2.3 PATH=/usr/local/bin:$PATH
```

In this example, `MY_VERSION` is set to `1.2.3`, and the `PATH` environment variable is appended with `/usr/local/bin`.

Environment variables are commonly used to:

- Configure settings for software in the container.
- Set paths to directories of interest.
- Define default values for container runtime.

Environment variables set using `ENV` are persistent; they exist inside the created container and any child images derived from it. If you need to pass environment variables only during the build and not have them persist in the built image, you would use build arguments (`ARG`) instead.

Remember, if an environment variable is set in the Dockerfile with `ENV`, it can be overridden when the container runs. This is done by passing the `-e` option in the `docker run` command:

```
docker run -e "MY_VERSION=2.0.0" myimage
```

This command will run a container from `myimage` where `MY_VERSION` is set to `2.0.0`, overriding the value set in the Dockerfile.

5.5.8.1 Environment Variable Best Practices

```
# Group related variables
ENV NODE_ENV=production \
    PORT=3000 \
    API_URL=https://api.example.com

# Use ARG for build-time only values
ARG BUILD_VERSION
ENV VERSION=${BUILD_VERSION:-1.0.0}

# Document environment variables
# ENV DATABASE_URL - Connection string for database
# ENV REDIS_HOST - Redis server hostname
```

5.6 Multi-Stage Builds

Multi-stage builds in Docker are a powerful feature that allows you to selectively copy artifacts from one stage to another, leaving behind everything you don't want in the final image. This results in a smaller, cleaner final image by separating the build-time dependencies from the runtime environment.

Benefits:

- **Reduce Image Size:** Multi-stage builds allow you to use one base image for building (with all the necessary build tools) and another, smaller base image for running the application.
- **Security:** By discarding unnecessary build artifacts, you minimize the attack surface of your final image.
- **Maintainability:** You can have both build and production environments in a single Dockerfile, which simplifies the management of the build pipeline.
- **Decoupling:** Multi-stage builds allow you to decouple application building and packaging stages, making it easier to reuse build stages for multiple projects.

Here's a walk-through of a Dockerfile using multi-stage builds for a hypothetical Node.js application:

```
# Stage 1: Build Stage
# Use the official Node.js 14 image as the build environment
FROM node:14 AS build-env
# Set the working directory within the container
WORKDIR /app
# Copy package.json and package-lock.json to the working directory
COPY package.json package-lock.json ./
```

```
# Install dependencies
RUN npm install
# Copy the entire application source code to the working directory
COPY . .
# Build the application
RUN npm run build

# Stage 2: Production Stage
# Use a smaller Node.js 14-slim image for the production stage
FROM node:14-slim
# Set the working directory within the container
WORKDIR /app
# Copy the built artifacts from the build environment to the production image
COPY --from=build-env /app/build ./build
# Copy package.json and package-lock.json to the working directory
COPY package.json package-lock.json ./
# Install only production dependencies
RUN npm install --production
# Expose port 3000
EXPOSE 3000
# Define the command to run when the container starts
CMD ["node", "build/index.js"]
```

- **Stage 1: Build Stage**

- We start with a `node:14` image which contains all the tools necessary to build the application.
- The application's source code and dependencies are copied and installed.
- The `npm run build` command builds the application.

- **Stage 2: Production Stage**

- We then start a new stage with `node:14-slim` which is a smaller image that doesn't contain build tools.
- Using the `COPY --from=build-env` instruction, we only copy the built application from the previous stage.
- We run `npm install --production` to install only the runtime dependencies necessary for the application to run.
- The application is then exposed on port 3000 and started with the `CMD` instruction.

By using multi-stage builds, you ensure that only the necessary artifacts required to run the application are included in the final image. The build dependencies and intermediate files produced during the build process are not included in the final image, resulting in a much cleaner and smaller image that is ideal for production deployment.

5.6.1 Advanced Multi-Stage Examples

```
# Example: Go application with minimal final image
FROM golang:1.20 AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o main .

FROM scratch
COPY --from=builder /app/main /main
ENTRYPOINT ["/main"]
```

```
# Example: Python app with virtual environment
FROM python:3.11 AS builder
WORKDIR /app
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.11-slim
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
WORKDIR /app
COPY . .
CMD ["python", "app.py"]
```

5.7 Efficient Dockerfiles

5.7.1 Minimizing Layers

Each instruction in a Dockerfile adds a new layer to the image, and each layer is cached. To create efficient Docker images, it's crucial to minimize the number of layers by combining instructions where possible. Here's how:

- **Combine related commands:** Group `RUN` instructions in a logical way using the shell's `&&` operator. This approach also helps in reducing intermediate images, thereby saving space.

```
RUN apt-get update && apt-get install -y git curl \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*
```

- **Minimize the contents of each layer:** Avoid adding unnecessary files. Only the files needed at each stage should be included.
- **Use multi-stage builds** (see above)

5.7.2 Optimizing Build Context

The build context is the set of files located in the directory from which the `docker build` command is run. Docker CLI sends this context to the Docker daemon, which can increase build time, especially if there are large or unnecessary files.

To optimize the build context:

- **Understand `.dockerignore`:** Like `.gitignore`, a `.dockerignore` file ensures that unwanted files are not sent to the daemon. Create a `.dockerignore` file in the root of the context directory.

```
# Sample .dockerignore file
node_modules
npm-debug.log
Dockerfile*
.dockerignore
.git
.gitignore
README.md
```

```
.env
.vscode
.idea
**/*.swp
**/*.bak
**/.DS_Store
```

- **Keep context small:** Store Dockerfiles outside of directories with large numbers of files when possible, and only add the necessary directories to the build context.

5.7.3 Using Build Arguments

Build arguments, specified by the **ARG** instruction, allow you to pass variables that can be used during the build of the image.

- **Declare an ARG:** At the top of your Dockerfile, before the first **FROM** instruction, or within a build stage.

```
ARG VERSION=latest
```

- **Use an ARG in instructions:** They can be used as variables in subsequent instructions in the Dockerfile.

```
FROM node:${VERSION}

ARG APP_DIR=/app

COPY . ${APP_DIR}
WORKDIR ${APP_DIR}
RUN npm install
```

- **Pass build arguments from the command line:** When building the Docker image, you can pass build-time variables.

```
docker build --build-arg VERSION=10.16.0 -t my-node-app .
```

Build arguments are not persisted in the final image, making them useful for passing secret information or environment-specific settings without bloating the image or leaving sensitive data in image layers.

5.7.3.1 ARG vs ENV

Feature	ARG	ENV
Scope	Build-time only	Build-time and runtime
Availability	Only during image build	In running containers
Override	<code>--build-arg</code> flag	<code>-e</code> flag or docker-compose
Persistence	Not in final image	Stored in image metadata
Use case	Build configuration, versions	Application configuration

Key Differences:

- **ARG** values are not available when you run a container - they only exist during the build process
- **ENV** values persist in the image and are available to running containers
- **ARG** is useful for sensitive build information that shouldn't be in the final image
- **ENV** can reference ARG values to make build-time values available at runtime

```
# ARG - Build-time only, not available in running container
ARG BUILD_DATE
ARG VERSION
RUN echo "Building version ${VERSION} on ${BUILD_DATE}"

# ENV - Available both during build and in running container
ENV APP_VERSION=${VERSION:-1.0.0}

# Common pattern: Use ARG to set ENV
ARG NODE_VERSION=18
FROM node:${NODE_VERSION}
ENV NODE_VERSION=${NODE_VERSION}
```

5.8 Security

5.8.1 Non-root User

Running containers as a non-root user enhances security by restricting the capabilities of any attackers who might compromise your container. Here's how to implement this:

- **Create a non-root user:** In your Dockerfile, add instructions to create a new user and group.

```
RUN groupadd -r appgroup && useradd -r -g appgroup appuser
```

- **Switch to non-root user:** Use the **USER** instruction to switch to the user you've created.

```
USER appuser
```

- **Permissions:** Ensure that the user has the appropriate permissions to read, write, and execute only what is necessary.

Complete Non-root Example

```
FROM node:18-alpine
# Create app directory with correct permissions
RUN mkdir -p /home/node/app && chown -R node:node /home/node/app
WORKDIR /home/node/app
# Switch to node user before copying files
USER node
# Copy with correct ownership
COPY --chown=node:node package*.json ./
RUN npm ci --only=production
COPY --chown=node:node . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Important considerations:

- Any files created before the **USER** instruction will be owned by root. Change ownership explicitly using **chown**.

- The node user (UID 1000) is already present in official Node.js images, so you do not need to create it.
- Ensure the application files and any directories the app writes to are owned by the non-root user.
- Some operations like binding to ports below 1024 require root privileges. Use higher port numbers or capabilities if needed.
- When using the `COPY` instruction in a Dockerfile, it's important to understand ownership behavior:
 - By default, `COPY` creates files with UID:GID 0:0 (root ownership) regardless of the `USER` instruction
 - Even when a non-root `USER` is specified in the Dockerfile, copied files will be owned by root unless explicitly changed
 - To set proper ownership during copy, use the `--chown` parameter in `COPY`

Secure Base Images

Base images are the foundation of your Docker image. Ensure they are secure:

- **Official Images**: Prefer official Docker images as they are maintained and regularly scanned **for** vulnerabilities.
- **Minimal Base Images**: Use minimal base images (like Alpine) that have fewer components and therefore, fewer potential vulnerabilities.
- **Regular Updates**: Regularly rebuild your images to include the latest base images, which will have the most recent security patches.
- **Scanning Tools**: Use image scanning tools to detect vulnerabilities in your images. Docker Hub offers security scanning that can be used to automatically scan images pushed to a repository.

Health Checks

Health checks are instructions in a Dockerfile that can be used to check the status of your application running inside a container.

- **Implementing `HEALTHCHECK`**: Add a `HEALTHCHECK` instruction in your Dockerfile to specify a command that determines the health of your container.

```
```docker
HEALTHCHECK --interval=5m --timeout=3s \
 CMD curl -f http://localhost/ || exit 1
```

#### • Parameters:

- `--interval=5m`: The time to wait between checks.
- `--timeout=3s`: The time to wait before considering the check to have hung.
- `--start-period=30s`: Grace period for containers that need time to bootstrap.
- `--retries=3`: Number of consecutive failures needed to consider container unhealthy.
- `CMD curl -f http://localhost/ || exit 1`: The command to check the health. If this command exits with a non-zero status, the container is considered unhealthy.

#### • Benefits:

- **Self-healing**: In orchestrated environments like Kubernetes, unhealthy containers can be automatically restarted.
- **Availability**: It ensures that traffic is only sent to healthy containers in load-balanced setups.

#### • Considerations:

- Do not rely solely on `HEALTHCHECK` for security.
- Combine it with monitoring solutions that can provide alerts and detailed health reports of your running containers.

#### 5.8.1.1 Advanced Health Check Examples

```
For a web application
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
 CMD node healthcheck.js || exit 1

For a database
HEALTHCHECK --interval=10s --timeout=5s --retries=5 \
 CMD pg_isready -U postgres || exit 1

Using wget instead of curl (Alpine)
HEALTHCHECK CMD wget --no-verbose --tries=1 --spider http://localhost
```



## 6 WORKDIR deep dive

### 6.1 Syntax

```
WORKDIR /path/to/workdir
```

The `WORKDIR` instruction is used to define the working directory of a Docker container at any given time. The working directory refers to the directory in which commands will be executed, and it affects multiple aspects of your container's behavior.

#### 6.1.1 What WORKDIR Affects

The `WORKDIR` instruction sets the working directory for any `RUN`, `CMD`, `ENTRYPOINT`, `COPY` and `ADD` instructions that follow it in the Dockerfile. This is crucial to understand - `WORKDIR` doesn't just affect `RUN` commands, but nearly all instructions that deal with paths:

- **RUN:** Commands execute in the `WORKDIR`
- **CMD:** The default command runs from `WORKDIR`
- **ENTRYPOINT:** The container's main process starts in `WORKDIR`
- **COPY:** Destination paths are relative to `WORKDIR` (if not absolute)
- **ADD:** Destination paths are relative to `WORKDIR` (if not absolute)

#### 6.1.2 Directory Creation Behavior

If the `WORKDIR` doesn't exist, it will be created even if it's not used in any subsequent Dockerfile instruction. This means:

```
This automatically creates /app if it doesn't exist
WORKDIR /app

No need to do this:
RUN mkdir -p /app # Unnecessary!
WORKDIR /app
```

If a directory already exists at the specified path, Docker simply uses it - no error occurs and existing contents remain untouched.

#### 6.1.3 Relative vs Absolute Paths

If a relative path is provided, it will be relative to the path of the previous `WORKDIR` instruction. Understanding this is critical:

```
WORKDIR /a
WORKDIR b
WORKDIR c
RUN pwd
Output: /a/b/c

But with absolute paths:
WORKDIR /a
WORKDIR /b
WORKDIR /c
RUN pwd
Output: /c
```

**Best Practice:** For clarity and reliability, you should always use absolute paths for your WORKDIR. This prevents confusion and makes your Dockerfile more maintainable.

## 6.1.4 Common Mistakes and How to Avoid Them

### 6.1.4.1 Mistake 1: Not Setting WORKDIR (Default to Root)

Forgetting to set WORKDIR: Leads to commands running in the root (/) directory, which can cause permission issues or unintentional file placement.

```
BAD - No WORKDIR set
FROM node:18
COPY . . # Copies to root directory!
RUN npm install # Runs in root directory!

GOOD - Explicit WORKDIR
FROM node:18
WORKDIR /app
COPY . . # Copies to /app
RUN npm install # Runs in /app
```

### 6.1.4.2 Mistake 2: Using cd in RUN Commands

Using CD in RUN commands: These commands only affect the shell used for that layer and do not persist. Always prefer WORKDIR for persistent directory context. Each RUN command starts fresh from the WORKDIR:

```
BAD - cd doesn't persist
RUN cd /app && npm install
RUN npm start # ERROR! Still in previous WORKDIR or root

GOOD - Use WORKDIR
WORKDIR /app
RUN npm install
RUN npm start # Works! Still in /app
```

You should use WORKDIR instead of proliferating instructions like RUN cd ... && do-something, which are hard to read, troubleshoot, and maintain.

### 6.1.4.3 Mistake 3: Confusing Build Context with Container Paths

You can think of WORKDIR like a cd inside the container (it affects commands that come later in the Dockerfile, like the RUN command). Remember that WORKDIR has nothing to do with your host machine's directory structure:

```
Host machine structure:
/home/user/myproject/
└── Dockerfile
└── src/
└── package.json

FROM node:18
WORKDIR /app # This is INSIDE the container, not on host

COPY source is relative to build context (host)
COPY destination is relative to WORKDIR (container)
COPY package.json . # Copies from host ./package.json to container /app/package.json
COPY src ./src # Copies from host ./src to container /app/src
```

### 6.1.4.4 Mistake 4: Multiple WORKDIR Without Understanding Path Resolution

```
Confusing example
WORKDIR /usr
WORKDIR src # Now in /usr/src
WORKDIR /app # Now in /app (absolute path)
WORKDIR data # Now in /app/data
```

Always trace through your WORKDIR instructions to understand where you are: - Absolute paths (starting with /) reset the working directory completely - Relative paths append to the current working directory

### 6.1.4.5 Mistake 5: Not Aligning WORKDIR with Application Expectations

Your application might expect to run from a specific directory:

```
BAD - Application expects to be in /app but WORKDIR is different
WORKDIR /usr/src
COPY . .
CMD ["node", "server.js"] # server.js might look for ./config/app.json

GOOD - Match application's expectations
WORKDIR /app
COPY . .
CMD ["node", "server.js"] # Now ./config/app.json is at /app/config/app.json
```

## 6.1.5 Using Environment Variables with WORKDIR

The WORKDIR instruction can resolve environment variables previously set using ENV. You can only use environment variables explicitly set in the Dockerfile.

```
ENV APP_HOME=/application
ENV MODULE_NAME=backend

WORKDIR $APP_HOME
Now in /application
```

```

WORKDIR $MODULE_NAME
Now in /application/backend

This also works:
ENV DIRPATH=/path
WORKDIR $DIRPATH/$MODULE_NAME
RUN pwd # Output: /path/backend

```

**Note:** Variables must be defined with ENV before using them in WORKDIR. Shell variables from RUN commands won't work:

```

This DOESN'T work:
RUN export MY_DIR=/app
WORKDIR $MY_DIR # MY_DIR is not defined for WORKDIR

This DOES work:
ENV MY_DIR=/app
WORKDIR $MY_DIR

```

### 6.1.6 WORKDIR and User Permissions

When running containers as non-root users, ensure the WORKDIR is accessible:

```

Create user and set up directory with proper permissions
FROM node:18
RUN useradd -m -u 1001 appuser

Create and set ownership BEFORE switching user
WORKDIR /app
RUN chown appuser:appuser /app

USER appuser
Now appuser can write to /app
COPY --chown=appuser:appuser . .

```

### 6.1.7 WORKDIR in Multi-Stage Builds

Each stage can have its own WORKDIR, and they don't affect each other:

```

Build stage
FROM node:18 AS builder
WORKDIR /build # Only affects builder stage
COPY . .
RUN npm run build

Production stage
FROM node:18-alpine
WORKDIR /app # Fresh WORKDIR for this stage
COPY --from=builder /build/dist ./dist # Must use full path from builder

```

### 6.1.8 Best Practices Summary

1. **Set WORKDIR early:** Set your main WORKDIR early in the Dockerfile, typically right after the FROM instruction.
2. **Use absolute paths:** Always use absolute paths for clarity (/app not app)

3. **Create consistent structure:** Establish a standard directory layout across your images
4. **Avoid redundant mkdir:** WORKDIR creates directories automatically
5. **Group related operations:** Use WORKDIR to logically group operations by directory
6. **Document your choice:** Add comments explaining why you chose specific directories

```
Good example incorporating best practices
FROM python:3.11-slim

Set working directory early and use absolute path
WORKDIR /app

Dependencies first (better layer caching)
COPY requirements.txt .
RUN pip install -r requirements.txt

Then application code
COPY src ./src
COPY config ./config

WORKDIR persists for CMD
CMD ["python", "src/main.py"]
```

### 6.1.9 Default Working Directory

If no WORKDIR is specified in the Dockerfile then the default path is /. However, the base image you're using might set its own WORKDIR. You can check the working directory of your base image:

```
Check the working directory of an image
docker run --rm alpine:latest pwd # Output: /
docker run --rm node:18 pwd # Output might be different
```

### 6.1.10 Runtime Override

The WORKDIR can be overridden at runtime:

```
Override WORKDIR when running container
docker run -w /custom/path myimage pwd
Or
docker run --workdir /custom/path myimage pwd
```

This is useful for debugging but shouldn't be relied upon for production use.

### 6.1.11 Debugging WORKDIR Issues

To debug WORKDIR-related problems:

```
Add debug statements
FROM node:18
WORKDIR /app
RUN pwd && ls -la # Shows current directory and contents
COPY . .
RUN pwd && ls -la # Verify files were copied correctly
```

Or inspect the image configuration:

```
docker image inspect myimage | grep -A 2 "WorkingDir"
```

Remember: In regular command line, if you `cd` somewhere, it stays there until you change it. However, in a Dockerfile, each `RUN` command starts back at the root directory! This is why `WORKDIR` is essential - it provides persistent directory context across instructions.

## 7 COPY deep dive

### 7.1 Syntax

```
COPY [--chown=<user>:<group>] [--chmod=<perms>] <src>... <dest>
```

The `COPY` instruction copies files and directories from the build context (the set of files located in the directory from which the `docker build` command is run) into the Docker image. Understanding `COPY`'s behavior is crucial as it's one of the most error-prone instructions in Dockerfiles.

### 7.2 Understanding Build Context

Before diving into `COPY`, you must understand build context. When copying source files from the build context, paths are interpreted as relative to the root of the context. The build context is the directory you specify when running `docker build`:

```
docker build -t myimage . # Current directory is the build context
docker build -t myimage /path/to/context # /path/to/context is the build context
```

**Critical Points:** - `COPY` can only access files within the build context - You cannot use `COPY ../something` to access parent directories - The Dockerfile itself doesn't have to be in the build context (you can use `-f`)

### 7.3 Source Path Behavior

#### 7.3.1 Leading Slashes and Parent Navigation

Specifying a source path with a leading slash or one that navigates outside the build context, such as `COPY ../something /something`, automatically removes any parent directory navigation (`../`). This is a safety feature:

```
These won't work as expected:
COPY /etc/passwd /app/ # Leading slash is removed, looks for etc/passwd in context
COPY ../config.json /app/ # Parent navigation removed, looks for config.json in context
COPY ../../data /app/ # Becomes COPY data /app/
```

#### 7.3.2 Trailing Slashes in Source

Trailing slashes in the source path are also disregarded, making `COPY something/ /something` equivalent to `COPY something /something`:

```
These are equivalent:
COPY src/ /app/
COPY src /app/
```

## 7.4 Destination Path Rules

### 7.4.1 The Trailing Slash Significance

In this case, if ends with a trailing slash /, it will be considered a directory and the contents of will be written at /base(). If does not end with a trailing slash, it will be considered a regular file and the contents of will be written at .

This is one of the most common sources of confusion:

```
Copying a single file
COPY file.txt /app/file.txt # file.txt becomes /app/file.txt
COPY file.txt /app/ # file.txt becomes /app/file.txt (directory assumed)
COPY file.txt /app # file.txt becomes /app (RENAMED to 'app'!)

If /app exists as a directory:
COPY file.txt /app # file.txt becomes /app/file.txt
If /app doesn't exist:
COPY file.txt /app # file.txt becomes /app (a file named 'app')
```

### 7.4.2 Multiple Sources Require Trailing Slash

If multiple resources are specified, either directly or due to the use of a wildcard, then must be a directory, and it must end with a slash /:

```
ERROR - multiple sources need trailing slash on destination
COPY file1.txt file2.txt /app

CORRECT
COPY file1.txt file2.txt /app/

ERROR - wildcards mean multiple sources
COPY *.json /app

CORRECT
COPY *.json /app/
```

## 7.5 Directory Copying Behavior

If the source is a directory, the contents of the directory are copied, including filesystem metadata. The directory itself isn't copied, only its contents. This is a critical distinction:

```
Directory structure:
myapp/
| src/
| | main.py
| config/
| | settings.json
```

```
This copies CONTENTS of myapp, not myapp itself
COPY myapp /app/
Result in container:
/app/
└── src/
└── main.py
└── config/
└── settings.json

To preserve the directory name:
COPY myapp /app/myapp/
Or:
COPY myapp /myapp
```

## 7.6 File Overwriting Behavior

If it contains subdirectories, these are also copied, and merged with any existing directories at the destination. Any conflicts are resolved in favor of the content being added, on a file-by-file basis, except if you're trying to copy a directory onto an existing file, in which case an error is raised.

```
Existing container content:
/app/config.json (existing file)
/app/data/ (existing directory)

COPY config.json /app/ # Overwrites existing config.json
COPY data/ /app/data/ # Merges with existing directory
COPY mydir /app/config.json # ERROR! Can't copy directory to file
```

## 7.7 Common Mistakes and Solutions

### 7.7.1 Mistake 0: Paths with Spaces

If either source or destination includes white space, enclose the path in square brackets and double quotes:

```
Windows with spaces in path
COPY ["My Docs/file.txt", "/opt/My App/"]
```

### 7.7.2 Mistake 1: Missing Trailing Slash with Multiple Files

```
ERROR - Docker doesn't know if config is a file or directory
COPY package.json package-lock.json config

CORRECT - explicitly indicate directory
COPY package.json package-lock.json config/
```

### 7.7.3 Mistake 2: Expecting Directory Name to be Preserved

```
Source structure: src/app.py

WRONG expectation - creates /app/app.py, not /app/src/app.py
COPY src /app

CORRECT - preserve directory name
COPY src /app/src
Or copy contents only:
COPY src/ /app/
```

### 7.7.4 Mistake 3: Using Absolute Paths from Host

```
WRONG - Can't use host absolute paths
COPY /home/user/project/config.json /app/

CORRECT - Use relative to build context
COPY config.json /app/
```

### 7.7.5 Mistake 4: Build Context Confusion

```
Project structure:
/home/user/project/
└── docker/
└── Dockerfile
└── src/
└── app.py

If running from /home/user/project/docker/:
docker build -t myapp .

This FAILS - src is not in build context
COPY src/app.py /app/

Solution 1: Run from project root
cd /home/user/project
docker build -f docker/Dockerfile -t myapp .
Now this works:
COPY src/app.py /app/

Solution 2: Use parent directory as context
docker build -t myapp ..
```

### 7.7.6 Mistake 5: Inefficient Layer Ordering

Docker builds images in layers, and each RUN, COPY, or ADD instruction creates a new layer. Inefficient ordering can lead to longer build times:

```
BAD - Any code change invalidates dependency cache
COPY . /app
RUN npm install

GOOD - Dependencies cached separately
COPY package*.json /app/
RUN npm install
COPY . /app
```

### 7.7.7 Mistake 6: Wildcards and Hidden Files

```
This doesn't copy hidden files (.env, .gitignore, etc.)
COPY * /app/

This copies everything including hidden files
COPY . /app/
```

## 7.8 Advanced COPY Usage

### 7.8.1 Ownership and Permissions

```
Copy with ownership change (reduces layers)
COPY --chown=user:group source destination

Copy with specific permissions (BuildKit required)
COPY --chmod=755 script.sh /usr/local/bin/

Combined
COPY --chown=node:node --chmod=644 config.json /app/
```

### 7.8.2 Copying from Multi-Stage Builds

```
First stage
FROM golang:1.19 AS builder
WORKDIR /build
COPY . .
RUN go build -o myapp

Second stage
FROM alpine:latest
Copy from previous stage
COPY --from=builder /build/myapp /usr/local/bin/
```

### 7.8.3 Using .dockerignore

Create a `.dockerignore` file to exclude files from the build context:

```
.dockerignore
node_modules/
*.log
.git/
.env
temp/
*.md
!README.md # Exception - include README.md
```

\* = one level, \*\* = any depth, / = path separator or root anchor.

## 7.9 Platform-Specific Considerations

### 7.9.1 Building Linux Containers on Different Platforms

Whether you're on Windows (PowerShell/WSL2), Mac, or Linux, you're building Linux containers, so paths inside the container always use Linux conventions:

```
Always use forward slashes in Dockerfile (Linux container paths)
WORKDIR /app
COPY src/ /app/src/
RUN chmod +x /app/entrypoint.sh
```

### 7.9.2 File Permissions and Line Endings

When developing on Windows but deploying Linux containers: - File permissions set on Windows may not translate correctly - Watch out for CRLF vs LF line ending issues

```
Good practice for cross-platform development
COPY --chmod=755 entrypoint.sh /usr/local/bin/
Convert line endings if needed (when files come from Windows)
RUN apt-get update && apt-get install -y dos2unix && \
 dos2unix /usr/local/bin/entrypoint.sh
```

### 7.9.3 Using .dockerignore Cross-Platform

The `.dockerignore` file works the same across platforms, but be aware of case sensitivity: - Windows/-Mac filesystems are typically case-insensitive - Linux is case-sensitive

```
.dockerignore
node_modules/
*.log
.git/
.env
Temp/ # Won't match 'temp/' on Linux
temp/ # Add both to be safe
```

## 7.10 Best Practices Summary

1. **Always use trailing slashes for directories** to be explicit about your intent
2. **Copy specific files before directories** for better cache utilization
3. **Use .dockerignore** to exclude unnecessary files
4. **Order COPY instructions from least to most frequently changing**
5. **Use -chown with COPY** instead of separate RUN chown (saves a layer)
6. **Prefer COPY over ADD** unless you need ADD's special features
7. **Use forward slashes** even on Windows
8. **Be explicit about file vs directory** destinations

## 7.11 Debugging COPY Issues

```
Debug what's in your build context
RUN ls -la /

Check what was copied
COPY . /app/
RUN ls -la /app/

Verify file permissions
COPY --chown=1000:1000 app /app/
RUN stat /app/app

Check current working directory impact
WORKDIR /workspace
COPY . . # Copies to /workspace
RUN pwd && ls -la
```

## 7.12 Quick Reference

```
Single file to specific location
COPY file.txt /app/file.txt

Single file to directory (trailing slash required for clarity)
COPY file.txt /app/

Multiple files (MUST have trailing slash on destination)
COPY file1.txt file2.txt /app/

Directory contents only
COPY src/ /app/

Directory itself (preserving directory name)
COPY src /app/src

Everything from current context
COPY . /app/

With ownership (Linux containers)
COPY --chown=node:node . /app/

With permissions (requires BuildKit)
COPY --chmod=755 script.sh /bin/

From another stage in multi-stage build
COPY --from=builder /build/app /app/

Using wildcards (destination must have trailing slash)
COPY *.json /app/

Files with spaces (use quotes and brackets)
COPY ["my file.txt", "/app/my file.txt"]
```

Remember: When in doubt, use trailing slashes for directory destinations and test your build with a minimal example first!



## 8 docker storage

### 8.1 Introduction

By default, files created inside a container are stored on a *writable container layer*, which means:

- Data doesn't persist after the container stops, making it hard to retrieve.
- The writable layer is tied to the host machine, complicating data movement.
- Writing to this layer requires a storage driver, which can reduce performance compared to using data volumes.

Docker offers two options for persistent storage: **volumes** and **bind mounts**. Additionally, files can be stored in-memory using **tmpfs** mounts, which are not persisted.

### 8.2 Mount types

Regardless of the mount type, data appears as a directory or file in the container's filesystem. Here's a quick overview:

- **Volumes:** Managed by Docker and stored in `/var/lib/docker/volumes/`. They are the best choice for data persistence.
- **Bind mounts:** Can be stored anywhere on the host and may include important system files. They can be modified by both Docker and non-Docker processes.
- **tmpfs mounts:** Stored in memory only and not written to disk.

### 8.3 Mount type details

- **Volumes:** Created and managed by Docker. They can be named or anonymous and support volume drivers for remote storage options.
- **Bind mounts:** Allow direct access to host files, which can pose security risks as they enable modification of critical files.
- **tmpfs mounts:** Used for temporary storage of non-persistent data or sensitive information.

For mounting, use the `-v` or `--volume` flag for bind mounts and volumes, and the `--tmpfs` flag for **tmpfs** mounts. The `--mount` flag is recommended for clarity across all types.

## 8.4 Volume use cases

Volumes are the preferred method for persisting data in Docker containers. Key use cases include:

- **Data Sharing:** Volumes allow multiple containers to share data, persisting even after the container stops or is removed.
- **Decoupling Configuration:** They help separate the Docker host's file structure from container runtime requirements.
- **Remote Storage:** Volumes can store data on remote hosts or cloud providers.
- **Backup and Migration:** Easier to back up and migrate data between Docker hosts.
- **High-Performance I/O:** Volumes stored in the Linux VM provide lower latency and higher throughput.
- **Native File System Behavior:** Ideal for applications needing precise control over disk operations, such as databases.

## 8.5 bind mount use cases

In general, prefer using volumes. Bind mounts are suitable for:

- Sharing configuration files, such as `/etc/resolv.conf`, for DNS resolution.
- Sharing source code or build artifacts, like Maven's `target/` directory, between the host and containers.
- Ensuring production Dockerfiles copy artifacts directly into images instead of relying on bind mounts.
- Maintaining a consistent file or directory structure on the host for required bind mounts.

## 8.6 tmpfs mount use cases

`tmpfs` mounts are ideal for scenarios where data should not persist on the host or within the container, often for security or performance reasons when handling large volumes of non-persistent data.

## 8.7 Tips for using bind mounts or volumes

If you use bind mounts or volumes, consider the following:

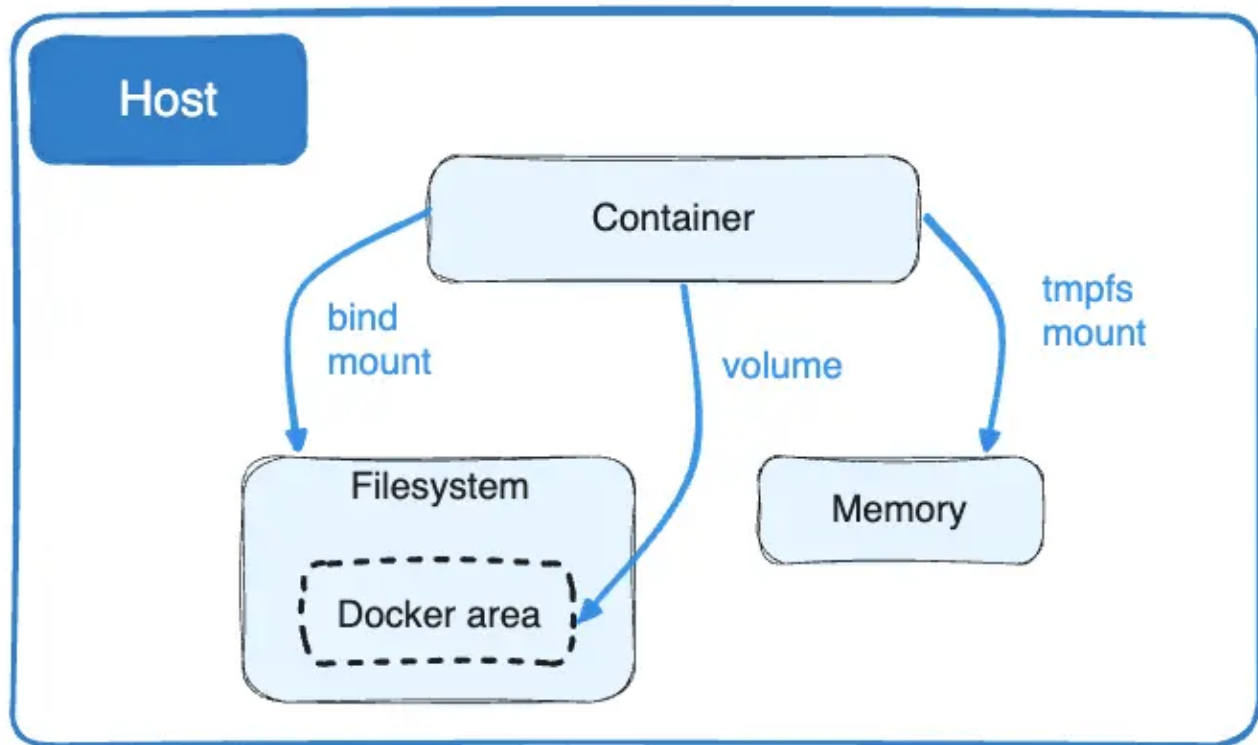
- Mounting an **empty volume** into a directory with existing files will copy those files into the volume. If the volume doesn't exist, Docker creates an empty one, allowing for pre-population of data.
- Mounting a **bind mount** or **non-empty volume** into a directory with existing files will obscure those files, similar to mounting a USB drive over a directory. The original files remain intact but are inaccessible while the mount is active.

## 8.8 Volumes

Volumes are the preferred method for persisting data in Docker containers. Unlike bind mounts, which depend on the host's directory structure, volumes are fully managed by Docker and offer several advantages:

- Easier to back up or migrate.
- Managed via Docker CLI or API.
- Safer sharing among multiple containers.
- Support for volume drivers for remote storage and encryption.
- Can be pre-populated with content from a container.

Volumes are also better than using a container's writable layer, as they don't increase container size and exist independently of container lifecycle.



**Figure 8.1:** Alt text

For non-persistent state data, consider using a `tmpfs` mount to enhance performance and avoid permanent storage.

### 8.8.1 Choose the `-v` or `--mount` flag (again)

The `--mount` flag is more explicit and allows for specifying volume driver options, while `-v` combines options into a single field. Here's a comparison:

- `-v` or `--volume`: Consists of three fields separated by colons (:):
  - For named volumes, the first field is the volume name (unique on the host). For anonymous volumes, this field is omitted.
  - The second field is the mount path in the container.
  - The third field (optional) is a comma-separated list of options (e.g., `ro`).
- `--mount`: Consists of key-value pairs separated by commas:
  - `type`: Can be `bind`, `volume`, or `tmpfs`.

- **source**: For named volumes, this is the volume name; for anonymous volumes, this field is omitted.
- **destination**: The mount path in the container.
- **readonly**: If present, mounts the bind as read-only (can be **readonly** or **ro**).
- **volume-opt**: Can be specified multiple times as key-value pairs.

**Warning:** If your volume driver accepts a comma-separated list, escape it by surrounding it with double quotes (") and the entire mount parameter with single quotes (').

### 8.8.2 Create and manage volumes

Unlike a bind mount, you can create and manage volumes outside the scope of any container.

Create a volume:

```
docker volume create my-vol
```

List volumes:

```
docker volume ls
```

```
local my-vol
```

Inspect a volume:

```
docker volume inspect my-vol
[
 {
 "Driver": "local",
 "Labels": {},
 "Mountpoint": "/var/lib/docker/volumes/my-vol/_data",
 "Name": "my-vol",
 "Options": {},
 "Scope": "local"
 }
]
```

Remove a volume:

```
docker volume rm my-vol
```

### 8.8.3 Start a container with a volume

If you start a container with a volume that doesn't yet exist, Docker creates the volume for you. The following example mounts the volume **myvol2** into **/app/** in the container.

```
docker run -d --name devtest --mount source=myvol2,target=/app nginx:latest
```

or

```
docker run -d --name devtest -v myvol2:/app nginx:latest
```

Use **docker inspect devtest** to verify that Docker created the volume and it mounted correctly. Look for the Mounts section:

```
"Mounts": [
 {
 "Type": "volume",
 "Name": "myvol2",
 "Source": "/var/lib/docker/volumes/myvol2/_data",
 "Destination": "/app",
 "Driver": "local",
 "Mode": "",
 "RW": true,
 "Propagation": ""
 }
],
```

This shows that the mount is a volume, it shows the correct source and destination, and that the mount is read-write.

Stop the container and remove the volume. Note volume removal is a separate step.

```
docker container stop devtest
docker container rm devtest
docker volume rm myvol2
```

#### 8.8.4 Populate a volume using a container

If you start a container which creates a new volume, and the container has files or directories in the directory to be mounted such as `/app/`, Docker copies the directory's contents into the volume. The container then mounts and uses the volume, and other containers which use the volume also have access to the pre-populated content.

To show this, the following example starts an nginx container and populates the new volume `nginx-vol` with the contents of the container's `/usr/share/nginx/html` directory. This is where Nginx stores its default HTML content.

The `--mount` and `-v` examples have the same end result.

```
docker run -d --name=nginxtest --mount source=nginx-vol,destination=/usr/share/nginx/html nginx:latest
```

or

```
docker run -d --name=nginxtest -v nginx-vol:/usr/share/nginx/html nginx:latest
```

Use `docker inspect nginxtest` to verify that Docker created the volume and it mounted correctly. Look for the Mounts section:

```
"Mounts": [
 {
 "Type": "volume",
 "Name": "nginx-vol",
 "Source": "/var/lib/docker/volumes/nginx-vol/_data",
 "Destination": "/usr/share/nginx/html",
 "Driver": "local",
 "Mode": "z",
 "RW": true,
 "Propagation": ""
 }
]
```

```
],
```

After running either of these examples, run the following commands to clean up the containers and volumes. Note volume removal is a separate step.

```
docker container stop nginxtest
docker container rm nginxtest
docker volume rm nginx-vol
```

### 8.8.5 Use a read-only volume

For development applications, containers may need to write to a bind mount to propagate changes to the Docker host, or they may only require read access. Multiple containers can share the same volume, allowing some to mount it as read-write while others mount it as read-only.

To mount a directory as read-only, add `ro` to the options after the mount point. Separate multiple options with commas.

The `--mount` and `-v` examples have the same result.

```
docker run -d --name=nginxtest --mount source=nginx-vol,destination=/usr/share/nginx/html,readonly nginx:latest
```

or

```
docker run -d --name=nginxtest -v nginx-vol:/usr/share/nginx/html:ro nginx:latest
```

Use `docker inspect nginxtest` to verify that Docker created the read-only mount correctly. Look for the Mounts section:

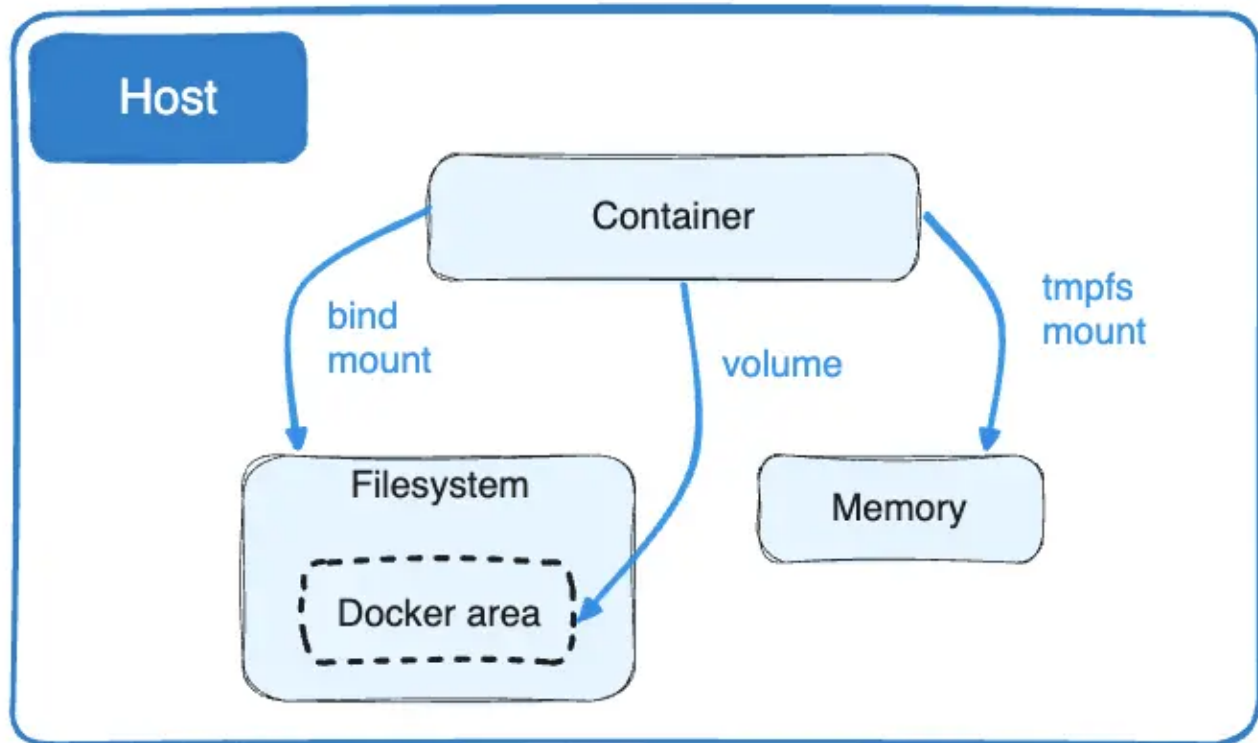
```
"Mounts": [
 {
 "Type": "volume",
 "Name": "nginx-vol",
 "Source": "/var/lib/docker/volumes/nginx-vol/_data",
 "Destination": "/usr/share/nginx/html",
 "Driver": "local",
 "Mode": "",
 "RW": false,
 "Propagation": ""
 }
],
```

Stop and remove the container, and remove the volume. Volume removal is a separate step.

```
docker container stop nginxtest
docker container rm nginxtest
docker volume rm nginx-vol
```

## 8.9 Bind mounts

Bind mounts allow you to mount a file or directory from the host into a container using its absolute path. Unlike volumes, which are managed by Docker, bind mounts reference the host's filesystem directly. They are performant but depend on the host's directory structure. For new Docker applications, consider using named volumes instead, as bind mounts cannot be managed with Docker CLI commands.



**Figure 8.2:** Alt text

### 8.9.1 Choose the `-v` or `--mount` flag

- `-v` or `--volume`: Uses three fields separated by colons (:):
  - First field: path on the host.
  - Second field: path in the container.
  - Third field (optional): options like `ro`.
- `--mount`: Uses key-value pairs separated by commas:
  - `type`: `bind`, `volume`, or `tmpfs`.
  - `source`: path on the host.
  - `destination`: path in the container.
  - `readonly`: mounts as read-only if specified.

### 8.9.2 Differences between `-v` and `--mount` behavior

The `-v` flag creates a directory if it doesn't exist, while `--mount` does not and will generate an error.

### 8.9.3 Start a container with a bind mount

To bind-mount the `target/` directory into your container at `/app/`, run:

```
docker run -d -it --name devtest --mount type=bind,source="$(pwd)"/target,target=/app
nginx:latest
```

or

```
docker run -d -it --name devtest -v "$(pwd)"/target:/app nginx:latest
```

Verify the bind mount with:

```
"Mounts": [
 {
 "Type": "bind",
 "Source": "/tmp/source/target",
 "Destination": "/app",
 "Mode": "",
 "RW": true,
 "Propagation": "rprivate"
 }
],
```

Stop the container:

```
docker container stop devtest
docker container rm devtest
```

### 8.9.4 Path Conversion on Windows

On Linux, mounting a path is straightforward. For example, the command:

```
docker run --rm -ti -v /home/user/work:/work alpine
```

creates a `/work` directory in the container that mirrors the specified path.

On Windows, you need to adjust the source path. For instance, using the legacy Windows shell (`cmd.exe`), you would run:

```
docker run --rm -ti -v C:\Users\user\work:/work alpine
```

This command starts the container and makes the volume usable, as Docker Desktop detects the Windows-style path and converts it for mounting.

You can also use Unix-style paths with Docker Desktop, like this:

```
docker run --rm -ti -v /c/Users/user/work:/work alpine ls /work
```

### 8.9.5 Mount into a Non-Empty Directory

When you bind-mount a directory into a non-empty directory in a container, the existing contents are obscured. This can be useful for testing new application versions without rebuilding the image.

For example, replacing the contents of the container's `/usr/` directory with the `/tmp/` directory on the host can lead to a non-functioning container:

```
docker run -d -it --name broken-container --mount type=bind,source=/tmp,target=/usr
nginx:latest
```

or

```
docker run -d -it --name broken-container -v /tmp:/usr nginx:latest
```

Output:

```
docker: Error response from daemon: oci runtime error: container_linux.go:262:
starting container process caused "exec: \"nginx\": executable file not found in $PATH".
```

The container is created but does not start. Remove it:

```
docker container rm broken-container
```

### 8.9.6 Use a Read-Only Bind Mount

For some applications, containers may need to write to a bind mount, while others may only require read access. To create a read-only bind mount, add `ro` to the options:

```
docker run -d -it --name devtest --mount type=bind,source="$(pwd)"/target,target=/app,
readonly nginx:latest
```

or

```
docker run -d -it --name devtest -v "$(pwd)"/target:/app:ro nginx:latest
```

Verify the bind mount with:

```
"Mounts": [
 {
 "Type": "bind",
 "Source": "/tmp/source/target",
 "Destination": "/app",
 "Mode": "ro",
 "RW": false,
 "Propagation": "rprivate"
 }
],
```

Stop the container:

```
docker container stop devtest
docker container rm devtest
```

## 8.10 tmpfs Mounts

`tmpfs` mounts allow temporary file storage in memory, not persisted after the container stops. This is useful for sensitive files that shouldn't persist.

### 8.10.1 Limitations of tmpfs Mounts

- Cannot share `tmpfs` mounts between containers.

- Permissions may reset after a restart.

### 8.10.2 The `--tmpfs` or `--mount` Flag

`--mount` is more explicit, while `--tmpfs` does not support configurable options.

To create a `tmpfs` mount at `/app` in a container:

```
docker run -d -it --name tmptest --mount type=tmpfs,destination=/app nginx:latest
```

or

```
docker run -d -it --name tmptest --tmpfs /app nginx:latest
```

Verify with:

```
docker inspect tmptest --format '{{ json .Mounts }}'
```

Stop and remove the container:

```
docker stop tmptest
docker container rm tmptest
```

### 8.10.3 Specify `tmpfs` Options

To set options like `tmpfs-mode`, use the `--mount` flag:

```
docker run -d -it --name tmptest --mount type=tmpfs,destination=/app,tmpfs-mode=1770
nginx:latest
```

## 9 docker networking

### 9.1 Introduction

Container networking enables communication between containers and non-Docker workloads. Each container perceives only its network interface, IP address, gateway, routing table, and DNS services, regardless of the network type or peer workloads, unless using the `none` network driver.

### 9.2 Published ports

By default, when you create or run a container using `docker create` or `docker run`, the container doesn't expose any of its ports to the outside world.

Use the `--publish` or `-p` flag to make a port available to services outside of Docker. This creates a firewall rule in the host, mapping a container port to a port on the Docker host to the outside world. Here are some examples:

Flag Value	Description
<code>-p 8080:80</code>	Map port 8080 on the Docker host to TCP port 80 in the container.
<code>-p 192.168.1.100:8080:80</code>	Map port 8080 on the Docker host IP 192.168.1.100 to TCP port 80 in the container.
<code>-p 8080:80/udp</code>	Map port 8080 on the Docker host to UDP port 80 in the container.
<code>-p 8080:80/tcp -p 8080:80/udp</code>	Map TCP port 8080 on the Docker host to TCP port 80 in the container, and map UDP port 8080 on the Docker host to UDP port 80 in the container.

#### Important

Publishing container ports is insecure by default. Meaning, when you publish a container's ports it becomes available not only to the Docker host, but to the outside world as well.

If you include the localhost IP address (127.0.0.1) with the publish flag, only the Docker host can access the published container port.

```
docker run -p 127.0.0.1:8080:80 nginx
```

If you want to make a container accessible to other containers, it isn't necessary to publish the container's ports. You can enable inter-container communication by connecting the containers to the same network, usually a bridge network.

## 9.3 IP address and hostname

By default, containers receive an IP address from each network they attach to. The Docker daemon handles dynamic IP allocation from the network's subnet.

Containers can only attach to one network at startup using `--network`, but can connect to additional networks via `docker network connect`. Use `--ip` or `--ip6` flags to specify IP addresses.

By default, a container's hostname is its container ID. Override this with `--hostname`, or use `--alias` when connecting to a network with `docker network connect`.

## 9.4 DNS services

Containers inherit DNS settings from the host's `/etc/resolv.conf`. Containers on the default `bridge` network receive a copy of this file, while those on custom networks use Docker's embedded DNS server, which forwards external lookups to the host's DNS servers.

Configure DNS per-container using flags with `docker run` or `docker create`:

Flag	Description
<code>--dns</code>	The IP address of a DNS server. To specify multiple DNS servers, use multiple <code>--dns</code> flags. If the container can't reach any of the IP addresses you specify, it uses Google's public DNS server at <code>8.8.8.8</code> . This allows containers to resolve internet domains.
<code>--dns-search</code>	A DNS search domain to search non-fully qualified hostnames. To specify multiple DNS search prefixes, use multiple <code>--dns-search</code> flags.
<code>--dns-opt</code>	A key-value pair representing a DNS option and its value. See your operating system's documentation for <code>resolv.conf</code> for valid options.
<code>--hostname</code>	The hostname a container uses for itself. Defaults to the container's ID if not specified.

### 9.4.1 Nameservers with IPv6 addresses

If the host's `/etc/resolv.conf` contains IPv6 nameserver entries, they're copied to containers.

Note: On Alpine Linux (musl libc), this can cause a race condition where hostname lookups sporadically fail if the external IPv6 DNS server responds faster than Docker's embedded DNS server.

## 9.4.2 Custom hosts

Custom hosts, defined in `/etc/hosts` on the host machine, aren't inherited by containers. You can add other hosts into a container's `/etc/hosts` file by using one or more `--add-host` flags. This example adds a static address for a host named `docker`:

```
docker run --add-host=docker:93.184.216.34 --rm -it alpine

/ # ping docker
PING docker (93.184.216.34): 56 data bytes
64 bytes from 93.184.216.34: seq=0 ttl=37 time=93.052 ms
64 bytes from 93.184.216.34: seq=1 ttl=37 time=92.467 ms
64 bytes from 93.184.216.34: seq=2 ttl=37 time=92.252 ms
^C
--- docker ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 92.209/92.495/93.052 ms
```

## 9.5 Network drivers

Docker's networking subsystem uses pluggable drivers that provide core networking functionality:

- **bridge**: Default driver for containers that need to communicate with other containers on the same host.
- **host**: Removes network isolation, allowing containers to use the host's networking directly.
- **ipvlan**: Provides full control over IPv4/IPv6 addressing and layer 2 VLAN tagging.
- **macvlan**: Assigns MAC addresses to containers, making them appear as physical devices on your network. Useful for legacy applications expecting direct physical network connections.
- **none**: Completely isolates a container from the host and other containers.
- **Network plugins**: Third-party plugins for specialized network stacks.

## 9.6 Bridge

A bridge network uses a software bridge to enable communication between containers on the same network while isolating them from other networks. The Docker bridge driver automatically prevents direct communication between containers on different bridge networks.

Docker creates a default bridge network automatically at startup. Containers connect to it by default unless specified otherwise. You can create user-defined bridge networks, which are recommended over the default bridge network.

### 9.6.1 user-defined bridges vs default bridge

**User-defined bridges** offer key advantages:

- Automatic DNS resolution between containers (default bridge requires IP addresses)
- Hot-pluggable network connections (default bridge requires container recreation)
- Better isolation and automatic port exposure between containers on the same network

### 9.6.2 Manage a user-defined bridge

Use the `docker network create` command to create a user-defined bridge network.

```
docker network create my-net
```

You can specify the subnet, the IP address range, the gateway, and other options. See the `docker network create` reference or the output of `docker network create --help` for details.

Use the `docker network rm` command to remove a user-defined bridge network. If containers are currently connected to the network, disconnect them first.

```
docker network rm my-net
```

### 9.6.3 Connect a container to a user-defined bridge

When you create a new container, you can specify one or more `--network` flags. This example connects a Nginx container to the `my-net` network. It also publishes port 80 in the container to port 8080 on the Docker host, so external clients can access that port. Any other container connected to the `my-net` network has access to all ports on the `my-nginx` container, and vice versa.

```
docker create --name my-nginx --network my-net --publish 8080:80 nginx:latest
```

To connect a running container to an existing user-defined bridge, use the `docker network connect` command. The following command connects an already-running `my-nginx` container to an already-existing `my-net` network:

```
docker network connect my-net my-nginx
```

### 9.6.4 Disconnect a container from a user-defined bridge

To disconnect a running container from a user-defined bridge, use the `docker network disconnect` command. The following command disconnects the `my-nginx` container from the `my-net` network.

```
docker network disconnect my-net my-nginx
```

### 9.6.5 Use the default bridge network

The default `bridge` network is considered a legacy detail of Docker and is not recommended for production use.

## 9.7 Host

If you use the `host` network mode, the container shares the host's networking namespace and does not receive its own IP address. For example, if a container binds to port 80, its application will be accessible on port 80 of the host's IP address.

## 9.8 IPvlan

The IPvlan driver provides full control over IPv4 and IPv6 addressing, while the VLAN driver enhances this by allowing complete control of layer 2 VLAN tagging and IPvlan L3 routing for underlay network integration.

## 9.9 Macvlan

Some applications, particularly legacy ones, require direct access to the physical network. In such cases, use the `macvlan` driver to assign a MAC address to each container's virtual interface, making it appear as a physical network interface. You'll need to specify a physical interface on the Docker host, along with the subnet and gateway for the Macvlan network. You can also isolate Macvlan networks using different physical interfaces.

## 9.10 None

If you want to completely isolate the networking stack of a container, you can use the `--network none` flag when starting the container. Within the container, only the loopback device is created.



# 10 Docker Compose

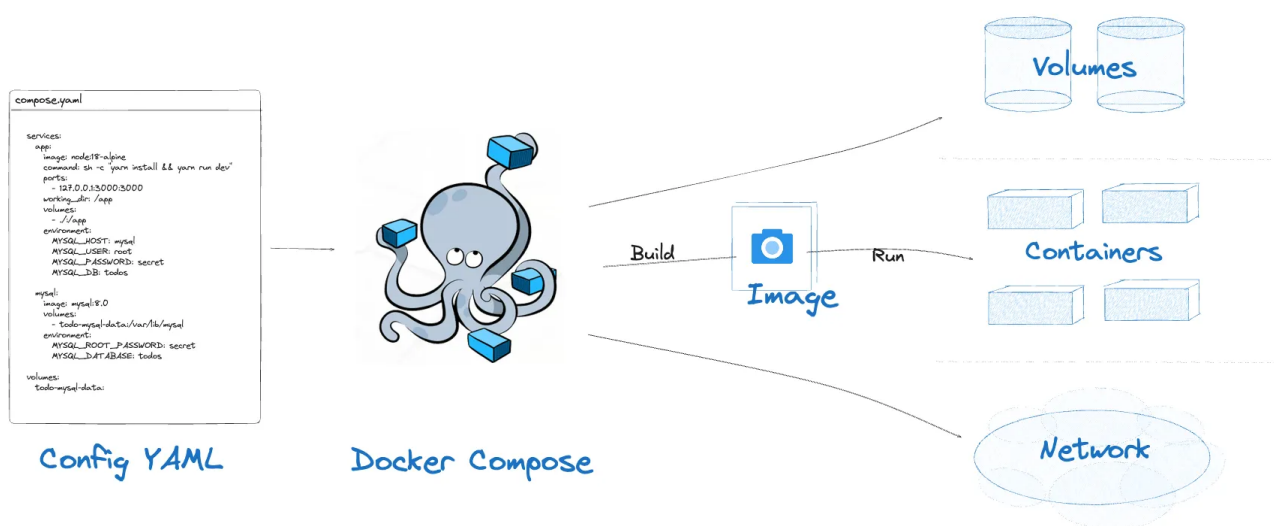
- Introduction
- Compose Specification and Compose File Essentials
  - The Compose File
  - Compose File Versions
  - Basic Structure
  - Services Section
  - Example Compose File
  - Multiple Compose Files and Overrides
- Docker Compose CLI
  - Starting and Stopping Services
  - Start services
  - Stop services
  - Start and run in detached mode
  - Managing Containers
  - Stopping and removing containers, networks, and volumes
  - Pausing services
  - Unpausing services
  - Logs and Monitoring
  - Viewing logs
  - Following log output
  - Scaling and Updating Services
  - Scaling a service
  - Rebuilding and restarting a service
  - Advanced Management
  - Removing stopped containers
  - Running a one-off command
  - Viewing the status of services
  - Pulling service images
- Service Definition
  - Defining a Service
  - Service Configuration Examples
  - Command
    - \* Overriding the Default Command
    - \* Syntax Styles
    - \* Interaction with Entrypoint

- [Ports and Networking](#)
  - [Environment Variables](#)
  - [Profiles](#)
- [Networks and Volumes](#)
  - [Networks in Compose](#)
  - [Volumes for Persistent Data](#)
  - [Combining Networks and Volumes](#)
  - [Networking Best Practices](#)
- [Network Types](#)
  - [Overview of Docker Network Types](#)
  - [Configuring Networks in Compose](#)
  - [Network Scopes and Drivers](#)
  - [Network Options and IPAM Configuration](#)
- [Service Discovery](#)
  - [Understanding Service Discovery](#)
  - [DNS-Based Service Discovery](#)
  - [Service Discovery Across Multiple Networks](#)
  - [Service Discovery Example](#)
- [Network Aliases](#)
- [Host Networking](#)
- [Volumes](#)
  - [1. Automatically Created Volumes](#)
  - [2. Bind Mounts](#)
  - [Volume Persistence](#)
- [Environment Variables](#)
  - [Using .env File](#)
  - [environment Attribute in Compose File](#)
  - [env\\_file Attribute in Compose File](#)
  - [Substituting via Shell](#)
  - [Using --env-file](#)
- [Restarting services](#)
- [Startup / Shutdown Order](#)
  - [Simple example](#)
  - [Example using service\\_started and service\\_healthy](#)
- [Compose build](#)
  - [Build Context](#)
    - \* [Short Syntax](#)
    - \* [Long Syntax](#)

- ★ Important Constraints
- ★ Specifying a Dockerfile Outside the Context
- Compose Secrets (AVOID!)
  - Why avoid this solution?
  - How should I inject secrets securely into my ci/cd pipelines?
  - Compose secrets explained

## 10.1 Introduction

Compose is a tool for defining and running multi-container Docker applications. With Compose, you use a YAML file to configure your application's services. Then, with a single command, you create and start all the services from your configuration.



**Figure 10.1:** alt text

Compose works in all environments; production, staging, development, testing, as well as CI workflows. It also has commands for managing the whole lifecycle of your application:

- Start, stop, and rebuild services
- View the status of running services
- Stream the log output of running services
- Run a one-off command on a service

The key features of Compose that make it effective are:

- Have multiple isolated environments on a single host
- Preserve volume data when containers are created
- Only recreate containers that have changed

- Support variables and moving a composition between environments
- 

## 10.2 Compose Specification and Compose File Essentials

Modern Docker Compose (Compose V2) is based on the Compose Specification. The old 2.x and 3.x file format versions have been unified into a single spec. New Compose files should follow this spec and usually omit the legacy `version` field.

### 10.2.1 The Compose File

- A Compose file `compose.yaml` or `compose.yml` is a YAML file that defines services, networks, and volumes for Docker applications.
- YAML is a human-readable data serialization standard that is ideal for configuration files.

### 10.2.2 Compose File Versions

With Compose V2 and the Compose Specification:

- You should omit the top-level `version` key and start the file directly with `services:`.
- Older files using `version: "2.x"` or `version: "3.x"` still work, but the `version` field is now legacy and ignored by modern Compose implementations.

For Compose V1, version 3 was commonly used for Docker Swarm, while 2 was suitable for standalone Docker environments. Docker Swarm is an alternative to Kubernetes that did not really make it.

### 10.2.3 Basic Structure

- A typical Compose file defines `services`, `networks`, and `volumes`.

Correct basic example:

```
services:
 web:
 image: nginx
 networks:
 - frontend
 volumes:
 - data_volume:/var/lib/data

networks:
 frontend:

volumes:
 data_volume:
```

What the Compose file does:

- Service: web

- Runs the official **nginx** image
- Connects to a custom network called **frontend**
- Mounts a named volume **data\_volume** into `/var/lib/data` (note: this is not the usual Nginx web content path)

#### 10.2.4 Services Section

- Services are the core part of the Compose file, defining the containers you want to run.
- Basic syntax includes the service name followed by its configuration (like `image`, `build`, `ports`).
- Example:

```
services:
 web:
 image: nginx
 ports:
 - "80:80"
```

#### 10.2.5 Example Compose File

- Here is a simple example illustrating a web service using the Nginx image:

```
services:
 web:
 image: nginx
 ports:
 - "80:80"
 volumes:
 - data_volume:/usr/share/nginx/html

networks:
 default:

volumes:
 data_volume:
```

#### 10.2.6 Multiple Compose Files and Overrides

In real projects you often combine multiple Compose files:

- `compose.yaml`: base configuration
- `compose.override.yaml`: local overrides (used automatically if present)
- Extra files per environment, for example:

```
docker compose -f compose.yaml -f compose.prod.yaml up -d
```

Compose merges the files from left to right; later files override earlier ones. This is commonly used to:

- Change images or environment variables per environment
- Add or remove services for production vs development

## 10.3 Docker Compose CLI

The Docker Compose CLI provides a suite of commands to manage the lifecycle of multi-container applications. Below is an overview of the main Docker Compose CLI commands with actual examples for common management tasks.

### 10.3.1 Starting and Stopping Services

#### 10.3.2 Start services

To start the services defined in `compose.yaml`, navigate to the directory containing the file and run:

```
docker compose up
```

#### 10.3.3 Stop services

To stop the services without removing them:

```
docker compose stop
```

#### 10.3.4 Start and run in detached mode

To start the services in the background:

```
docker compose up -d
```

#### 10.3.5 Managing Containers

#### 10.3.6 Stopping and removing containers, networks, and volumes

```
docker compose down
```

#### 10.3.7 Pausing services

Temporarily suspends running containers:

```
docker compose pause
```

#### 10.3.8 Unpausing services

Resumes paused containers:

```
docker compose unpause
```

### 10.3.9 Logs and Monitoring

#### 10.3.10 Viewing logs

To see the logs for running services:

```
docker compose logs
```

#### 10.3.11 Following log output

To tail the logs:

```
docker compose logs -f
```

#### 10.3.12 Scaling and Updating Services

#### 10.3.13 Scaling a service

To scale a service to a specific number of replicas:

```
docker compose up -d --scale [service_name]=[number_of_replicas]
```

For example, scaling a web service to 3 instances:

```
docker compose up -d --scale web=3
```

#### 10.3.14 Rebuilding and restarting a service

If you make changes to the service's configuration or Dockerfile:

```
docker compose up --build
```

#### 10.3.15 Advanced Management

#### 10.3.16 Removing stopped containers

```
docker compose rm
```

#### 10.3.17 Running a one-off command

For example, running a shell inside a container:

```
docker compose run [service_name] sh
```

#### 10.3.18 Viewing the status of services

```
docker compose ps
```

### 10.3.19 Pulling service images

```
docker compose pull
```

## 10.4 Service Definition

### 10.4.1 Defining a Service

- Each service in the `services` block represents a container.
- Basic elements include `image`, `build`, `command`, `ports`, `volumes`, `environment`, `depends_on`.

### 10.4.2 Service Configuration Examples

- Image-based service:

```
services:
 redis:
 image: redis
```

- Build-based service:

```
services:
 app:
 build: .
 ports:
 - "5000:5000"
```

The build context `.` refers to the current directory containing the Dockerfile.

### 10.4.3 Command

The `command` instruction allows you to override the default command declared in the container image (specifically, the `CMD` instruction in the Dockerfile).

This is valid for both services built from context and services using pre-existing images.

#### 10.4.3.1 Overriding the Default Command

When you define a `command` in your `compose.yaml`, Docker ignores the `CMD` defined in the Dockerfile and runs your specified command instead.

```
services:
 # Example 1: Overriding a built service
 backend:
 build: ./api
 # The Dockerfile might say 'CMD ["flask", "run"]'
```

```
But this service will run a migration script instead
command: python manage.py migrate

Example 2: Overriding a pre-built image
database:
 image: postgres:15
 # Override standard postgres startup options
 command: postgres -c 'max_connections=200'
```

### 10.4.3.2 Syntax Styles

You can specify the command as either a string (shell form) or a list (exec form).

**Shell Form (String):** Docker runs this command in a shell (`/bin/sh -c`). This allows you to use environment variable expansion provided by the shell.

```
command: python app.py --port=80
```

**Exec Form (List):** This runs the executable directly without a wrapping shell. This is generally preferred to ensure signals (like `SIGTERM` when stopping a container) are passed correctly to the process.

```
command: ["python", "app.py", "--port=80"]
```

### 10.4.3.3 Interaction with Entrypoint

It is important to understand the relationship between `command` and `entrypoint`.

- **Entrypoint:** The executable that starts the container.
- **Command:** The arguments passed to that executable.

If an image has an `ENTRYPOINT` defined, the `command` you write in `compose.yaml` effectively becomes the **arguments** for that entrypoint.

```
services:
 backup-tool:
 image: my-backup-tool
 # Dockerfile has: ENTRYPOINT ["/bin/backup"]

 # This results in executing: /bin/backup --dry-run
 command: --dry-run
```

If you need to override the executable itself, you must override the `entrypoint` in your Compose file as well.

### 10.4.4 Ports and Networking

- `ports` are used to expose container ports to the host.
- Syntax: `<host_port>:<container_port>`
- Example:

```
services:
 web:
 image: nginx
```

```
ports:
 - "80:80"
```

### 10.4.5 Environment Variables

- `environment` key is used to set environment variables inside the container.
- Example:

```
services:
 db:
 image: postgres
 environment:
 POSTGRES_PASSWORD: example
```

### 10.4.6 Profiles

Profiles allow you to enable or disable services conditionally without editing files:

```
services:
 db:
 image: postgres
 dev-tools:
 image: some/dev-image
 profiles:
 - dev
```

Run with a profile:

```
docker compose --profile dev up -d
```

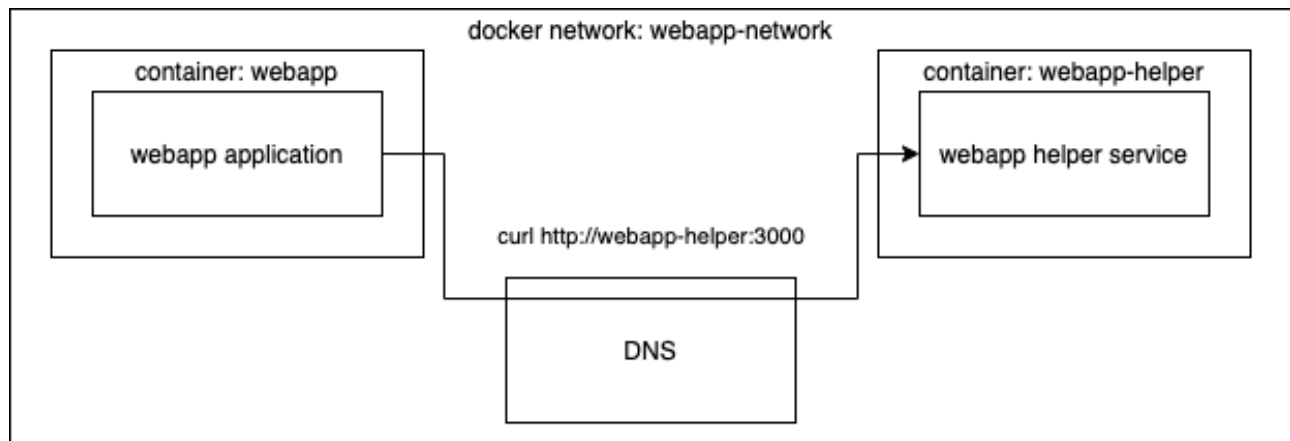
Services with profiles only start when their profile is active. This is useful for optional tools (debuggers, admin UIs, etc.) that you do not want in all environments.

---

## 10.5 Networks and Volumes

### 10.5.1 Networks in Compose

- Networks define how containers communicate with each other.
- By default, all services are added to a default network.



**Figure 10.2:** alt text

- Custom networks can be defined for specific communication requirements.
- Example:

```
networks:
 backend:

services:
 db:
 image: postgres
 networks:
 - backend
```

### 10.5.2 Volumes for Persistent Data

- Volumes are used for persisting data generated by and used by Docker containers.
- Defined under the `volumes` key at the top level and referenced by services.
- Example:

```
volumes:
 db_data:

services:
 db:
 image: postgres
 volumes:
 - db_data:/var/lib/postgresql/data
```

### 10.5.3 Combining Networks and Volumes

- Services can be configured to use both custom networks and volumes.
- Example:

```
services:
 db:
 image: postgres
 networks:
 - backend
 volumes:
```

```
- db_data:/var/lib/postgresql/data

networks:
 backend:

volumes:
 db_data:
```

### 10.5.4 Networking Best Practices

- Prefer internal service-to-service communication over private networks.
  - Avoid publishing ports unless the service must be reachable from outside the Compose project.
  - Databases, queues, and similar backing services usually should not expose host ports at all; other services should access them by service name over the internal network.
- 

## 10.6 Network Types

### 10.6.1 Overview of Docker Network Types

- Docker supports several network types: `bridge`, `host`, `overlay`, `macvlan`, and `none`.
- `bridge` is the default network type for containers.
- `host` network removes network isolation between the container and the Docker host.
- `overlay` networks are used in Docker Swarm (or other orchestrators) to connect multiple Docker daemons; they are not available in a plain single-host `docker compose up` setup.
- `macvlan` networks allow containers to appear as physical devices on your network.
- `none` disables all networking.

### 10.6.2 Configuring Networks in Compose

- Networks are defined at the top level of the Compose file and referenced by services.
- Example of defining a custom bridge network:

```
networks:
 my_bridge:
 driver: bridge

services:
 web:
 image: nginx
 networks:
 - my_bridge
```

### 10.6.3 Network Scopes and Drivers

- Network `scope` can be `swarm`, `global`, or `local`.
- Network `driver` determines how the network behaves.

### 10.6.4 Network Options and IPAM Configuration

- Customizing network options like `subnet`, `ip_range`, `gateway`.
  - IP Address Management (IPAM) configuration in Docker networks.
- 

## 10.7 Service Discovery

### 10.7.1 Understanding Service Discovery

- Service discovery allows Docker containers to locate each other and communicate.
- In Docker Compose, services can discover each other by service name.

### 10.7.2 DNS-Based Service Discovery

- Docker Compose uses an internal DNS server to resolve service names to container IP addresses.
- For example, a service named `db` can be reached at `db:3306` from other services.

Example:

```
services:
 web:
 image: nginx
 depends_on:
 - app
 app:
 build: .
 depends_on:
 - db
 db:
 image: postgres
```

### 10.7.3 Service Discovery Across Multiple Networks

- When services are connected to multiple networks, they can still discover each other, but there are some nuances.
- A service is only reachable on the networks it is explicitly connected to.
- If Service A is on Network 1 and Service B is on Network 2, they cannot communicate unless they share a common network or have some form of routing set up between the two networks.

### 10.7.4 Service Discovery Example

- Consider an example with two services, `service1` and `service2`, each in a separate network, `network1` and `network2`, respectively. We want these services to communicate with each other.

```
services:
 service1:
 image: some-image
 networks:
 - network1

 service2:
 image: another-image
 networks:
 - network2

networks:
 network1:
 network2:
```

In this setup, `service1` and `service2` are isolated in their respective networks.

- Add a shared network that both services are connected to, in addition to their separate networks, to enable communication between the services.

```
services:
 service1:
 image: some-image
 networks:
 - network1
 - shared_network

 service2:
 image: another-image
 networks:
 - network2
 - shared_network

networks:
 network1:
 network2:
 shared_network:
```

---

## 10.8 Network Aliases

- Network aliases in Docker allow you to assign alternative names to a service within a specific network context.
- This is particularly useful in scenarios where you have multiple instances of a service and need to balance the load or ensure high availability.
- Example:

```
services:
 db:
 image: postgres
 networks:
 default:
 aliases:
 - database
```

- In this example, the `db` service can be reached within the network using the alias `database`.

---

## 10.9 Host Networking

- Host networking mode (`network_mode: host`) is used to allow a container to share the host's networking namespace.
- This mode is typically used for performance-sensitive applications as it removes the network isolation between the container and the Docker host, but at the cost of security.
- On Linux, this behaves as described. On Docker Desktop (Mac/Windows), host networking has different semantics and is often not what you expect; check the platform-specific docs.
- Example:

```
services:
 high-performance-app:
 image: custom-app
 network_mode: host
```

- In this configuration, `high-performance-app` will have direct access to the host's network, bypassing the typical Docker-managed networking layers.
- 

## 10.10 Volumes

In Docker Compose, you can define volumes in two ways: as automatically created (managed) volumes, or as bind mounts which link directly to your host filesystem.

### 10.10.1 1. Automatically Created Volumes

To use an automatically created volume in Compose, define it under the top-level `volumes` key and then reference it in the service using the `volumes` key:

```
services:
 app:
 image: my-app
 volumes:
 - data_volume:/path/in/container

volumes:
 data_volume:
```

When you run `docker compose up`, Compose creates a new volume named `data_volume` if it does not exist and mounts it to `/path/in/container` inside the `app` service's containers.

### 10.10.2 2. Bind Mounts

To create a bind mount, you simply specify a path on the host in the `volumes` key of the service:

```
services:
 app:
 image: my-app
 volumes:
 - /path/on/host:/path/in/container
```

This mounts the host directory `/path/on/host` directly into the container at `/path/in/container`.

With both methods, the left side of the colon (`:`) specifies the source (either the volume name or the host path) and the right side specifies the target path inside the container.

### 10.10.3 Volume Persistence

Volumes do persist after you delete a Docker Compose deployment. This means if you run `docker compose down`, the services and network defined in your `compose.yaml` file will be stopped and removed, but the volumes will not be deleted by default. This persistent nature of volumes is by design, as it prevents data loss and allows you to reuse data across container restarts or even share it between different containers.

To remove volumes when you take down a Compose deployment, you need to use the `--volumes` or `-v` flag with the `down` command:

```
docker compose down --volumes
```

This command will remove not only the containers, networks, and the default network, but also the volumes defined in the `compose.yaml` file.

Remember, for bind mounts (which are directories from your host), no such deletion is necessary or possible from Docker Compose because they are managed by the host's filesystem and not Docker.

---

## 10.11 Environment Variables

General best practices:

- Do not commit real secrets into `compose.yaml` or `.env` files.
- Commit `.env.example` templates instead, and inject the real values via CI/CD or local shell environment.

### 10.11.1 Using .env File

A `.env` file is convenient for managing multiple environment variables. You place a `.env` file at the root of your project directory, and Docker Compose automatically picks up the variables defined within it when you run `docker compose up`. Here is an example of how you might use a `.env` file:

```
.env file
TAG=v1.5
```

```
compose.yaml
services:
 web:
 image: "webapp:${TAG}"
```

When the service starts, the `${TAG}` variable in `compose.yaml` is replaced with the value from the `.env` file, resulting in the image `webapp:v1.5` being used.

### 10.11.2 environment Attribute in Compose File

You can directly set environment variables for your services' containers using the `environment` attribute in your Compose file. This can either pass a variable's value explicitly or relay the value from your shell environment. For instance:

```
services:
 web:
 environment:
 - DEBUG=1
 - OTHER_VAR=${SOME_VAR}
```

This sets `DEBUG` to 1 and `OTHER_VAR` to the value of `SOME_VAR` from the shell environment. If `SOME_VAR` is not set, Compose will substitute an empty string and typically warn you.

### 10.11.3 env\_file Attribute in Compose File

The `env_file` attribute is used when you want to pass multiple environment variables from an external file to a service's containers. This is particularly useful when you want to maintain different environment files for different setups like staging, production, etc.

```
services:
 web:
 env_file:
 - web-variables.env
```

```
Example content of web-variables.env
DEBUG_MODE=true
DATABASE_HOST=db.production.example.com
DATABASE_PORT=5432
DATABASE_USER=prod_user
DATABASE_PASSWORD=prod_secret_password
API_KEY=abcdef12345
```

If you specify multiple files, they are evaluated in order, with the last file taking precedence in case of overlapping variable names.

### 10.11.4 Substituting via Shell

Docker Compose can substitute environment variables from the shell environment directly into the Compose file. Basic patterns:

- `${VAR}`: expands to the value of `VAR`, or the empty string if unset.
- `${VAR:-default}`: expands to `VAR` if set, otherwise to `default`.

- `${VAR:?message}`: fails with `message` if `VAR` is unset.

Example:

```
services:
 db:
 image: "postgres:${POSTGRES_VERSION:?POSTGRES_VERSION is required}"
```

If `POSTGRES_VERSION` is set in your shell to `9.3`, the `db` service will use the image `postgres:9.3`. If it is not set, Compose will abort with the given error message instead of silently using an invalid image reference.

Environment variables set in the shell where you run `docker compose` override those set in the `.env` file or with `--env-file` option.

### 10.11.5 Using `--env-file`

You can specify an environment file using the `--env-file` option when you run Docker Compose commands. This allows you to use different environment files without changing the Compose file itself. For example:

```
docker compose --env-file ./config/.env.dev up
```

This command tells Compose to use the `.env.dev` file from the `./config/` directory, which can override the default `.env` file at the project root.

---

## 10.12 Restarting services

The `restart` policy controls how Docker should handle restarts of a container under various conditions. Here are the options you can use in your `compose.yaml` file:

- `no`: Do not automatically restart the container when it exits.
- `always`: Always restart the container regardless of the exit status.
- `on-failure`: Restart the container only if it exits with a non-zero exit status.
- `unless-stopped`: Restart the container unless it has been stopped manually.

Here is an example of how to use the `restart` policy in a `compose.yaml` file:

```
services:
 web:
 image: nginx
 restart: always
```

In this example, the Nginx service will always be restarted if it stops for any reason, aiming for high availability.

## 10.13 Startup / Shutdown Order

You can control the order of service startup and shutdown with the `depends_on` option.

Compose always starts and stops containers in dependency order, where dependencies are determined by `depends_on`.

In Docker Compose, when services are started, they are considered “up” once the container itself is running. However, this does not mean the application within the container is ready to accept connections or fully operational, such as a database that may take additional time to initialize.

To handle this, Docker Compose offers a `depends_on` option that can specify conditions for starting services:

- `service_started`: Waits only for the container to be running. This is the default condition.
- `service_healthy`: Waits for the container to be in a “healthy” state, which you define in the `healthcheck` configuration for the service.
- `service_completed_successfully`: Used for one-off tasks, waits for a container to run to completion with a 0 exit status before starting dependent services.

Using `service_healthy` is a common way to ensure that a service like a database is ready to accept connections before another service (like a web application) starts.

Important nuance: `depends_on` only affects startup and shutdown ordering when you run `docker compose up` and `docker compose down`. It does not automatically restart or re-order services if a dependency crashes later at runtime.

Here are examples demonstrating how to use `depends_on` with different conditions in a `compose.yaml` file:

### 10.13.1 Simple example

```
services:
 web:
 build: .
 depends_on:
 - db
 - redis
 redis:
 image: redis
 db:
 image: postgres
```

We have three services defined: `web`, `redis`, and `db`.

- The `web` service is built from the Dockerfile in the current directory. It has dependencies on both the `db` and `redis` services, meaning it will only start after `db` and `redis` have started. There is no condition specified for their readiness, just that they need to be up and running.
- The `redis` service uses the official Redis image from Docker Hub.
- The `db` service uses the official PostgreSQL image from Docker Hub.

This setup is straightforward and does not consider whether `db` or `redis` are “ready” to handle connections; it only ensures that their containers start before the `web` service.

### 10.13.2 Example using `service_started` and `service_healthy`

```
services:
 web:
 build: .
 depends_on:
 db:
 condition: service_healthy
 redis:
 condition: service_started
 redis:
 image: redis
 db:
 image: postgres
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U postgres"]
 interval: 10s
 timeout: 5s
 retries: 5
```

The `healthcheck` field defines how Docker determines if a container is healthy. It runs a command at regular intervals:

- **test**: The command to execute.
  - format: `["CMD-SHELL", "command + arguments..."]`
  - The test is passed when the command exits with a status code of `0`.
- **interval**: How often to run the check (default: 30s).
- **timeout**: How long to wait for the command to complete (default: 30s).
- **retries**: Consecutive failures before marking unhealthy (default: 3).

The container is considered healthy only after passing the test, and unhealthy after exceeding the retry limit. Services waiting on `service_healthy` won't start until the dependency passes all health checks.

In this Compose setup:

- The `web` service is built from a Dockerfile in the current directory.
- It must wait for two services (`db` and `redis`) before starting.
  - It waits for the `db` service to be in a healthy state before starting. The health of `db` is determined by the `healthcheck` configuration which uses the `pg_isready` command to check if PostgreSQL is ready to accept connections.
  - It only waits for the `redis` service to start, without checking its health status.
- The `redis` service uses the default `redis` image from Docker Hub.
- The `db` service uses the default `postgres` image and checks its health every 10 seconds, with a 5-second timeout and up to 5 retries before considering the service unhealthy.

---

## 10.14 Compose build

```
services:
 frontend:
 image: example/webapp
 build: ./webapp
```

The `frontend` service is using an image named `example/webapp`. If this image is not found locally or in a registry, Docker will attempt to build it using the build context.

The `build` directive is given a path to the directory `./webapp`, which should contain a `Dockerfile`. Docker will use this Dockerfile to build the image for the `frontend` service.

### 10.14.1 Build Context

When defining a service that requires building an image from source, the `context` is one of the most critical configuration parameters. It defines the set of files and directories that are sent to the Docker daemon during the build process.

The build context dictates what files are accessible to the `COPY` and `ADD` commands inside your `Dockerfile`.

#### 10.14.1.1 Short Syntax

If your build configuration is simple, you can use a string to define the path to the build context. This path can be absolute or relative to the directory containing your `compose.yml` file.

```
services:
 webapp:
 # Sets the build context to the current directory
 build: .

 backend:
 # Sets the build context to the './api' subdirectory
 build: ./api
```

#### 10.14.1.2 Long Syntax

For more granular control, use the object syntax. This allows you to specify the `context` explicitly while also configuring other parameters, such as a specific `dockerfile` name or build arguments.

```
services:
 frontend:
 build:
 context: ./client
 dockerfile: Dockerfile.dev
```

In the example above:

- The **Context** is the `./client` directory.
- The **Dockerfile** is located at `./client/Dockerfile.dev`.

### 10.14.1.3 Important Constraints

1. **File Access:** The Docker builder cannot access files outside the specified build context. If your `Dockerfile` tries to `COPY ../some_file`, it will fail if `../some_file` is outside the directory tree defined by `context`.
2. **Performance:** The entire context is transferred to the Docker daemon before the build steps begin. Avoid setting the context to the root of your hard drive (`/`) or a directory containing large, unnecessary files (like `.git` or `node_modules`). Use a `.dockerignore` file in the context directory to exclude files from this transfer.

### 10.14.1.4 Specifying a Dockerfile Outside the Context

A common misconception is that the `dockerfile` directive must point to a file inside the build context. While this is the default behavior, you can point to a Dockerfile anywhere on your system.

However, even if the Dockerfile is outside the context, the commands inside that Dockerfile (like `COPY`) can **only** reference files strictly within the defined `context`.

```
services:
 worker:
 build:
 # The source code files are here
 context: ./src
 # The Dockerfile is one level up
 dockerfile: ../build_configs/Dockerfile.worker
```

In this scenario, `Dockerfile.worker` can `COPY` files from `./src`, but it cannot `COPY` a file sitting next to itself in `../build_configs`.

---

## 10.15 Compose Secrets (AVOID!)

### 10.15.1 Why avoid this solution?

Using Docker Compose for secrets is insecure because secrets are stored in plain text, lack encryption, and can easily leak through environment variables, which can be inspected, or version control. You do not want to expose your secrets to the world via github.

Instead, use dedicated secret management tools or solutions that offer encryption, access control, and lifecycle management. Examples are HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, Google Secret Manager, or a dedicated Kubernetes Secrets solution. Note that by default Kubernetes Secrets are only base64 encoded; they should be combined with encryption at rest and proper RBAC to be reasonably secure.

### 10.15.2 How should I inject secrets securely into my ci/cd pipelines?

Short answer: use secrets in GitHub Actions.

Although Docker Compose should *never* be used in a production set-up, it might be used in ci/cd pipelines. To securely inject secrets into CI/CD pipelines, follow these best practices:

- **Use Encrypted Secret Storage:** Store secrets in the CI/CD platform's secure storage (for example, GitHub Actions Secrets, Jenkins Credentials Store, GitLab CI/CD Variables) where they are encrypted at rest.
- **Access Secrets Dynamically:** Fetch secrets dynamically during the build or deployment stage using secure methods (for example, API calls to HashiCorp Vault, AWS Secrets Manager) with credentials stored securely in the CI/CD tool.
- **Avoid Hardcoding Secrets:** Never hardcode secrets in pipeline configuration files or scripts. Reference them using the platform's environment variable mechanism or secure plugins.
- **Mask Secrets in Logs:** Configure the CI/CD tool to automatically mask sensitive information in build and deployment logs.
- **Restrict Access:** Limit who and what can access the secrets by implementing fine-grained access controls.
- **Audit and Rotate Secrets Regularly:** Enable audit logging for secret access in the CI/CD tool and rotate secrets periodically or after personnel changes.

### 10.15.3 Compose secrets explained

Compose secrets involves understanding how to manage sensitive data, such as passwords or API keys, securely within your Compose deployments. Here is a step-by-step guide:

#### 1. Define Secrets in Compose File

- Use the top-level `secrets` element in your Compose file to define each secret.
- Specify the path to the file that contains the secret data.
- This definition does not give any service access to the secret; it merely makes it available to be granted access.

#### 2. Reference Secrets in Service Definitions

- Within each service that requires access to a secret, use the `secrets` attribute to list it.
- Docker Compose then grants access to these secrets on a per-service basis, which can be controlled via filesystem permissions within the container.

#### 3. Access Secrets in Containers

- Inside your containers, secrets are accessible at `/run/secrets/<secret_name>`.
- This approach is more secure than environment variables as it does not risk unintentional exposure through processes or logs.

In Docker Compose, when you define a secret and grant a service access to it, Docker internally creates a temporary file inside the container. The content of this file is the secret data you provided. This file is placed in a special directory, `/run/secrets/`.

For each secret, a file is created with the name you have assigned to the secret in your `compose.yaml` file. For example, if you have a secret named `my_secret`, inside the container, there will be a file at `/run/secrets/my_secret`. Your application can read this file to access the secret data.

This approach is more secure than plain environment variables, but you still need to secure the source files on the host and your CI/CD pipelines that provide them.

Here is an example of a Compose file that defines and grants access to secrets:

```
services:
 myapp:
 image: myapp:latest
 secrets:
 - my_secret

secrets:
 my_secret:
 file: ./my_secret.txt
```

In this example, `myapp` service has access to `my_secret`, which is located at `/run/secrets/my_secret` in the container and is populated with the contents of `./my_secret.txt` from the host machine.

For a more complex setup with a database and an application like WordPress, you can have a `compose.yaml` file that looks like this:

```
services:
 db:
 image: mysql:latest
 environment:
 MYSQL_ROOT_PASSWORD_FILE: /run/secrets/db_root_password
 MYSQL_DATABASE: wordpress
 MYSQL_USER: wordpress
 MYSQL_PASSWORD_FILE: /run/secrets/db_password
 secrets:
 - db_root_password
 - db_password

 wordpress:
 depends_on:
 - db
 image: wordpress:latest
 environment:
 WORDPRESS_DB_HOST: db:3306
 WORDPRESS_DB_USER: wordpress
 WORDPRESS_DB_PASSWORD_FILE: /run/secrets/db_password
 secrets:
 - db_password

secrets:
 db_password:
 file: db_password.txt
 db_root_password:
 file: db_root_password.txt
```

In the above configuration, the `db` service uses the secrets for the root password and the WordPress database user password. Similarly, the `wordpress` service uses the secret for the database user password.